

#Development Version 3

#bde PlotlyJS working 1.ipynb

#bde PlotlyJS working 1a.ipynb

#bde PlotlyJS working 1.jl

using Plots

plotlyjs()

x=linspace(-2 π ,2 π ,1000)

y=sin.(3x+4cos.(2x))

x=x/ π

plot(x,y,

xlabel="k (x=k π)",

xlim=(-3,3),

ylabel="sin(3x+4cos(2x))\n\n",

yscale=:identity, #log,ln,log2,log10 also works

ylim=(-1.5,1.5),

linecolor=:red,

```
linewidth=:2.0,  
linestyle=:solid, #solid, dashdot, dot,dash
```

```
legend=:true,
```

```
label="sin(3x+4cos(2x))",
```

```
title="f(x) = sin(3x+4cos(2x)) \n\n",
```

```
titlecolor=:red,
```

```
#top_margin=:25,
```

```
#left_margin=:25,
```

```
#bottom_margin=:25,
```

```
gridstyle=:solid,
```

```
grid=:3,
```

```
gridalpha=:0.3,
```

```
foreground_color_legend=:magenta,
```

```
foreground_color_grid=:black;
```

```
foreground_color_axis=:blue,
```

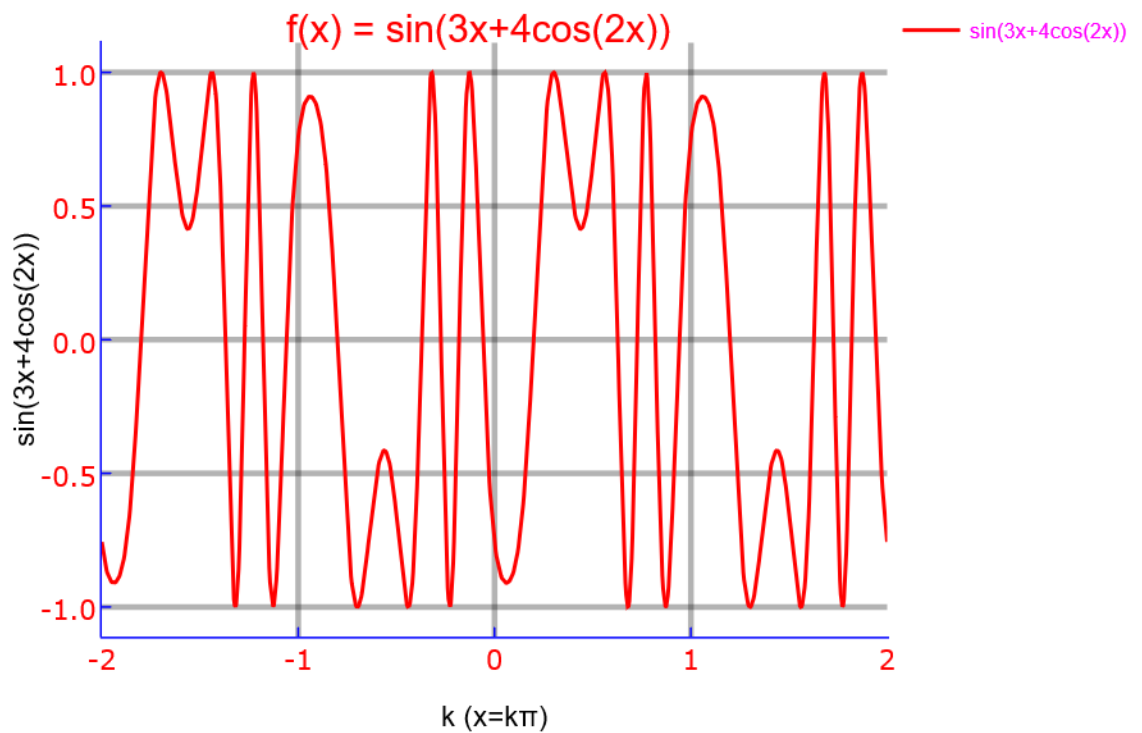
```
foreground_color_text=:red,
```

```
foreground_color_border=:blue,
```

```

xtickfont=font(10 , "verdana"),
ytickfont=font(10,"verdana"),
xticks=(-3:1:3),
yticks=(-1.5:0.5:2.5)
#aspectratio=1/0.5,  #better leave unassigned
)

```



#bde PlotlyJS working 1b.ipynb

```
#bde PlotlyJs working 1b.ipynb
```

```
using Plots
```

```
plotlyjs()
```

```
x=linspace(-2 $\pi$ ,2 $\pi$ ,1000);
```

```
y=sin.(3x+4cos.(2x))
```

```
plot(x,y,
```

```
  xlabel="x",
```

```
  xlim=(-2 $\pi$ ,2 $\pi$ ),
```

```
  ylabel="sin(3x+4cos(2x))\n\n",
```

```
 yscale=:identity, #log,ln,log2,log10 also works
```

```
  ylim=(-1.5,1.5),
```

```
  linecolor=:red,
```

```
  linewidth=:2.0,
```

```
  linestyle=:solid, #solid, dashdot, dot,dash
```

```
  legend=:true,
```

```
  label="sin(3x+4cos(2x))",
```

```
title="f(x) = sin(3x+4cos(2x)) \n",  
titlecolor=:red,
```

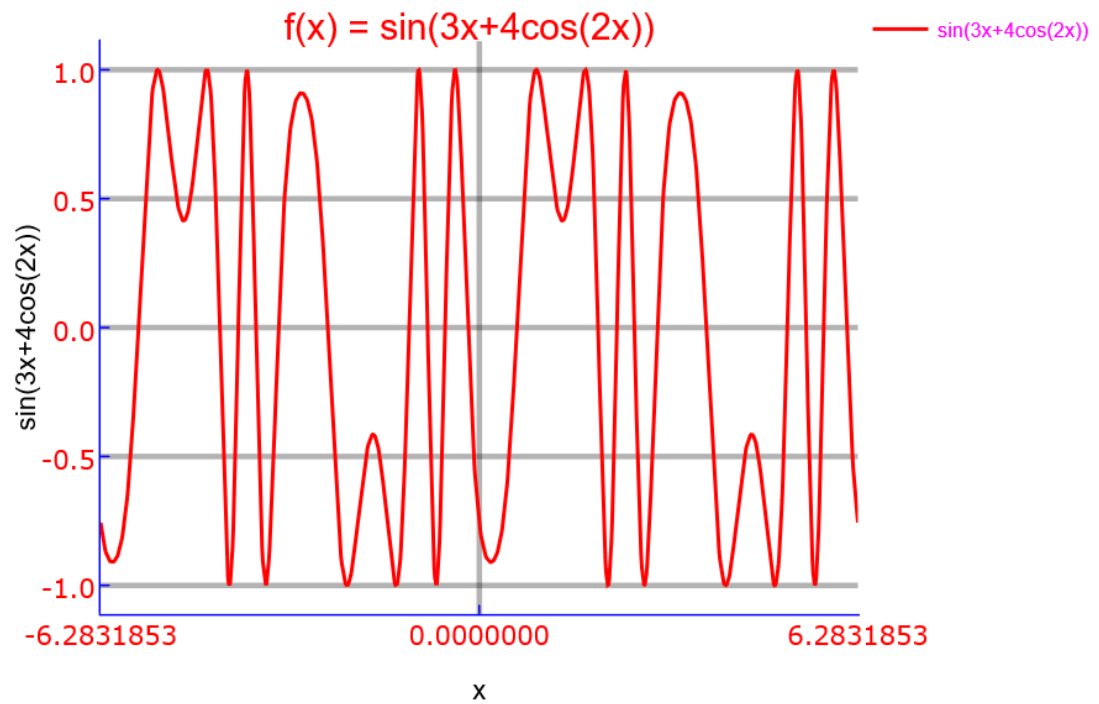
```
#top_margin=:25px,  
#left_margin=:25px,  
#bottom_margin=:25px,
```

```
gridstyle=:solid,  
grid=:3,  
gridalpha=:0.3,
```

```
foreground_color_legend=:magenta,  
foreground_color_grid=:black;  
foreground_color_axis=:blue,  
foreground_color_text=:red,  
foreground_color_border=:blue,
```

```
xtickfont=font(10, "verdana"),  
ytickfont=font(10, "verdana"),  
xticks=(-2 $\pi$ :2 $\pi$ :2 $\pi$ ),
```

```
yticks=(-1.5:0.5:2.5)
#aspectratio=1/0.5, better leave unassigned
)
```



#bde PlotlyJS working 1c.ipynb

#bde PlotlyJS working 1c.jl

using Plots

plotlyjs()

```

x=linspace(0,2π,1000);
y=sin.(3x+4cos.(2x))
plot(x/π,y,

xlabel="k (x=kπ)",
y="sin(3x+4cos(2x))",
xlim=(0,2),

ylabel="sin(3x+4cos(2x))\n\n",
yscale=:identity, #log,ln,log2,log10 also works
ylim=(-1.5,1.5),
linecolor=:red,
linewidth=:2.0,
linestyle=:solid, #solid, dashdot, dot,dash

legend=:true,

label="sin(3x+4cos(2x))",

title="f(x) = sin(3x+4cos(2x)) \n",
titlecolor=:red,

#top_margin=:25,
#left_margin=:25,

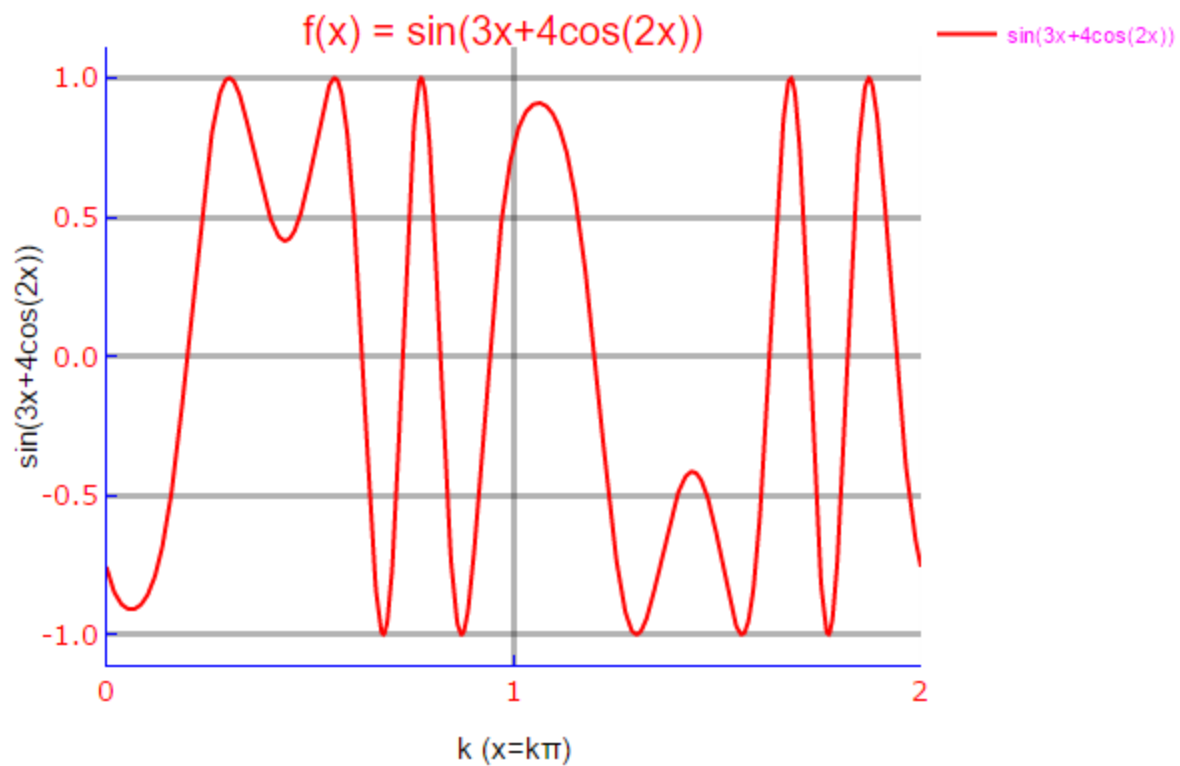
```

```
#bottom_margin=:25,

gridstyle=:solid,
grid=:3,
gridalpha=:0.3,

foreground_color_legend=:magenta,
foreground_color_grid=:black;
foreground_color_axis=:blue,
foreground_color_text=:red,
foreground_color_border=:blue,

xtickfont=font(10, "verdana"),
ytickfont=font(10, "verdana"),
xticks=(0:1:2),
yticks=(-1.5:0.5:2.5)
#aspectratio=1/0.5,  better leave unassigned
)
```

#bde PlotlyJs working 1d.jl

```
using Plots
plotlyjs()
```

```
x=linspace(0,2pi,1000);
y=sin.(3x+4cos.(2x))
```

```
plot(x,y,
```

```
  xlabel="x",
  y="sin.(3x+4cos(2x))",
  xlim=(0,2pi),
```

```
  ylabel="sin(3x+4cos(2x))\n\n",
 yscale=:identity, #log,ln,log2,log10 also works
```

```
ylim=(-1.5,1.5),

linecolor=:red,
linewidth=:2.0,
linestyle=:solid,  #solid, dashdot, dot,dash

legend=:true,

label="sin(3x+4cos(2x))",

title="f(x) = sin(3x+4cos(2x)) \n",
titlecolor=:red,

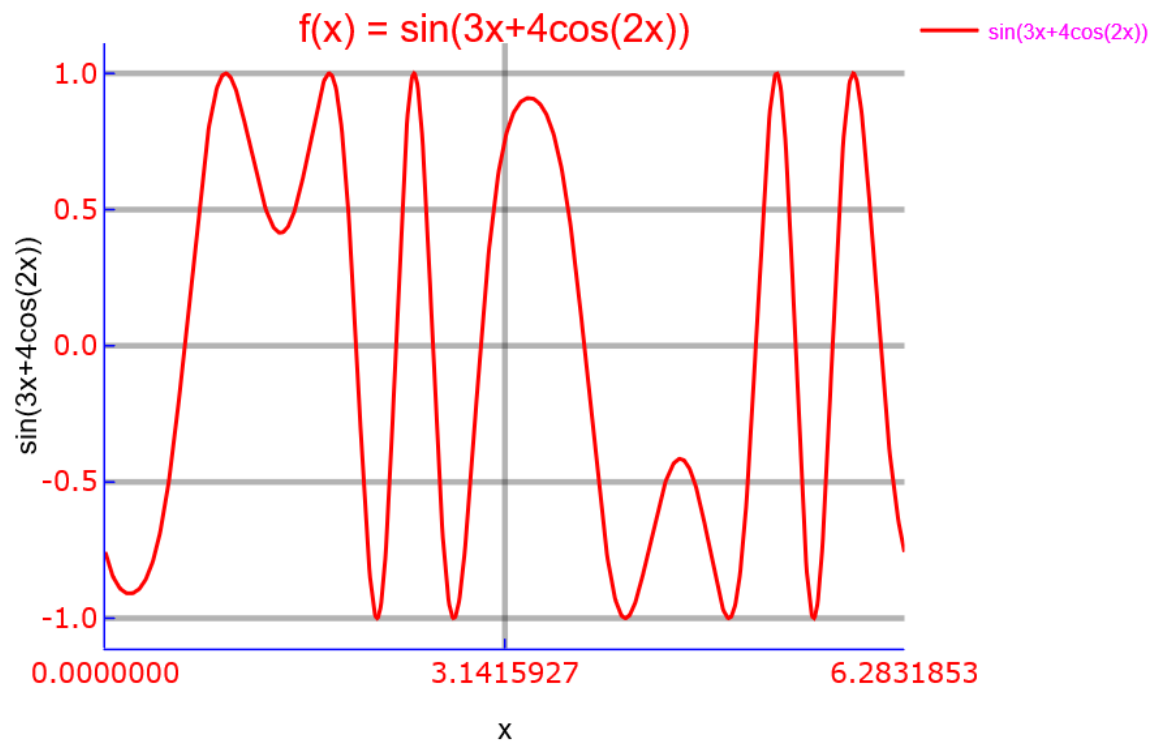
#top_margin=:25px,
#left_margin=:25px,
#bottom_margin=:25px,

gridstyle=:solid,
grid=:3,
gridalpha=:0.3,

foreground_color_legend=:magenta,
foreground_color_grid=:black;
foreground_color_axis=:blue,
foreground_color_text=:red,
foreground_color_border=:blue,

xtickfont=font(10, "verdana"),
ytickfont=font(10,"verdana"),

xticks=(-2pi:pi:2pi),
yticks=(-1.5:0.5:1.5) )
```



bde plotlyJS working 2

#bde PlotlyJS working 2a.ipynb

using Plots

plotlyjs()

a=[1,2,3]

b=a

plot(a,b,

```
title="Recursive Relation Between Elements of the set {a}, a={1,2,3}",  
titlecolor=:red,
```

```
label="Recursion",
```

```
xlabel="a\n",
```

```
ylabel="a\n",
```

```
yscale=:identity, #log,ln,log2,log10 also works
```

```
ylims=:autoscale,
```

```
line=:0,
```

```
legend=:false,
```

```
label="Recursion",
```

```
top_margin=:50px,
```

```
left_margin=:25px,
```

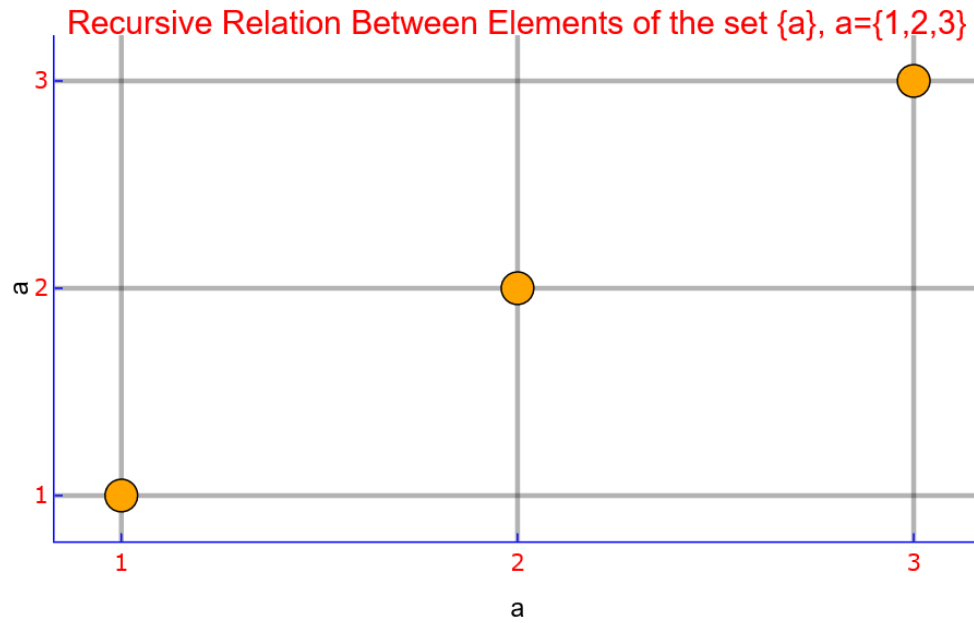
```
bottom_margin=:50px,
```

```
gridstyle=:solid,
```

```
grid=:3,
```

```
gridalpha=:0.3,
```

```
markershape=:circle ,  
markercolor=:orange,  
markersize=:10,  
  
foreground_color_legend=:magenta,  
foreground_color_grid=:black;  
foreground_color_axis=:blue,  
foreground_color_text=:red,  
foreground_color_border=:blue,  
  
xtickfont= font(10 , "verdana"),  
ytickfont= font(10 , "verdana"),  
xlims=(0,4),  
xticks=(0:1:4),  
ylims=(0,4),  
yticks=(0:1:4),  
)
```



#bde PlotlyJS working 2b.ipynb

using Plots

plotlyjs()

a=[1,2,3]

b=a

plot(a,b,

title="Recursive Relation Between Elements of the set $\{a\}$, $a=\{1,2,3\}$ ",

titlecolor=:red,

label="Recursion",

xlabel="a\n",

ylabel="a\n",

yscale=:identity, #log,ln,log2,log10 also works

ylims=:autoscale,

line=:0,

legend=:false,

label="Recursion",

#top_margin=:50px,

#left_margin=:25px,

#bottom_margin=:50px,

gridstyle=:solid,

grid=:3,

gridalpha=:0.3,

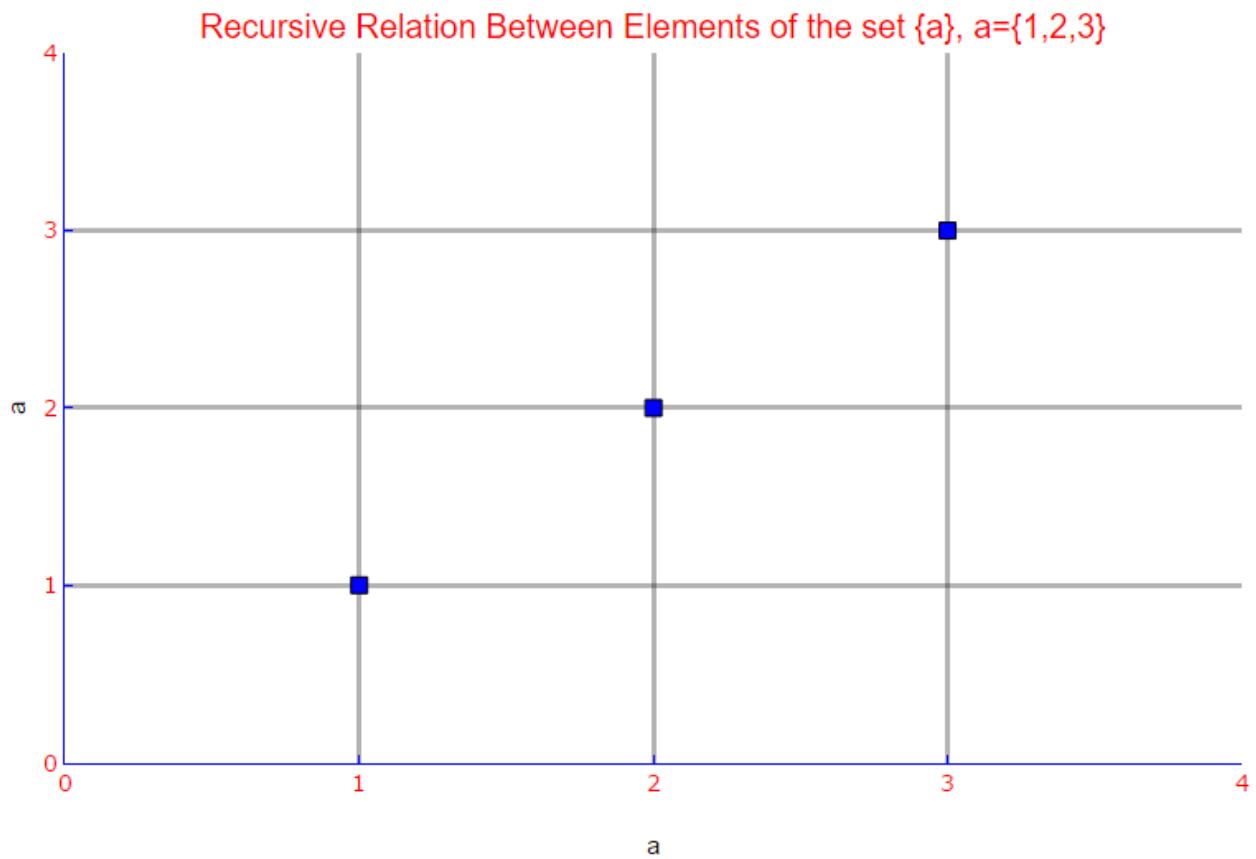
markershape=:square ,

markercolor=:blue,

markersize=:5,

```
foreground_color_legend=:magenta,  
foreground_color_grid=:black;  
foreground_color_axis=:blue,  
foreground_color_text=:red,  
foreground_color_border=:blue,
```

```
xtickfont= font(10 , "verdana"),  
ytickfont= font(10 , "verdana"),  
xlims=(0,4),  
xticks=(0:1:4),  
ylims=(0,4),  
yticks=(0:1:4),  
)
```

bde plotlyJS working 3

#bde PlotlyJS working 3a.ipynb

using Plots

plotlyjs()

```
x=linspace(-2π,2π,1000);
```

```
y=sin.(x)
```

```
plot(x/π,y,
```

```
xlabel="x/π",
```

```
xlim=(-3,3),
```

```
ylabel="sin(x)\n\n",
```

```
yscale=:identity, #log,ln,log2,log10 also works
```

```
ylim=(-1.5,1.5),
```

```
linecolor=:red,
```

```
linewidth=:2.0,
```

```
linestyle=:solid, #solid, dashdot, dot,dash
```

```
legend=:true,
```

```
label="sin(x)",
```

```
title="f(x) = sin(x) \n",
```

```
titlecolor=:red,
```

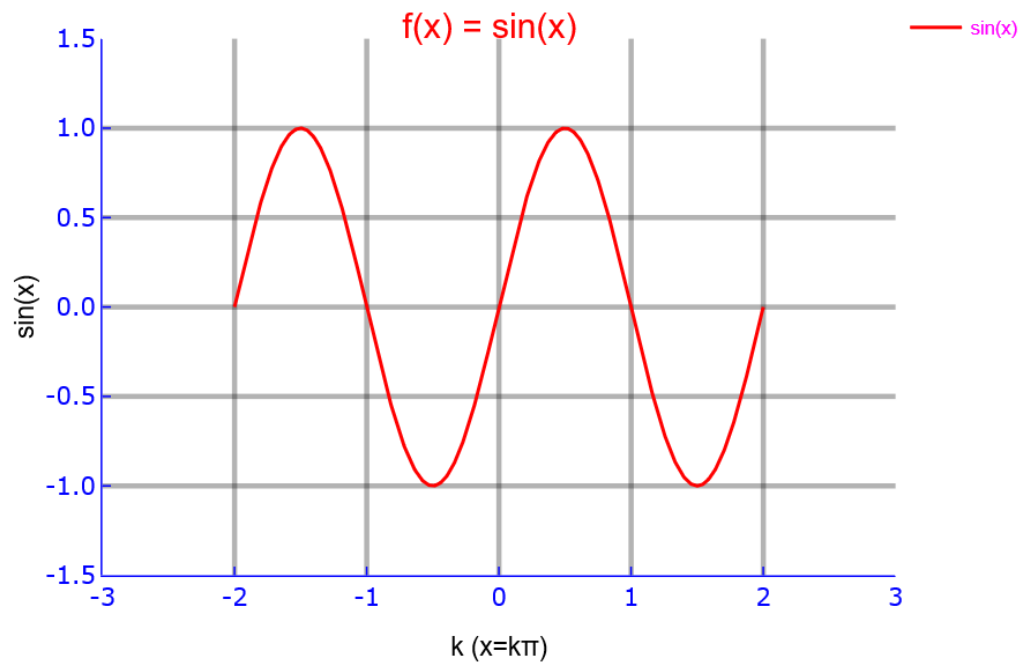
```
top_margin=:25px,
```

```
left_margin=:25px,  
bottom_margin=:25px,
```

```
gridstyle=:solid,  
grid=:3,  
gridalpha=:0.3,
```

```
foreground_color_legend=:magenta,  
foreground_color_grid=:black;  
foreground_color_axis=:blue,  
foreground_color_text=:blue,  
foreground_color_border=:blue,
```

```
xtickfont=font(10, "verdana"),  
ytickfont=font(10,"verdana"),  
xticks=(-3:1:3),  
yticks=(-1.5:0.5:2.5)  
#aspectratio=1/0.5  better leave unassigned  
)
```



#bde PlotlyJS working 3b.ipynb

using Plots

plotlyjs()

```
x=linspace(-2π,2π,1000);
```

```
y=sin.(x)
```

```
plot(x,y,
```

```
xlabel="x",
```

```
ylabel="sin(x)\n\n",
```

```
xlim=(-2 $\pi$ ,2 $\pi$ ),
```

```
ylim=(-1.5,1.5),
```

```
yscale=:identity, #log,ln,log2,log10 also works
```

```
linecolor=:blue,
```

```
linewidth=:2.0,
```

```
linestyle=:dashdot, #solid, dashdot, dot,dash
```

```
legend=:true,
```

```
label="sin(x)",
```

```
title="f(x) = sin(x) \n",
```

```
titlecolor=:red,
```

```
gridstyle=:solid,
```

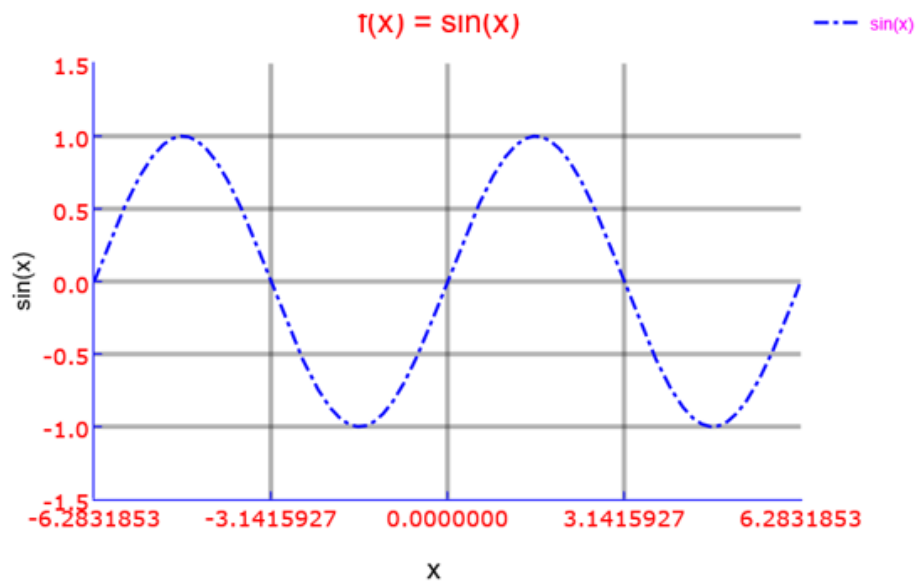
```
grid=:3,
```

```
gridalpha=:0.3,
```

```
foreground_color_legend=:magenta,
```

```
foreground_color_grid=:black;
```

```
foreground_color_axis=:blue,  
foreground_color_text=:red,  
foreground_color_border=:blue,  
  
xtickfont=font(10, "verdana"),  
ytickfont=font(10,"verdana"),  
  
xticks=(-2 $\pi$ : $\pi$ :2 $\pi$ ),  
yticks=(-1.5:0.5:2.5)  
  
#aspectratio=1/0.5  (better leave unassigned)  
)
```



#bde PlotlyJS working 3c.ipynb

using Plots

plotlyjs()

```
x=linspace(-4,4,20)
```

```
y=cos.(x)
```

```
plot(x,y,
```

```
line=:false,
```

```
xlabel="x",
ylabel="cos(x)",
grid=true,
gridcolor=:red,
grid=3,
gridalpha=0.3,
title="cos(x)",
label="cos(x)",
markershape=:circle ,
markercolor=:orange,
markersize=:10,
titlecolor=:red,
line=:0,
legend=:true,

foreground_color_legend=:magenta,
foreground_color_grid=:black;
foreground_color_axis=:blue,
foreground_color_text=:red,
foreground_color_border=:blue,

xtickfont= font(16 , "verdana"),
ytickfont= font(16 , "verdana"),
```



```
xlims=(-4,4),
```

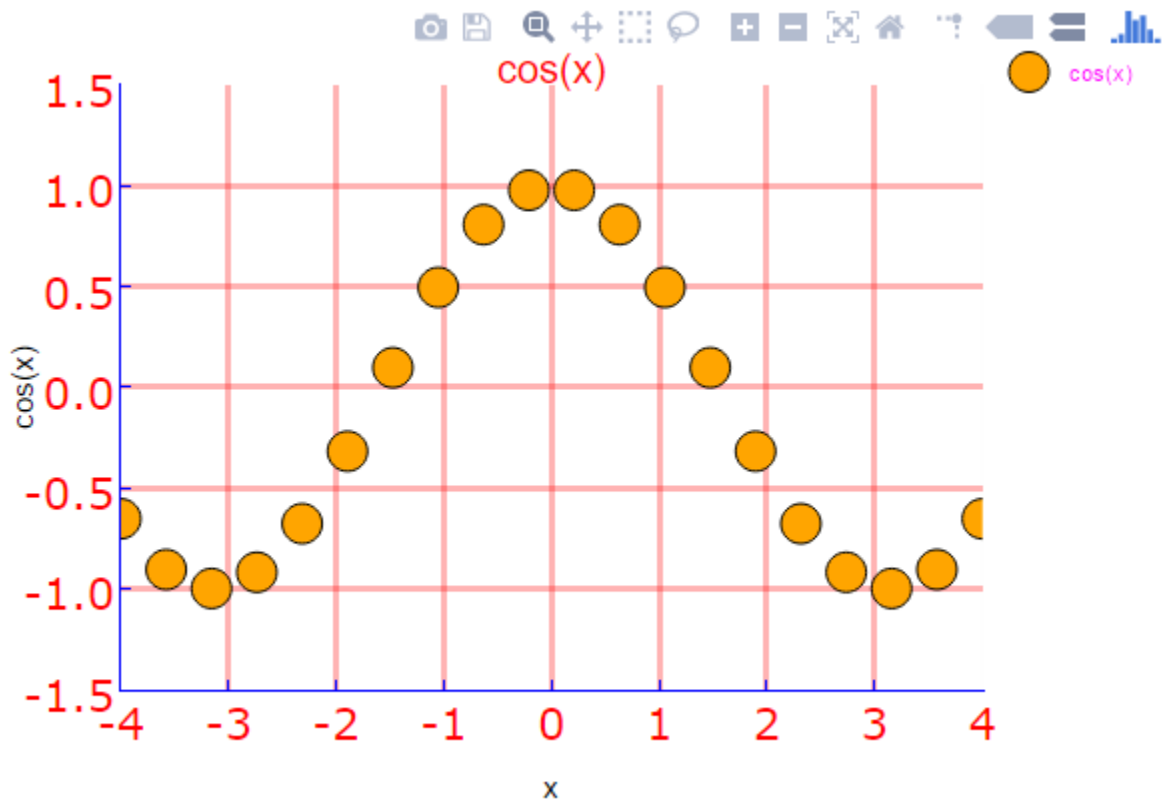
```
ylims=(-1.5,1.5),
```

```
xticks=(-4:1:4),
```

```
yticks=(-1.5:0.5:1.5),
```

```
#aspectratio=1/0.5, (better leave unassigned)
```

```
)
```



#bde PlotlyJS working 3d.ipynb

```
using Plots
```

```
plotlyjs()
```

```
x=linspace(-2 $\pi$ ,2 $\pi$ ,1000);
```

```
y1=sin.(x)
```

```
y2=cos.(x)
```

```
plot(x/ $\pi$ ,[y1,y2],
```

```
xlabel=:" k (x=k $\pi$ )",
```

```
xlim=(-3,3),
```

```
ylabel="sin(x) , cos(x)\n\n",
```

```
yscale=:identity, #log,ln,log2,log10 also works
```

```
ylim=(-1.5,1.5),
```

```
linecolor=:auto,
```

```
linewidth=:2.0,
```

```
linestyle=:solid, #solid, dashdot, dot,dash
```

```
legend=:true,
```

```
label=["sin(x)" "cos(x)"],
```

```
title="f1(x) = sin(x) , f2(x) = cos(x) \n",
```

```
titlecolor=:red,
```

```
gridstyle=:solid,
```

```
grid=:3,
```

```
gridalpha=:0.3,
```

```
foreground_color_legend=:magenta,
```

```
foreground_color_grid=:black;
```

```
foreground_color_axis=:blue,
```

```
foreground_color_text=:blue,
```

```
foreground_color_border=:blue,
```

```
xtickfont=font(10, "verdana"),
```

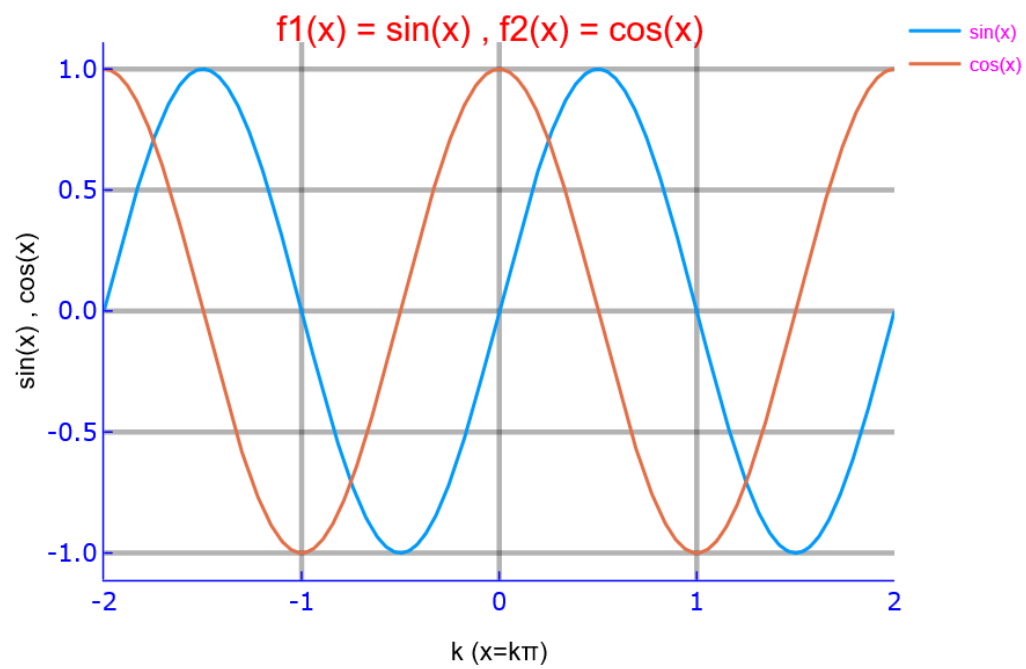
```
ytickfont=font(10,"verdana"),
```

```
xticks=(-3:1:3),
```

```
yticks=(-1.5:0.5:2.5)
```

```
#aspectratio=1/0.5  better leave unassigned
```

)



#bde PlotlyJS working 3e.ipynb

using Plots

plotlyjs()

```
x=linspace(-2π,2π,1000);
```

```
y1=sin.(x)
```

```
y2=cos.(x)
```

```
plot(x,[y1,y2],
```

```
xlabel="x",
ylabel="sin(x) , cos(x)\n\n",

yscale=:identity, #log,ln,log2,log10 also works

xlim=(-2.5π,2.5π),
ylim=(-1.5,1.5),

linecolor=:auto,
linewidth=:2.0,
linestyle=:solid, #solid, dashdot, dot,dash

legend=:true,

label=["sin(x)" "cos(x)"],

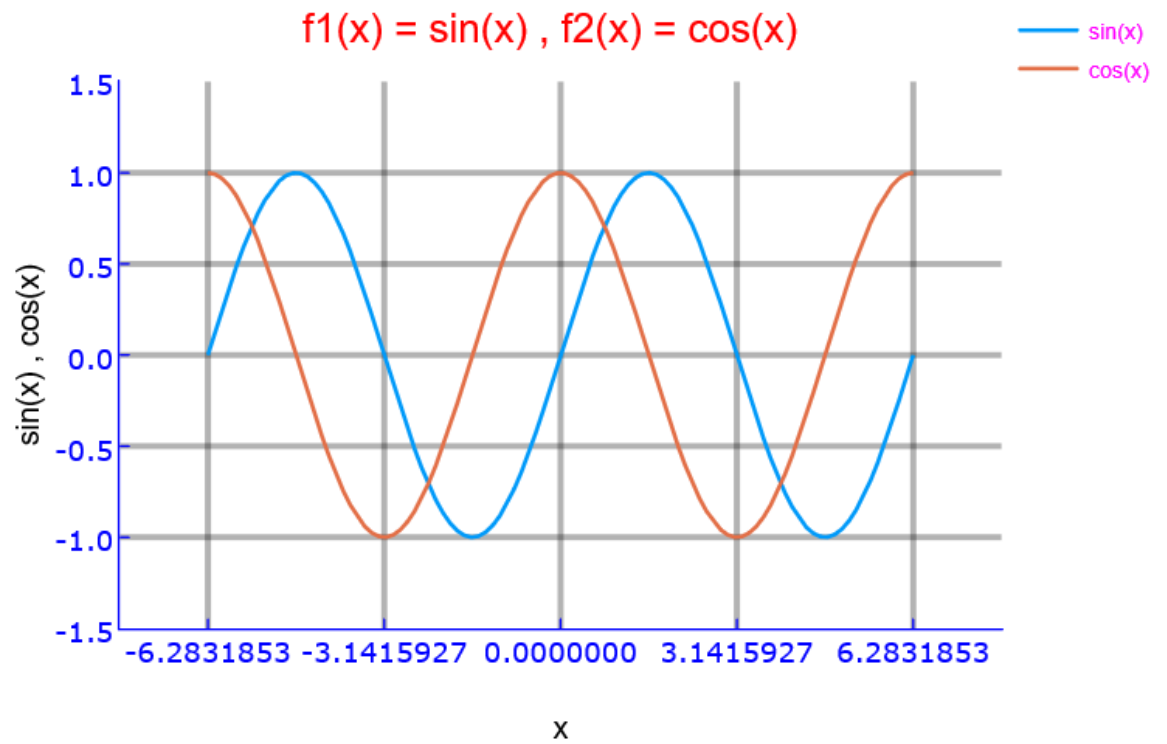
title="f1(x) = sin(x) , f2(x) = cos(x) \n",
titlecolor=:red,

gridstyle=:solid,
grid=:3,
gridalpha=:0.3,
```

```
foreground_color_legend=:magenta,  
foreground_color_grid=:black;  
foreground_color_axis=:blue,  
foreground_color_text=:blue,  
foreground_color_border=:blue,
```

```
xtickfont=font(10, "verdana"),  
ytickfont=font(10, "verdana"),
```

```
xticks=(-2 $\pi$ :(2//2) $\pi$ :2 $\pi$ ),  
yticks=(-1.5:0.5:2.5)  
#aspectratio=1/0.5  better leave unassigned  
)
```



bde plotlyJS working 4

#bde PlotlyJS working 4a.ipynb

using Plots

plotlyjs()

```
x=linspace(-2π,2π,100);
```

```
y1=sin.(x);
```

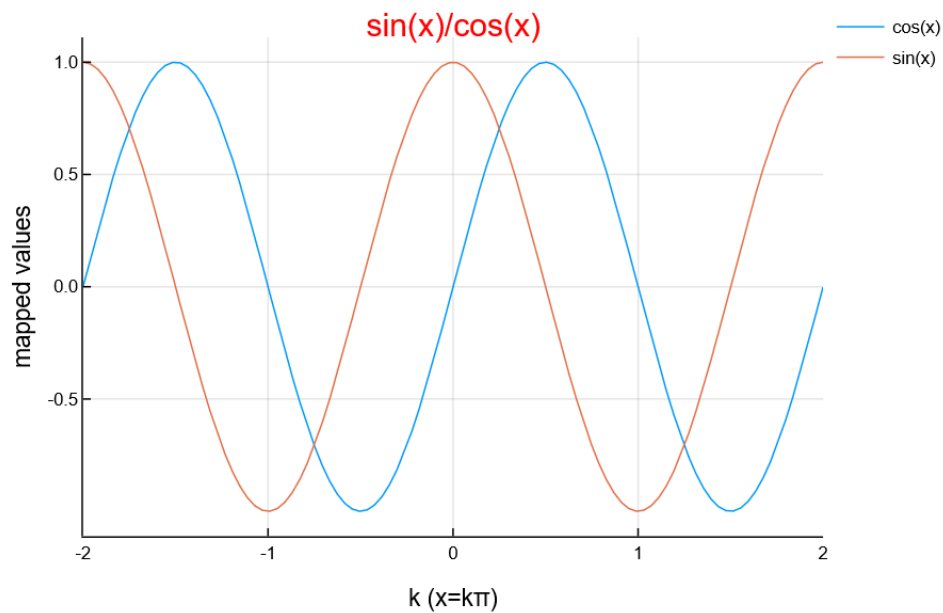
```
y2=cos.(x);
```

```
y3=tan.(x);
```

```

plot(x/pi,[y1,y2],
     line=(:path),
     label=["cos(x)" "sin(x)"],
     xlabel="k (x = kπ )",
     ylabel="mapped values\n",
     xticks=(minimum(x)/pi:1:maximum(x)/pi),
     title="sin(x)/cos(x)",
     titlecolor=:red,
)

```



#bde PlotlyJS working 4b.ipynb

using Plots


```
plotlyjs()
```

```
x=linspace(-2 $\pi$ ,2 $\pi$ ,100);
```

```
y1=sin.(x);
```

```
y2=cos.(x);
```

```
plot(x,[y1,y2],
```

```
    line=(:path),
```

```
    label=["cos(x)" "sin(x)"],
```

```
    xlabel="x values",
```

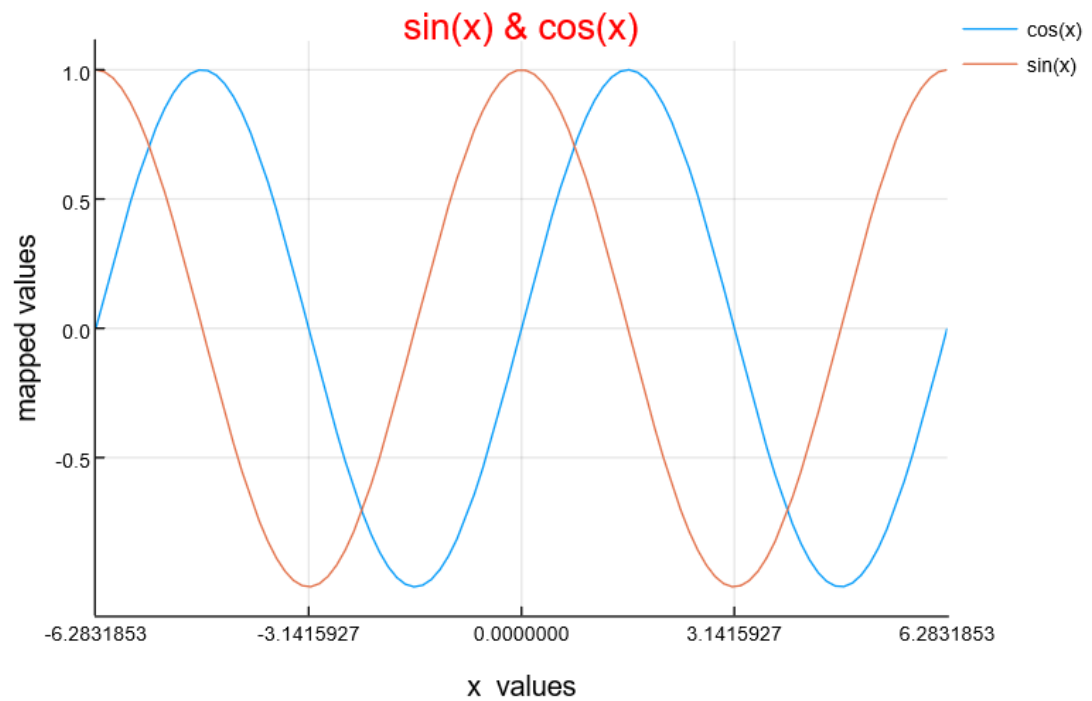
```
    ylabel="mapped values\n",
```

```
    xticks=(minimum(x): $\pi$ :maximum(x)),
```

```
    title="sin(x) & cos(x)",
```

```
    titlecolor=:red,
```

```
)
```



#bde PlotlyJS working 4c.jl

using Plots

plotlyjs()

x=linspace(-2π,2π,100);

y1=sin.(x);

y2=cos.(x);

y3=tan.(x);

plot(x/π,[y1,y2],

xlabel="k (x = kπ)",

xlim=(-2,2),

```
ylabel="sin(x) , cos(x)\n\n",  
yscale=:identity, #log,ln,log2,log10 also works  
ylim=(-1.5,1.5),
```

```
linecolor=:auto,  
linewidth=:2.0,  
linestyle=:solid, #solid, dashdot, dot,dash
```

```
legend=:true,
```

```
label=["sin(x)" "cos(x)"],
```

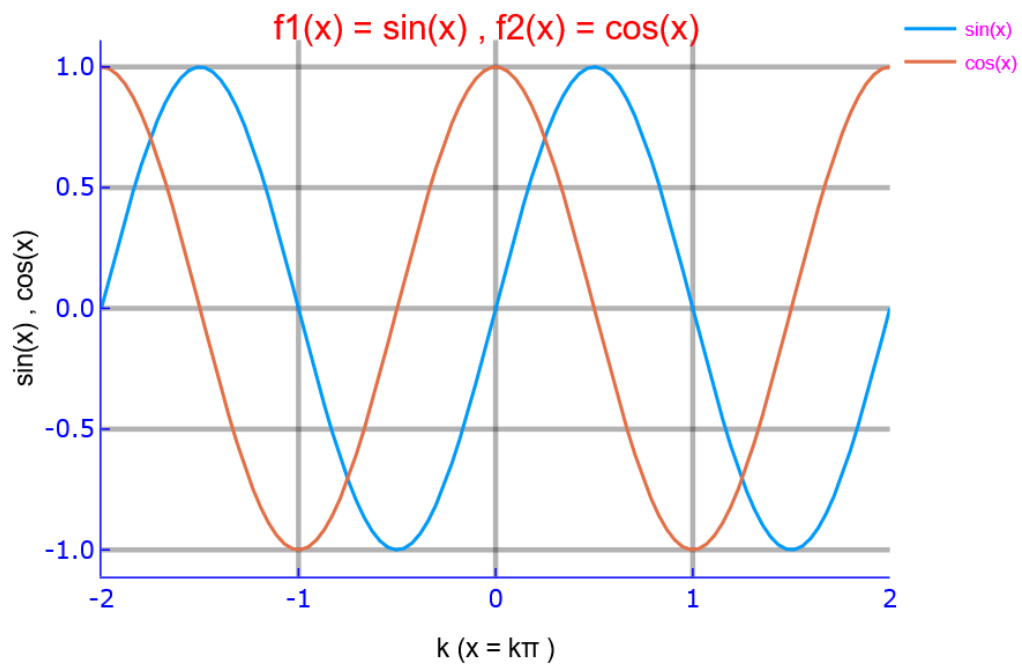
```
title="f1(x) = sin(x) , f2(x) = cos(x) \n",  
titlecolor=:red,
```

```
gridstyle=:solid,  
grid=:3,  
gridalpha=:0.3,
```

```
foreground_color_legend=:magenta,  
foreground_color_grid=:black;
```

```
foreground_color_axis=:blue,  
foreground_color_text=:blue,  
foreground_color_border=:blue,
```

```
xtickfont=font(10, "verdana"),  
ytickfont=font(10,"verdana"),  
xticks=(-2:1:2),  
yticks=(-1.5:0.5:2.5)  
#aspectratio=1/0.5, #better leave unassigned  
)
```



#bde PlotlyJS working 4d.jl

using Plots

plotlyjs()

x=linspace(-2 π ,2 π ,100)

y1=sin.(x)

y2=cos.(x)

plot(x,[y1,y2],

xlabel="x",

xlim=(-2.5 π ,2.5 π),

ylabel="sin(x) , cos(x)\n\n",

yscale=:identity, #log,ln,log2,log10 also works

ylim=(-1.5,1.5),

linecolor=:auto,

linewidth=:2.0,

linestyle=:solid, #solid, dashdot, dot,dash

legend=:true,

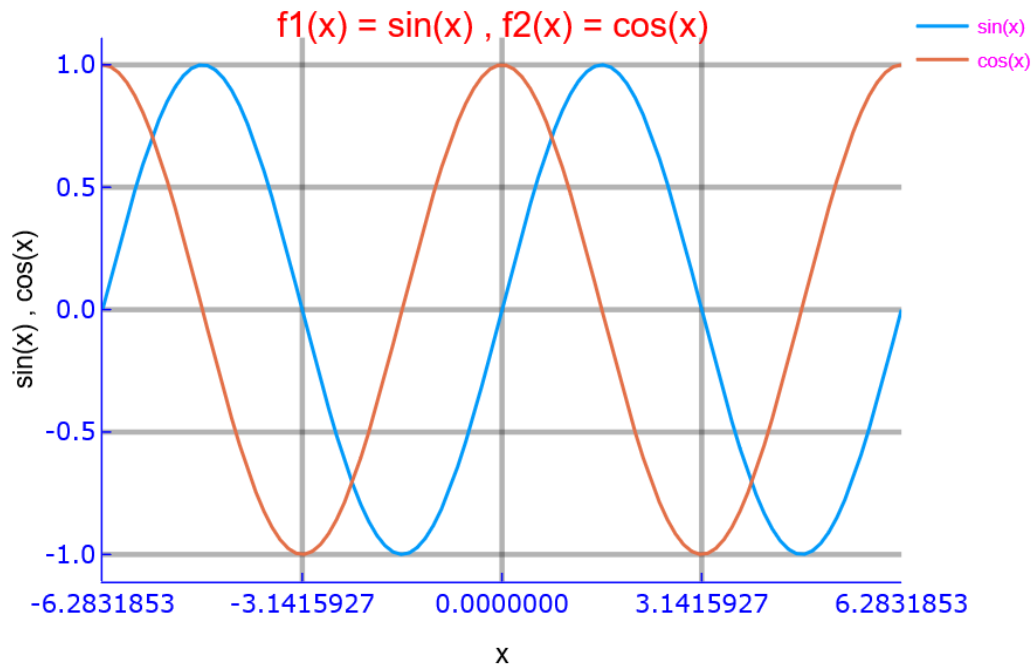
label=["sin(x)" "cos(x)"],

```
title="f1(x) = sin(x) , f2(x) = cos(x) \n",
titlecolor=:red,

gridstyle=:solid,
grid=:3,
gridalpha=:0.3,

foreground_color_legend=:magenta,
foreground_color_grid=:black;
foreground_color_axis=:blue,
foreground_color_text=:blue,
foreground_color_border=:blue,

xtickfont=font(10 , "verdana"),
ytickfont=font(10,"verdana"),
xticks=(-2 $\pi$ :(2//2) $\pi$ :2 $\pi$ ),
yticks=(-1.5:0.5:2.5)
#aspectratio=1/0.5,  #better leave unassigned
)
```



#bde PlotlyJS working 4e.jl

using Plots

plotlyjs()

$x = \text{linspace}(-2\pi, 2\pi, 100);$

$y_1 = \sin.(x);$

$y_2 = \cos.(x);$

$y_3 = \tan.(x);$

$\text{plot}(x/\pi, [y_1, y_2, y_3],$

$\text{title} = " \sin(x) , \cos(x) , \tan(x) / \pi \setminus n ",$

titlecolor=:red,

grid=:1,

gridstyle=:solid,

gridalpha=:0.3,

foreground_color_grid=:black,

line=:true,

#linecolor=:red,

linewidth=:1.0,

linestyle=:auto, #auto,solid,dot,dash,dashdot

xlabel="k ($x = k\pi$)",

ylabel="sin(x) / cos(x) / tg(x)",

legend=:true,

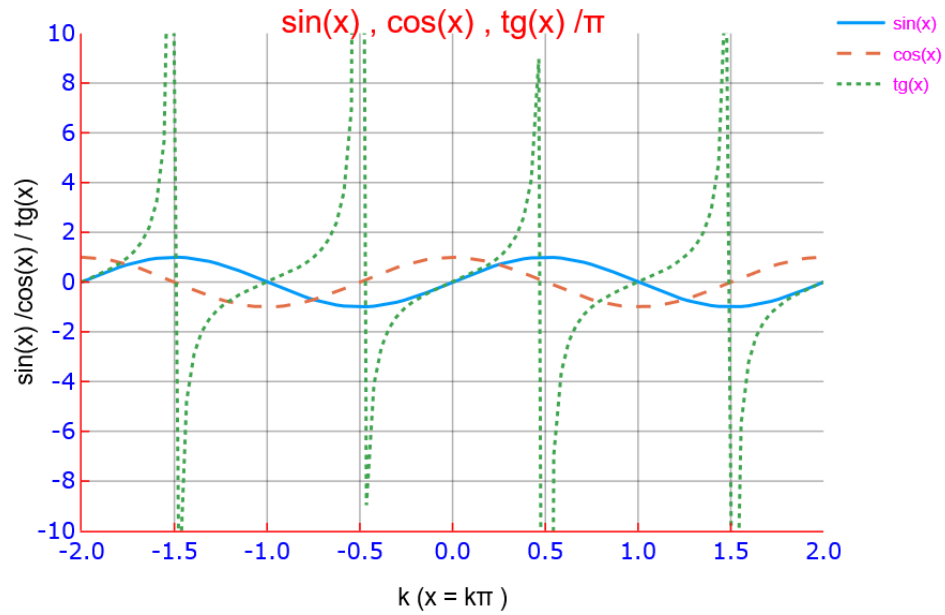
legendtitle="",

label=["sin(x)" "cos(x)" "tg(x)"],

foreground_color_legend=:magenta,

yscale=:identity, #log,ln,log2,log10 also works


```
foreground_color_axis=:red,  
foreground_color_text=:blue,  
foreground_color_border=:black,  
  
xtickfont=font(10 , "verdana"),  
ytickfont=font(10 , "verdana"),  
  
xlim=(-2,2),  
ylim=(-10,10),  
  
xticks=(-2:0.5:2),  
yticks=(-10:2:10)  
#aspectratio=1/0.5  better leave unassigned)  
  
)
```



#bde PlotlyJS working 4f.ipynb

using Plots

plotlyjs()

```
x=linspace(-2π,2π,100);
```

```
y1=sin.(x);
```

```
y2=cos.(x);
```

```
y3=tan.(x);
```

```
plot(x,[y1,y2,y3],
```

```
top_margin=:25px,
```

```
left_margin=:25px,  
bottom_margin=:25px,  
  
title="sin(x) , cos(x) , tg(x) / $\pi$  \n",  
titlecolor=:red,  
  
grid=:1,  
gridstyle=:solid,  
gridalpha=:0.3,  
foreground_color_grid=:black,  
  
line=:true,  
#linecolor=:red,  
linewidth=:1.0,  
linestyle=:auto, #auto,solid,dot,dash,dashdot  
  
xlabel =:"x",  
ylabel="sin(x) /cos(x) / tg(x)",  
  
legend=:true,  
label=["sin(x)" "cos(x) " "tg(x)"],  
legendtitle="",  
foreground_color_legend=:magenta,
```

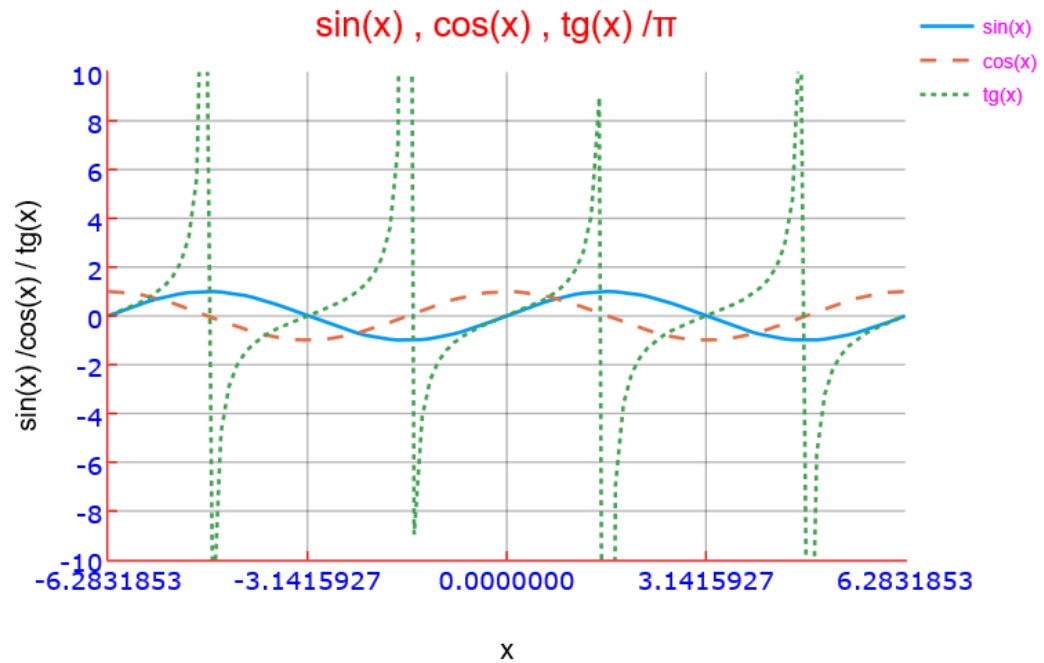
yscale=:identity, #log,ln,log2,log10 also works

foreground_color_axis=:red,
foreground_color_text=:blue,
foreground_color_border=:black,

xtickfont=font(10, "verdana"),
ytickfont=font(10, "verdana"),

xlim=(minimum(x),maximum(x)),
ylim=(-10,10),

xticks=(-2 π : π :2 π),
yticks=(-10:2:10)
#aspectratio=1/0.5 #better leave unassigned)
)



bde PlotlyJS working 5

#bde PlotlyJS working 5a.ipynb

using Plots

plotlyjs()

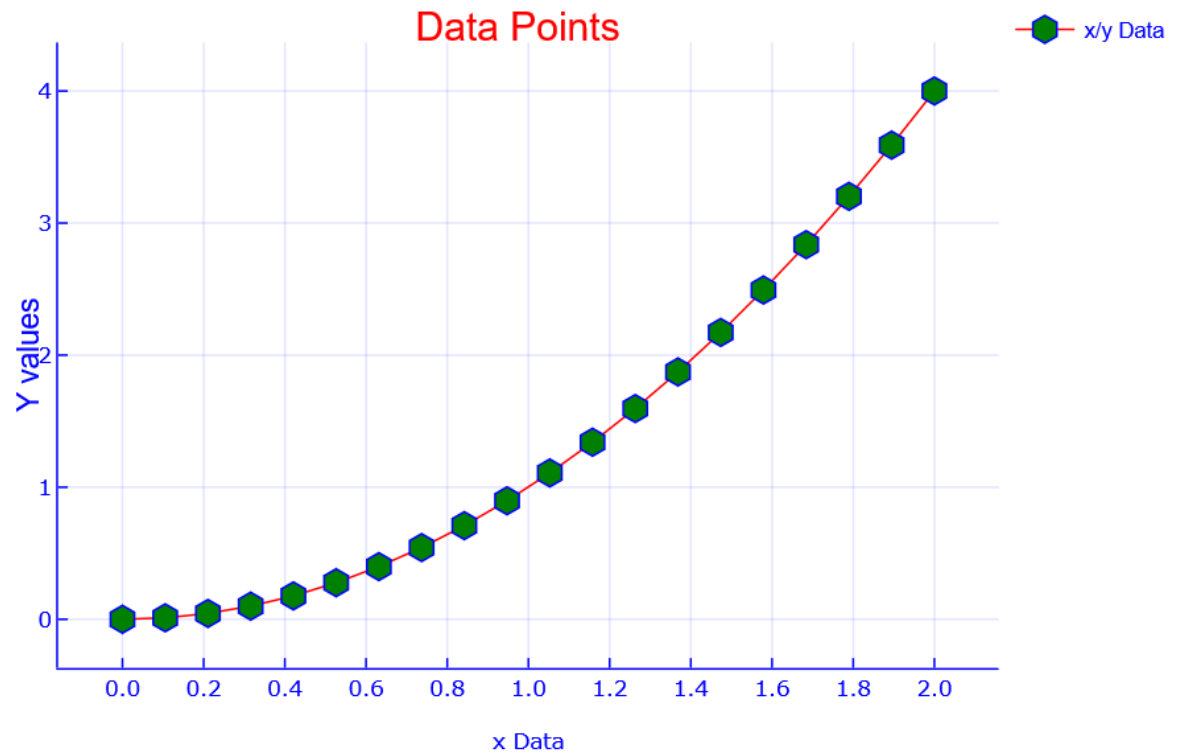
$x = \text{linspace}(0, 2, 20)$

$y = x.^2$

plot(x, y,

 markershape=:hexagon,

```
markersize=7,  
markercolor=:green,  
label="x/y Data",  
xaxis=(" \n x Data",(0,2), 0:0.2:2,:identity , font(8,"Verdana")), #flip (after  
identity), log,ln,log2,log10 also works in place of identity  
linestyle=:solid, #solid (default), dot, dash,dashdot  
linecolor=:red,  
title="Data Points \n",  
titlecolor=:red,  
foreground=:blue,#black (default)  
grid=:true, # false, otherwise  
#aspectratio=:1/20, #1/1 ,2/1 , equal leave default  
# alternative to tuple  
ylabel="Y values\n",  
ylims=(0,5),  
yticks=(0:1:5),  
yscale=:identity, #log,ln,log2,log10 also works  
ytickfont= font(8 , "verdana")  
)
```



#bde PlotlyJS working 5b.ipynb

using Plots

plotlyjs()

x=linspace(0,2,20)

y=x.^2

plot([x y],

note : this is x/x and x/y graphs you may add many more as z/x , w/x etc...

markershape=[:circle :rect], # valid shapes

[:none,:auto,:circle,:rect,star4,:star5,:star6,:star7,:diamond,:hexagon,:cross,:xcross,
s,:

#:utriangle,:dtriangle,:rtriangle,:ltriangle,:pentagon,:heptagon,:hexagon,:octagon,
:vline,:hline,:+,:x,:pixel]

markercolors=[:red :blue],

labels=["x Data" "x^2 Data"],

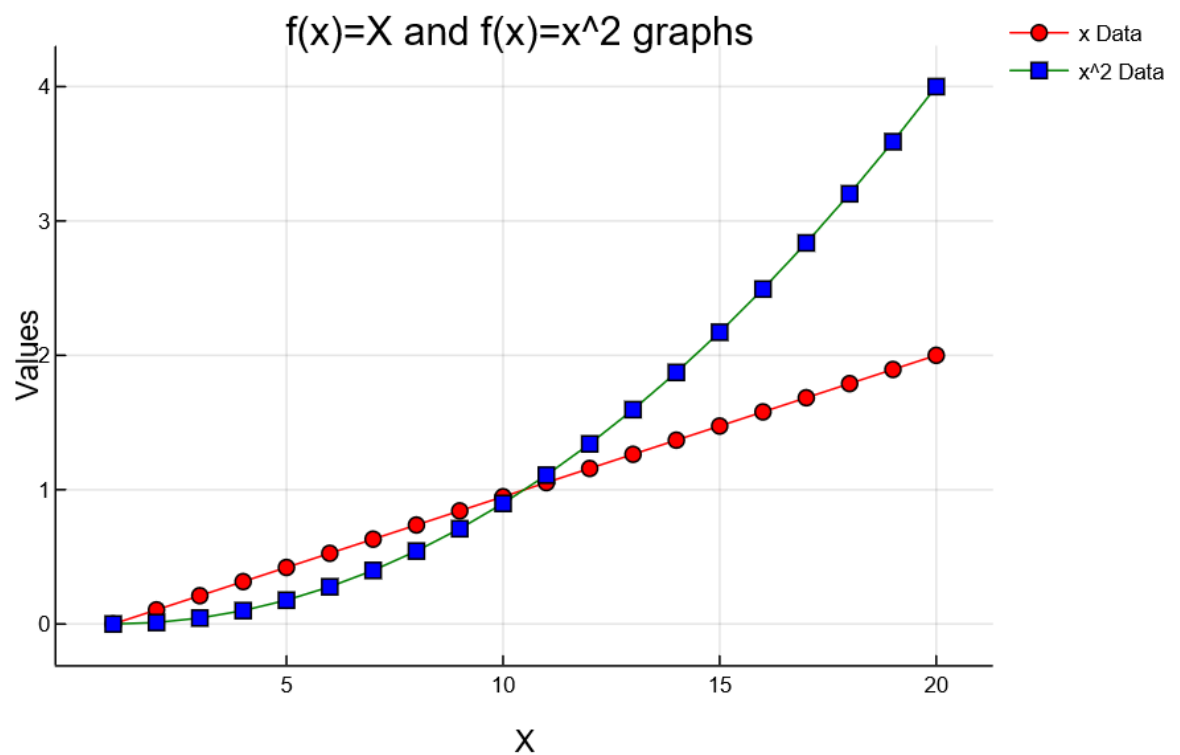
title = "f(x)=X and f(x)=x^2 graphs\n",

ylabel = "Values\n",

xlabel = "X",

linecolors=[:red :green]

)



#bde PlotlyJS working most simpler 5c.jl

using Plots

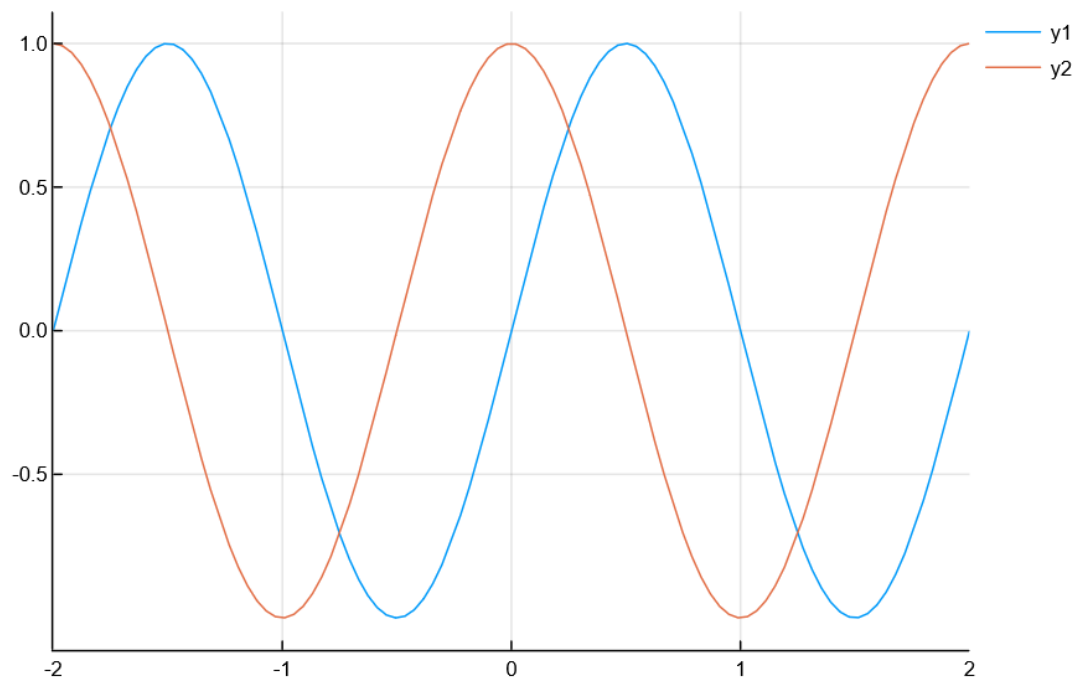
plotlyjs()

x=linspace(-2pi,2pi,100);

y1=sin.(x);

y2=cos.(x);

plot(x/pi,[y1,y2])

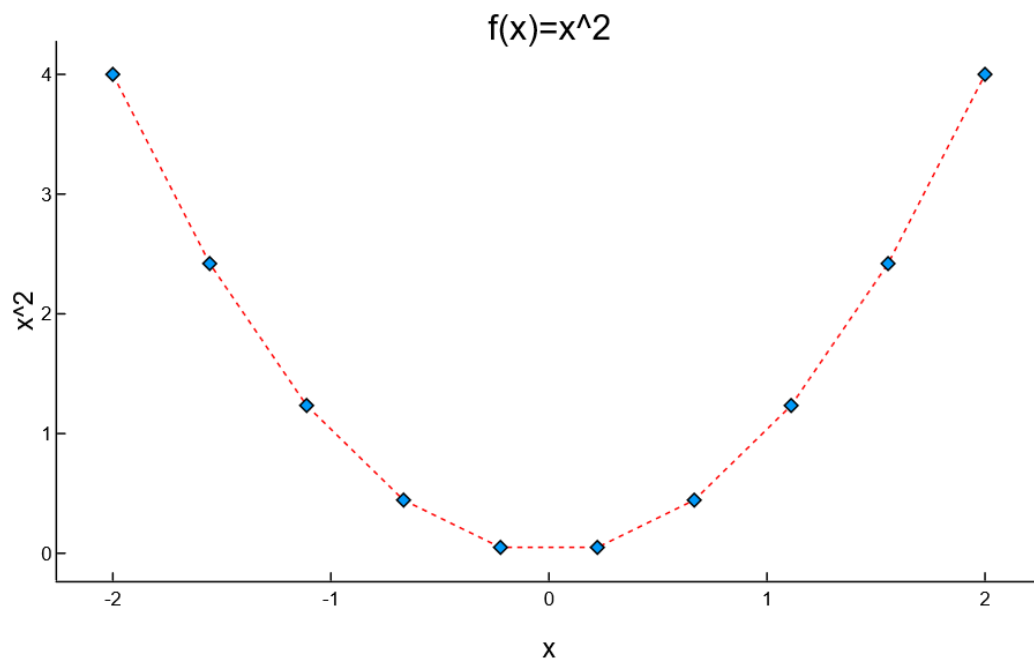


#bde PlotlyJS working most compact 5d.jl

using Plots

plotlyjs()

```
x=linspace(-2, 2, 10)
y=x.^2
plot(x,y,
title="f(x)=x^2",
xlabel="x",
ylabel="x^2",
marker=(:'diamond',3),
line=(:'path','dot','red'),
legend=false,
grid=false)
```



#bde Pyplot working 1

#bde Pyplot working 1a.ipynb

using Plots

pyplot()

$x = \text{linspace}(-2\pi, 2\pi, 1000);$

$y = \sin.(3x + 4\cos.(2x))$

$\text{plot}(x/\pi, y,$

$\text{xlabel} = "k \ (x = k\pi)",$

$\text{ylabel} = " \sin(3x + 4\cos(2x)) \backslash n \backslash n",$

$\text{xlim} = (-3, 3),$

$\text{ylim} = (-1.5, 1.5),$

$\text{yscale} = : \text{identity}, \text{ #log, ln, log2, log10 also works}$

$\text{linecolor} = : \text{red},$

$\text{linewidth} = : 2.0,$

$\text{linestyle} = : \text{solid}, \text{ #solid, dashdot, dot, dash}$

$\text{legend} = : \text{true},$

```

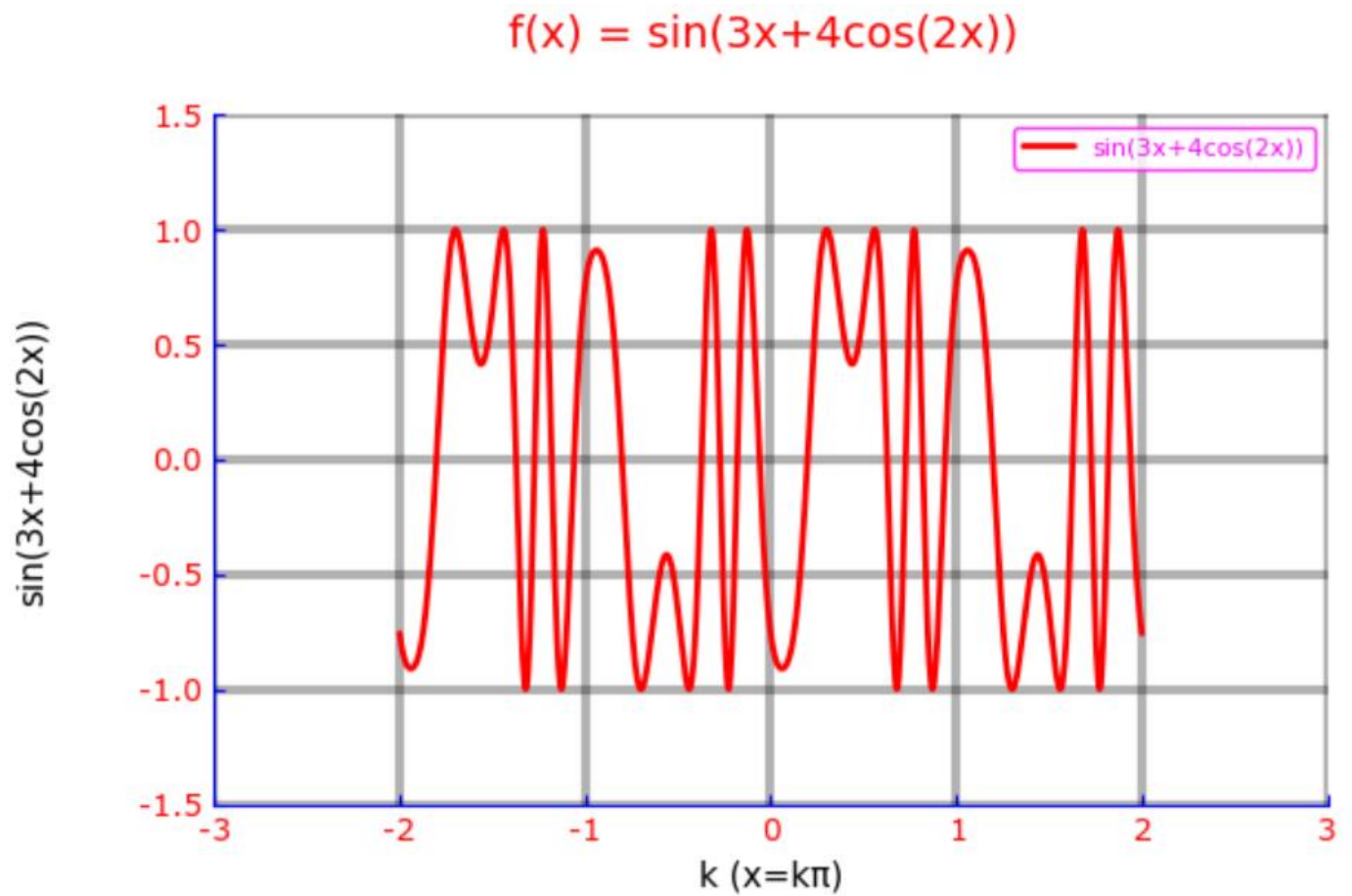
label="sin(3x+4cos(2x))",

title="f(x) = sin(3x+4cos(2x)) \n",
titlecolor=:red,

gridstyle=:solid,
grid=:3,
gridalpha=:0.3,

foreground_color_legend=:magenta,
foreground_color_grid=:black;
foreground_color_axis=:blue,
foreground_color_text=:red,
foreground_color_border=:blue,
xtickfont=font(10, "verdana"),
ytickfont=font(10, "verdana"),
xticks=(-3:1:3),
yticks=(-1.5:0.5:2.5)
#aspectratio=1/0.5,  better leave unassigned
)

```



[bde Pyplot working 1b.ipynb](#)

using Plots

pyplot()

```
x=linspace(-2π,2π,1000);
```

```
y=sin.(3x+4cos.(2x))
```

```
plot(x,y
```

```
xlabel="x",
```

```
xlim=(-2π,2π),  
ylim=(-1.5,1.5),  
  
ylabel="sin(3x+4cos(2x))\n\n",  
yscale=:identity, #log,ln,log2,log10 also works
```

```
linecolor=:red,  
linewidth=:2.0,  
linestyle=:solid, #solid, dashdot, dot,dash
```

```
legend=:true,  
label="sin(3x+4cos(2x))",
```

```
title="f(x) = sin(3x+4cos(2x)) \n",  
titlecolor=:red,
```

```
gridstyle=:solid,  
grid=:3,  
gridalpha=:0.3,
```

```
foreground_color_legend=:magenta,  
foreground_color_grid=:black;  
foreground_color_axis=:blue,  
foreground_color_text=:red,
```

```
foreground_color_border=:blue,
```

```
xtickfont=font(10, "verdana"),
```

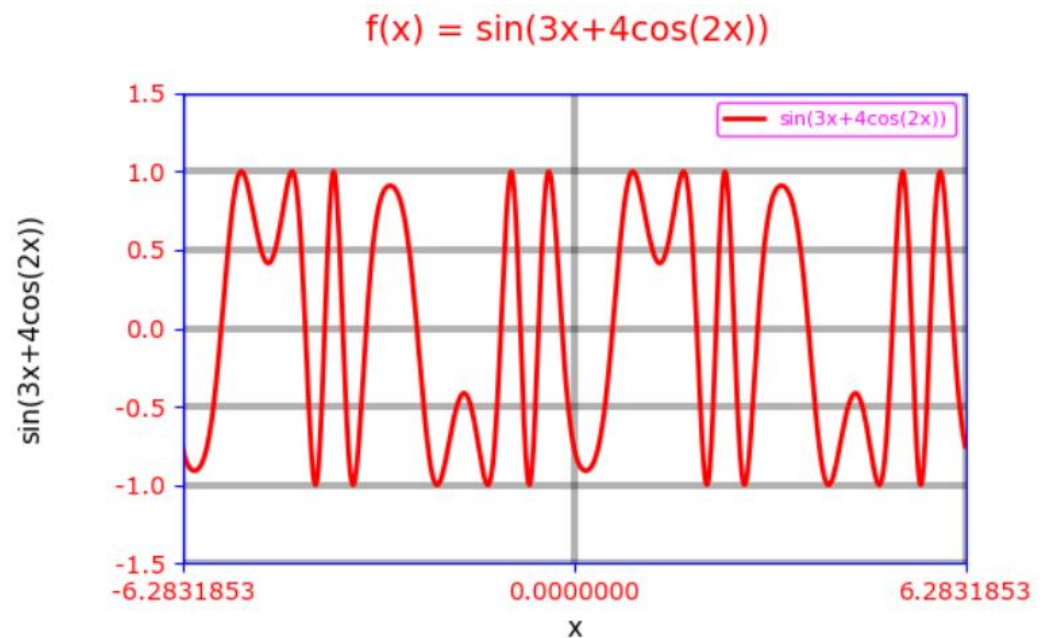
```
ytickfont=font(10,"verdana"),
```

```
xticks=(-2π:2π:2π),
```

```
yticks=(-1.5:0.5:2.5)
```

```
#aspectratio=1/0.5, better leave unassigned
```

```
)
```



bde Pyplot working 1c.ipynb

using Plots

pyplot()

x=linspace(0,2 π ,1000);

y=sin.(3x+4cos.(2x))

plot(x/ π ,y,

xlabel="k (x=k π)",

ylabel="sin(3x+4cos(2x))\n\n",

xlim=(0,2),

ylim=(-1.5,1.5),

yscale=:identity, #log,ln,log2,log10 also works

linecolor=:red,

linewidth=:2.0,

linestyle=:solid, #solid, dashdot, dot,dash

legend=:true,

label="sin(3x+4cos(2x))",


```
title="f(x) = sin(3x+4cos(2x)) \n",
```

```
titlecolor=:red,
```

```
gridstyle=:solid,
```

```
grid=:3,
```

```
gridalpha=:0.3,
```

```
foreground_color_legend=:magenta,
```

```
foreground_color_grid=:black;
```

```
foreground_color_axis=:blue,
```

```
foreground_color_text=:red,
```

```
foreground_color_border=:blue,
```

```
xtickfont=font(10 , "verdana"),
```

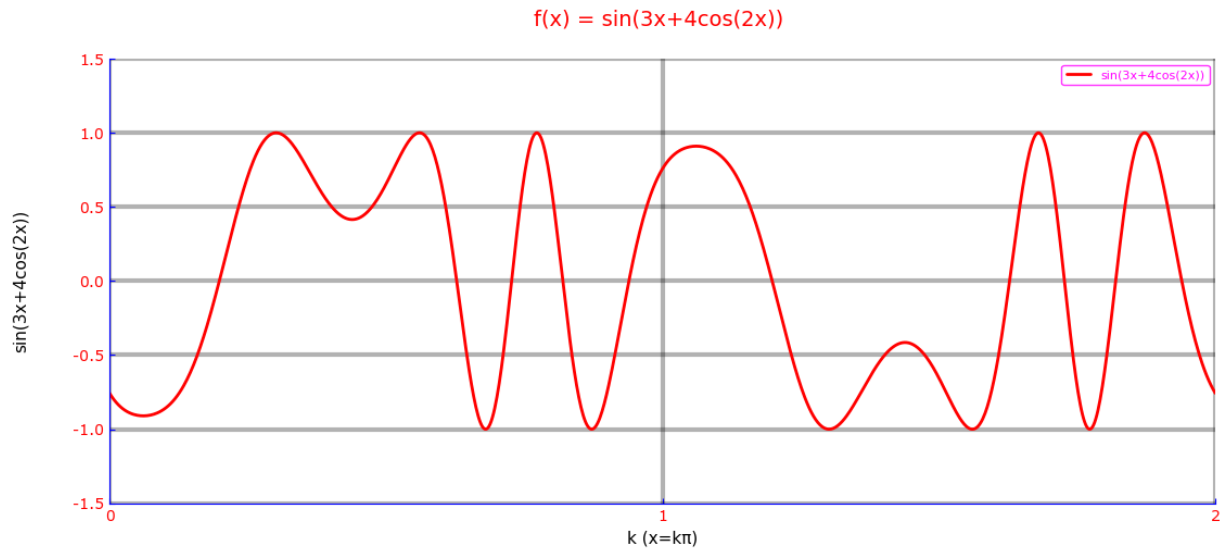
```
ytickfont=font(10,"verdana"),
```

```
xticks=(0:1:2),
```

```
yticks=(-1.5:0.5:2.5)
```

```
#aspectratio=1/0.5,  #better leave unassigned
```

```
)
```



Note : With Pyplot backend executed as `julia>include("bde Pyplot working 1c.jl")` in REPL environment we can stretch or expand the generated plot according to our needs.

Here the plot is expanded a little, to visualize better the roots of the function during a single period.

bde Pyplot working 1d.ipynb

using Plots

pyplot()

```
x=linspace(0,2π,1000);
```

```
y=sin.(3x+4cos.(2x))
```

```
plot(x,y,
```

```
xlabel="x",
```

```
y="sin.(3x+4cos(2x))",
```

```
xlim=(0,2π),
```

```
ylabel="sin(3x+4cos(2x))\n\n",
```

```
yscale=:identity, #log,ln,log2,log10 also works
```

```
ylim=(-1.5,1.5),
```

```
linecolor=:red,
```

```
linewidth=:2.0,
```

```
linestyle=:solid, #solid, dashdot, dot,dash
```

```
legend=:true,
```

```
label="sin(3x+4cos(2x))",
```

```
title="f(x) = sin(3x+4cos(2x)) \n",
```

```
titlecolor=:red,
```

```
top_margin=:25px,
```

```
left_margin=:25px,
```

```
bottom_margin=:25px,
```

```
gridstyle=:solid,
```

```
grid=:3,
```

```
gridalpha=:0.3,
```

```
foreground_color_legend=:magenta,
```

```
foreground_color_grid=:black;
```

```
foreground_color_axis=:blue,
```

```
foreground_color_text=:red,
```

```
foreground_color_border=:blue,
```

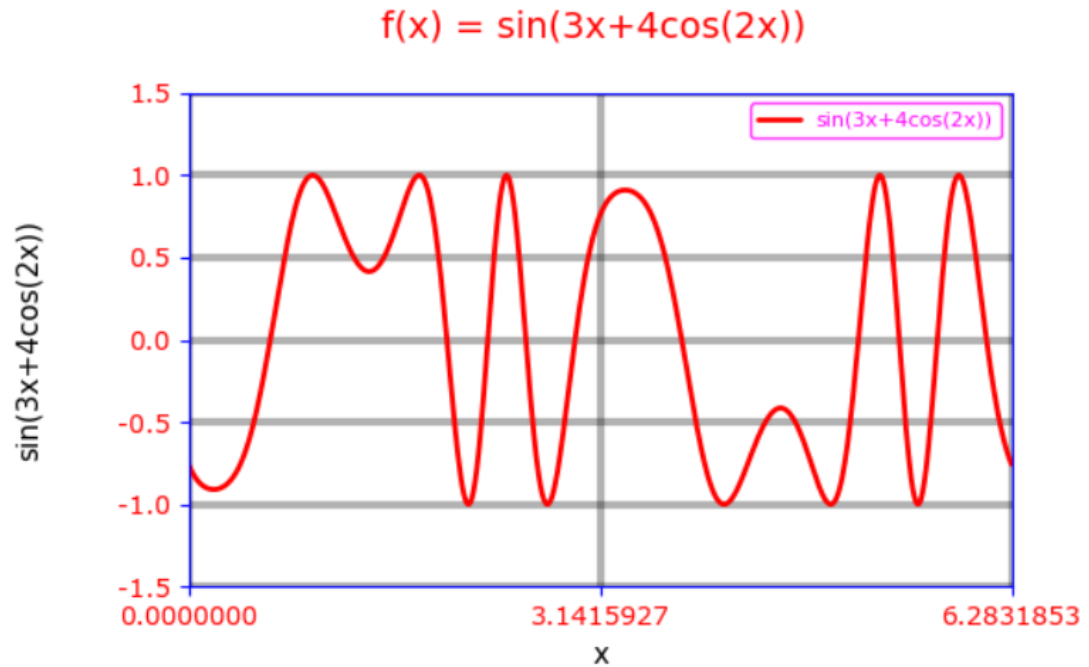
```
xtickfont=font(10 , "verdana"),
```

```
ytickfont=font(10,"verdana"),
```

```
xticks=(-2 $\pi$ : $\pi$ :2 $\pi$ ),
```

```
yticks=(-1.5:0.5:1.5)
```

```
)
```



bde Pyplot working 2

bde Pyplot working 2a.ipynb

using Plots

pyplot()

a=[1,2,3]

b=a

plot(a,b,

title="Recursive Relation Between Elements of the set $\{a\}$, $a=\{1,2,3\}$ \n",

titlefont=font(10, "verdana"),

titlecolor=:red,

label="Recursion",

xlabel="\na\n",

ylabel="a\n",

yscale=:identity, #log,ln,log2,log10 also works

ylims=:autoscale,

line=:0,

legend=:false,

label="Recursion",

gridstyle=:solid,

grid=:3,

gridalpha=:0.3,

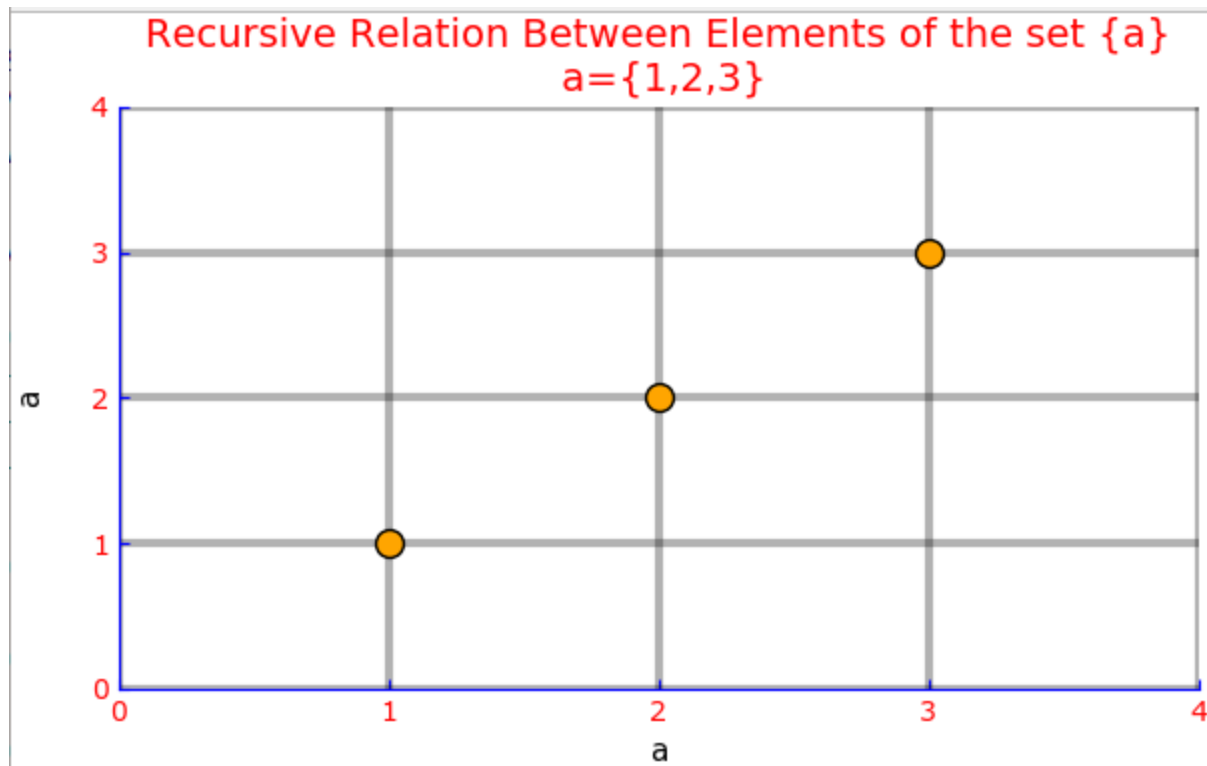
markershape=:circle ,

markercolor=:orange,

markersize=:10,

```
foreground_color_legend=:magenta,  
foreground_color_grid=:black;  
foreground_color_axis=:blue,  
foreground_color_text=:red,  
foreground_color_border=:blue,
```

```
xtickfont= font(10 , "verdana"),  
ytickfont= font(10 , "verdana"),  
xlims=(0,4),  
xticks=(0:1:4),  
ylims=(0,4),  
yticks=(0:1:4),  
)
```



bde Pyplot working 2b.ipynb

using Plots

pyplot()

$a = [1, 2, 3]$

$b = a$

plot(a, b,

title="Recursive Relation Between Elements of the set $\{a\}$ $a = \{1, 2, 3\}$ ",


```
titlefont=font(10,"verdana"),
```

```
titlecolor=:red,
```

```
label="Recursion",
```

```
xlabel="a\n",
```

```
ylabel="a\n",
```

```
yscale=:identity, #log,ln,log2,log10 also works
```

```
ylims=:autoscale,
```

```
line=:0,
```

```
legend=:false,
```

```
label="Recursion",
```

```
gridstyle=:solid,
```

```
grid=:3,
```

```
gridalpha=:0.3,
```

```
markershape=:square ,
```

```
markercolor=:red,
```

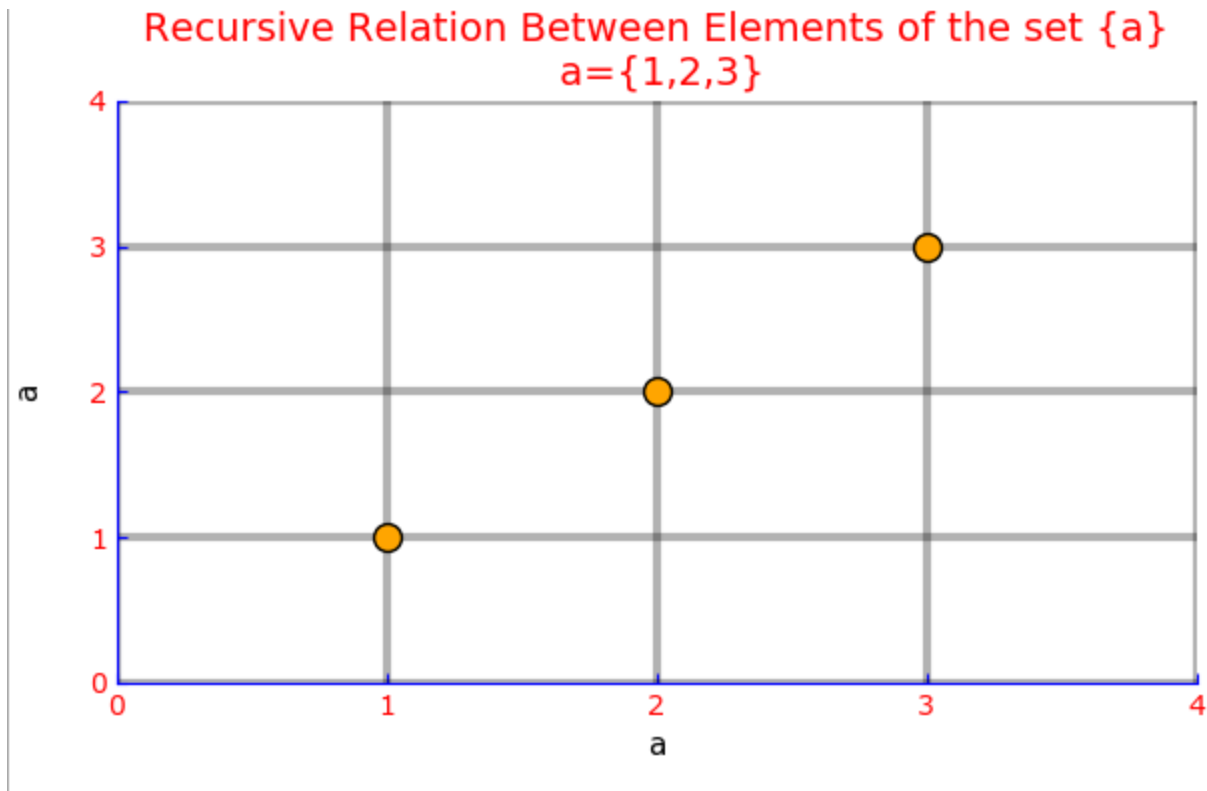
```
markersize=:7,
```

```
foreground_color_legend=:magenta,  
foreground_color_grid=:black;  
foreground_color_axis=:blue,  
foreground_color_text=:red,  
foreground_color_border=:blue,
```

```
xtickfont= font(10 , "verdana"),  
ytickfont= font(10 , "verdana"),
```

```
xlims=(0,4),  
xticks=(0:1:4),
```

```
ylims=(0,4),  
yticks=(0:1:4),  
)
```



#bde Pyplot working 3

#bde Pyplot working 3a.ipynb

using Plots

pyplot()

$x = \text{linspace}(-2\pi, 2\pi, 1000);$

$y = \sin.(x)$

```
plot(x/π,y,
```

```
xlabel="k ( $x=k\pi$ ),
```

```
ylabel="sin(x)\n\n",
```

```
xlim=(-3,3),
```

```
ylim=(-1.5,1.5),
```

```
yscale=:identity, #log,ln,log2,log10 also works
```

```
linecolor=:red,
```

```
linewidth=:2.0,
```

```
linestyle=:solid, #solid, dashdot, dot,dash
```

```
legend=:true,
```

```
label="sin(x)",
```

```
title="f(x) = sin(x) \n",
```

```
titlecolor=:red,
```

```
gridstyle=:solid,
```

```
grid=:3,
```

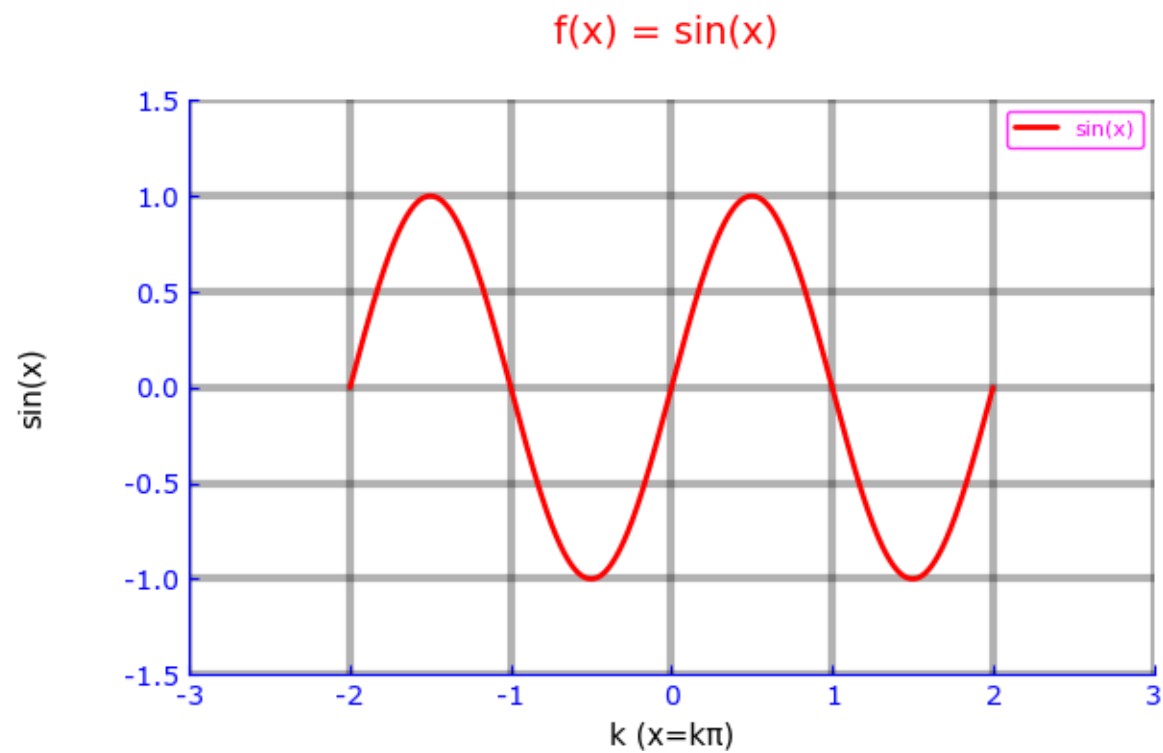
```
gridalpha=:0.3,
```

```

foreground_color_legend=:magenta,
foreground_color_grid=:black;
foreground_color_axis=:blue,
foreground_color_text=:blue,
foreground_color_border=:blue,

xtickfont=font(10, "verdana"),
ytickfont=font(10,"verdana"),
xticks=(-3:1:3),
yticks=(-1.5:0.5:2.5)
#aspectratio=1/0.5,  #better leave unassigned
)

```



Note : Inserted from the downloaded image file “bde Plot working 3a.png”.

#bde Pyplot working 3b.ipynb

using Plots

pyplot()

$x = \text{linspace}(-2\pi, 2\pi, 1000);$

$y = \sin.(x)$

plot(x,y,

xlabel="x",

ylabel="sin(x)\n\n",

xlim=(-2 π ,2 π),

ylim=(-1.5,1.5),

yscale=:identity, #log,ln,log2,log10 also works

linecolor=:blue,

linewidth=:2.0,

linestyle=:dashdot, #solid, dashdot, dot,dash

legend=:true,

label="sin(x)",

title="f(x) = sin(x) \n",

titlecolor=:red,

gridstyle=:solid,

grid=:3,

gridalpha=:0.3,

foreground_color_legend=:magenta,

foreground_color_grid=:black;

foreground_color_axis=:blue,

foreground_color_text=:red,

foreground_color_border=:blue,

xtickfont=font(10, "verdana"),

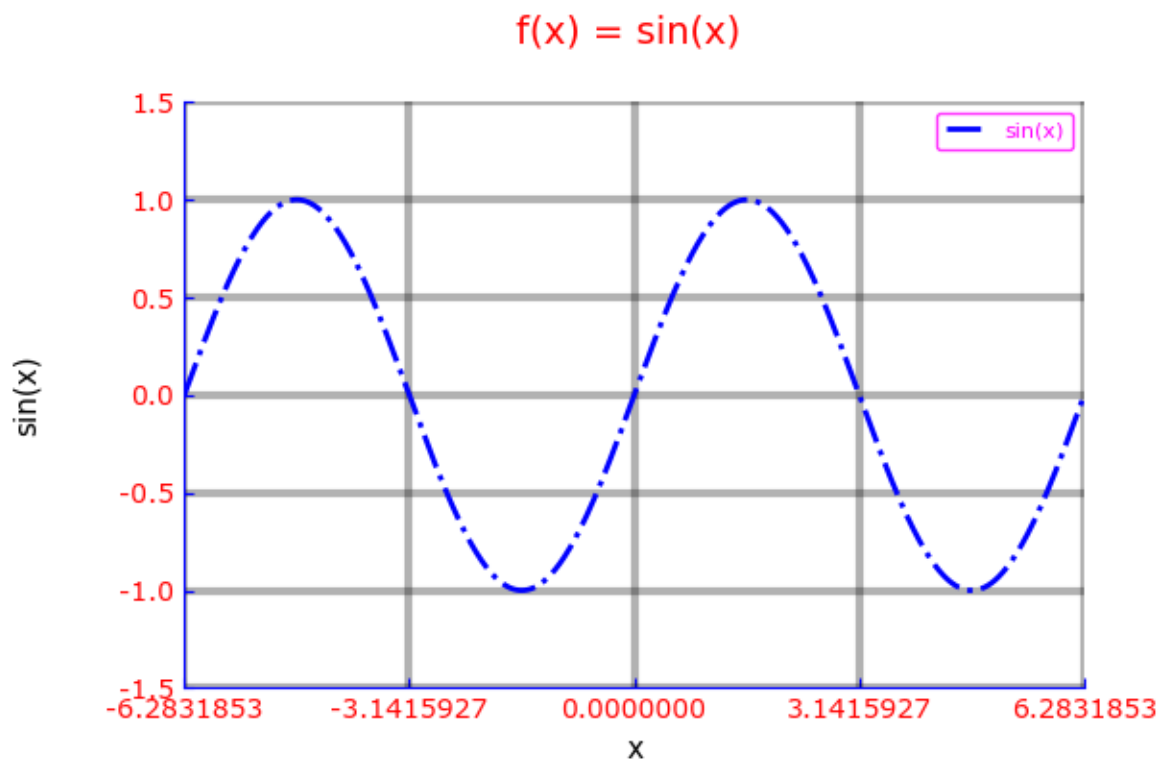
ytickfont=font(10,"verdana"),

xticks=(-2 π : π :2 π),

yticks=(-1.5:0.5:2.5)

#aspectratio=1/0.5, #better leave unassigned

)



#bde Pyplot working 3c.jl

using Plots

pyplot()

$x = \text{linspace}(0, 2\pi, 30)$

$y = \cos.(x)$

$\text{plot}(x, y,$


```
line=:false,  
xlabel="k ( $x=k \pi$ )",  
ylabel="cos(x)",  
grid=:true,  
gridcolor=:red,  
grid=3,  
gridalpha=0.3,  
title="cos(x)",  
label="cos(x)",  
markershape=:circle ,  
markercolor=:orange,  
markersize=:6,  
titlecolor=:red,  
line=:0,  
legend=:true,  
  
foreground_color_legend=:magenta,  
foreground_color_grid=:black;  
foreground_color_axis=:blue,  
foreground_color_text=:red,  
foreground_color_border=:blue,  
  
xtickfont= font(8 , "verdana"),
```

```
ytickfont= font(8 , "verdana"),
```

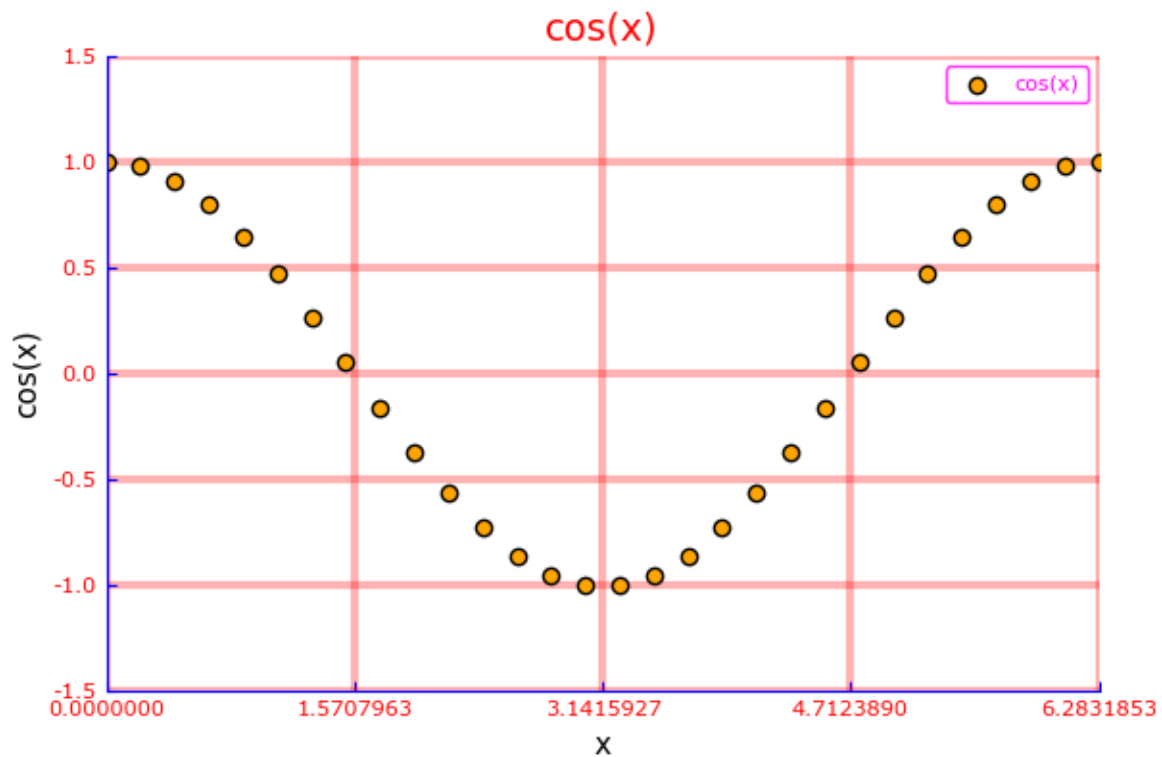
```
xlims=(0,2 $\pi$ ),
```

```
xticks=(0: $\pi$ /2:2 $\pi$ ),
```

```
ylims=(-1.5,1.5),
```

```
yticks=(-1.5:0.5:1.5),
```

```
)
```



bde Pyplot working 4

#bde Pyplot working 4a.ipynb

using Plots

pyplot()

x=linspace(-2π , 2π ,100);

y1=sin.(x);

y2=cos.(x);

plot(x/ π , [y1,y2],

line=(:path),

label=["cos(x)" "sin(x)"],

xlabel="k (x=k π ",

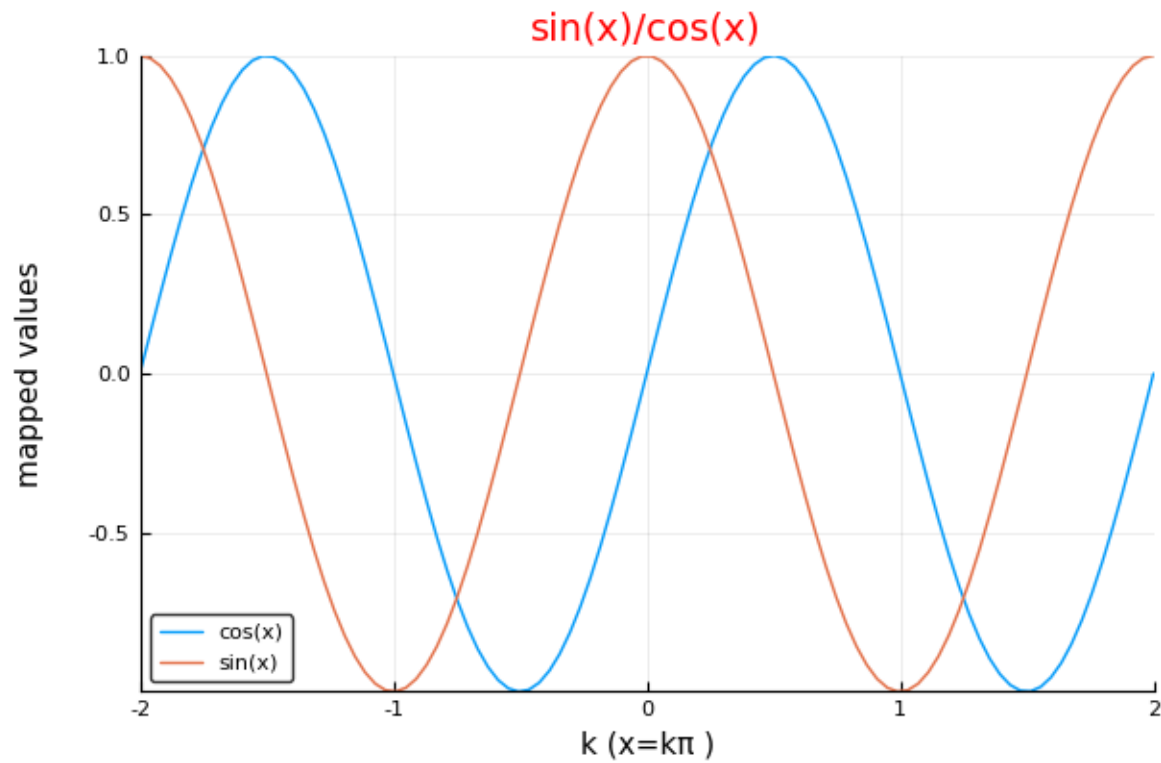
ylabel="mapped values\n",

xticks=(minimum(x)/ π :1:maximum(x)/ π),

title="sin(x)/cos(x)",

titlecolor=:red,

)



#bde Pyplot working 4b.ipynb

using Plots

pyplot()

$x = \text{linspace}(-2\pi, 2\pi, 100);$

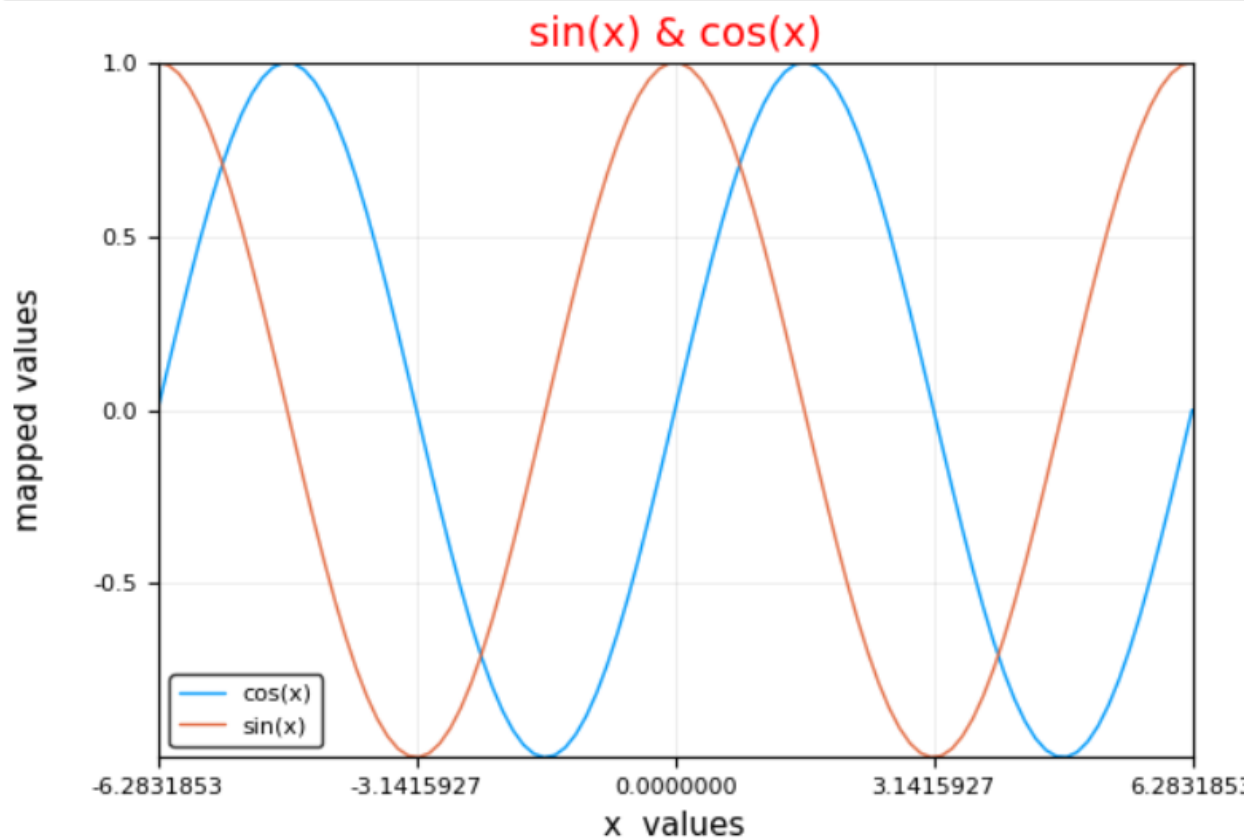
$y1 = \sin.(x);$

$y2 = \cos.(x);$

$\text{plot}(x, [y1, y2],$

line=(:path),

```
label=["cos(x)" "sin(x)"],
xlabel="x values",
ylabel="mapped values\n",
xticks=(minimum(x): $\pi$ :maximum(x)),
title="sin(x) & cos(x)",
titlecolor=:red,
)
```



#bde Pyplot working 4c.ipynb

using Plots

```
pyplot()
```

```
x=linspace(-2 $\pi$ ,2 $\pi$ ,100);
```

```
y1=sin.(x);
```

```
y2=cos.(x);
```

```
plot(x/ $\pi$ ,[y1,y2],
```

```
xlabel="k (x=k $\pi$ )",
```

```
xlim=(-2,2),
```

```
ylabel="sin(x) , cos(x)\n\n",
```

```
yscale=:identity, #log,ln,log2,log10 also works
```

```
ylim=(-1.5,1.5),
```

```
linecolor=:auto,
```

```
linewidth=:2.0,
```

```
linestyle=:solid, #solid, dashdot, dot,dash
```

```
legend=:true,
```

```
label=["sin(x)" "cos(x)"],
```

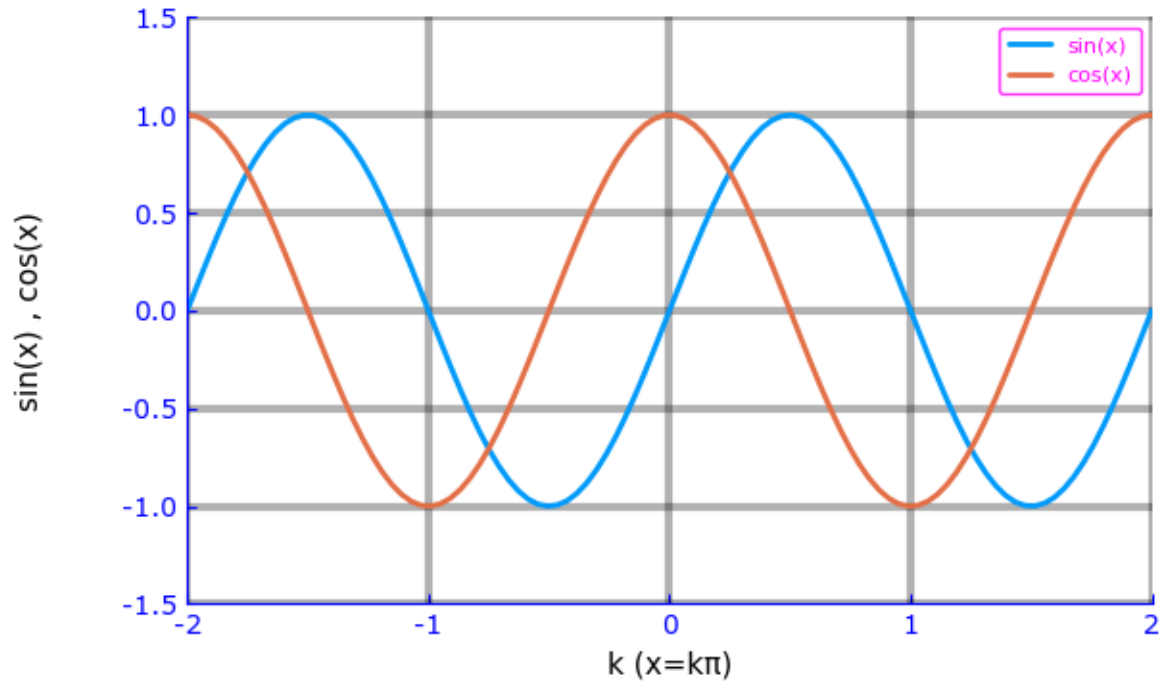
```
title="f1(x) = sin(x) , f2(x) = cos(x) \n",  
titlecolor=:red,
```

```
gridstyle=:solid,  
grid=:3,  
gridalpha=:0.3,
```

```
foreground_color_legend=:magenta,  
foreground_color_grid=:black;  
foreground_color_axis=:blue,  
foreground_color_text=:blue,  
foreground_color_border=:blue,
```

```
xtickfont=font(10, "verdana"),  
ytickfont=font(10,"verdana"),  
xticks=(-2:1:2),  
yticks=(-1.5:0.5:2.5)  
#aspectratio=1/0.5,  #better leave unassigned  
)
```

$$f_1(x) = \sin(x) , f_2(x) = \cos(x)$$



#bde Pyplot working 4d.ipynb

using Plots

pyplot()

```
x=linspace(-2π,2π,100);
```

```
y1=sin.(x);
```

```
y2=cos.(x);
```

```
plot(x,[y1,y2],
```

```
xlabel=:"x",
```



```
xlim=(-2.5 $\pi$ ,2.5 $\pi$ ),
```

```
ylabel="sin(x) , cos(x)\n\n",
```

```
yscale=:identity, #log,ln,log2,log10 also works
```

```
ylim=(-1.5,1.5),
```

```
linecolor=:auto,
```

```
linewidth=:2.0,
```

```
linestyle=:solid, #solid, dashdot, dot,dash
```

```
legend=:true,
```

```
label=["sin(x)" "cos(x)"],
```

```
title="f1(x) = sin(x) , f2(x) = cos(x) \n",
```

```
titlecolor=:red,
```

```
gridstyle=:solid,
```

```
grid=:3,
```

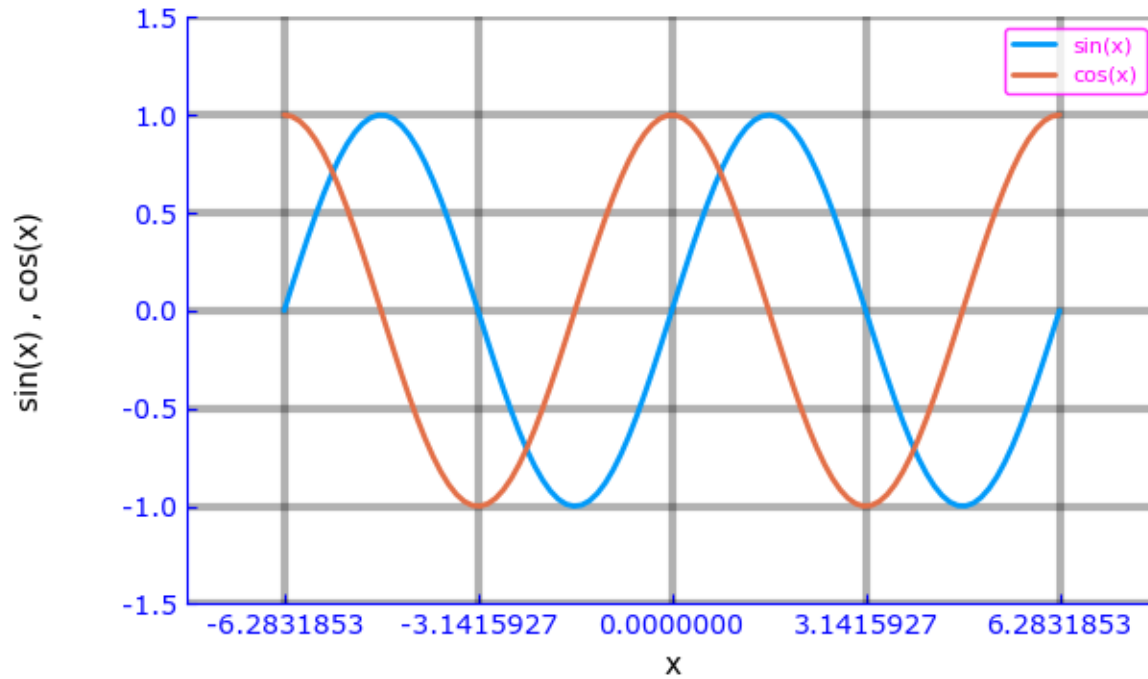
```
gridalpha=:0.3,
```

```
foreground_color_legend=:magenta,
```

```
foreground_color_grid=:black;  
foreground_color_axis=:blue,  
foreground_color_text=:blue,  
foreground_color_border=:blue,
```

```
xtickfont=font(10, "verdana"),  
ytickfont=font(10, "verdana"),  
xticks=(-2 $\pi$ :(2//2) $\pi$ :2 $\pi$ ),  
yticks=(-1.5:0.5:2.5)  
#aspectratio=1/0.5, #better leave unassigned  
)
```

$$f_1(x) = \sin(x) , f_2(x) = \cos(x)$$



#bde Pyplot working 4e.ipynb

using Plots

pyplot()

```
x=linspace(-2π,2π,100);
```

```
y1=sin.(x);
```

```
y2=cos.(x);
```

```
y3=tan.(x);
```

```
plot(x/π,[y1,y2,y3],
```

```
title="sin(x) , cos(x) , tg(x) \n",
```

```
titlecolor=:red,
```

```
grid=:1,
```

```
gridstyle=:solid,
```

```
gridalpha=:0.3,
```

```
foreground_color_grid=:black,
```

```
line=:true,
```

```
#linecolor=:red,
```

```
linewidth=:1.0,
```

```
linestyle=:auto, #auto,solid,dot,dash,dashdot
```

```
xlabel="k ( $x=k\pi$ )",
```

```
ylabel="sin(x) /cos(x) / tg(x)",
```

```
legend=:true,
```

```
label=["sin(x)" "cos(x) " "tg(x)"],
```

```
legendtitle="",
```

```
foreground_color_legend=:magenta,
```

yscale=:identity, #log,ln,log2,log10 also works

foreground_color_axis=:red,
foreground_color_text=:blue,
foreground_color_border=:black,

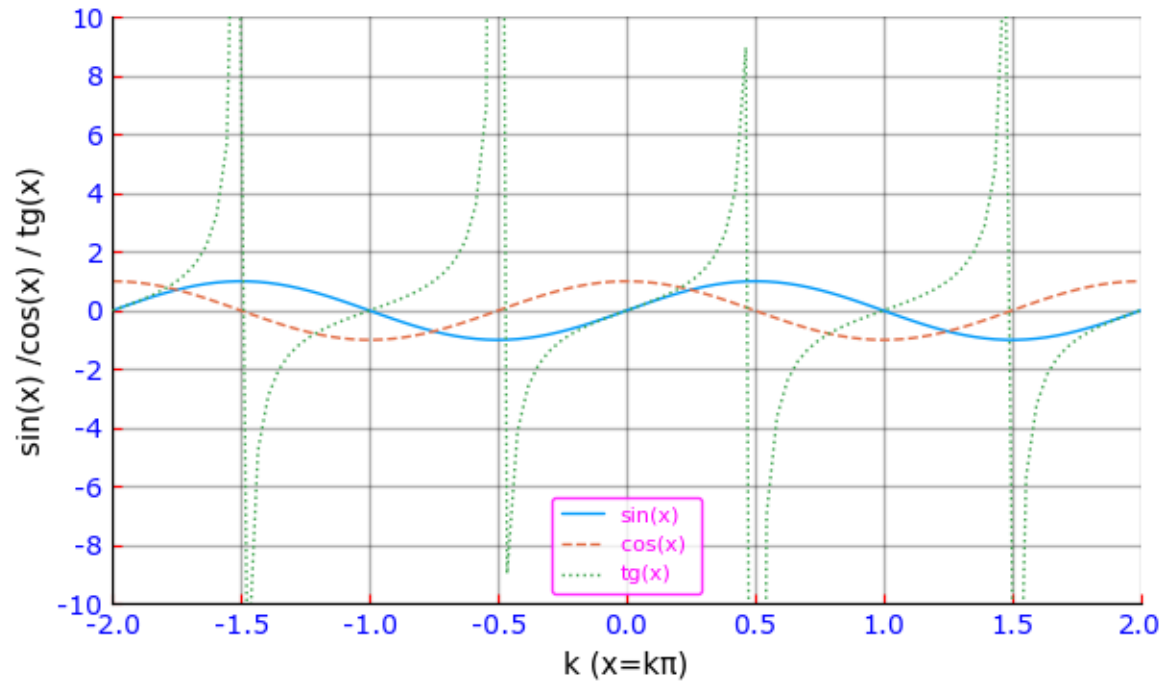
xtickfont=font(10 , "verdana"),
ytickfont=font(10 , "verdana"),

xlim=(-2,2),
ylim=(-10,10),

xticks=(-2:0.5:2),
yticks=(-10:2:10)
#aspectratio=1/0.5, #better leave unassigned)

)

$\sin(x)$, $\cos(x)$, $\tan(x)$



#bde Pyplot working 4f.ipynb

using Plots

pyplot()

```
x=linspace(-2π,2π,100);
```

```
y1=sin.(x);
```

```
y2=cos.(x);
```

```
y3=tan.(x);
```

```
plot(x,[y1,y2,y3],
```

```
title="sin(x) , cos(x) , tan(x) / $\pi$  \n",
```

```
titlecolor=:red,
```

```
grid=:1,
```

```
gridstyle=:solid,
```

```
gridalpha=:0.3,
```

```
foreground_color_grid=:black,
```

```
line=:true,
```

```
#linecolor=:red,
```

```
linewidth=:1.0,
```

```
linestyle=:auto, #auto,solid,dot,dash,dashdot
```

```
xlabel =:"x",
```

```
ylabel="sin(x) /cos(x) / tan(x)",
```

```
legend=:true,
```

```
label=["sin(x)" "cos(x)" "tan(x)"],
```

```
legendtitle="",
```

```
foreground_color_legend=:magenta,
```

```
yscale=:identity, #log,ln,log2,log10 also works
```

```
foreground_color_axis=:red,
```

```
foreground_color_text=:blue,
```

```
foreground_color_border=:black,
```

```
xtickfont=font(10 , "verdana"),
```

```
ytickfont=font(10 , "verdana"),
```

```
xlim=(minimum(x),maximum(x)),
```

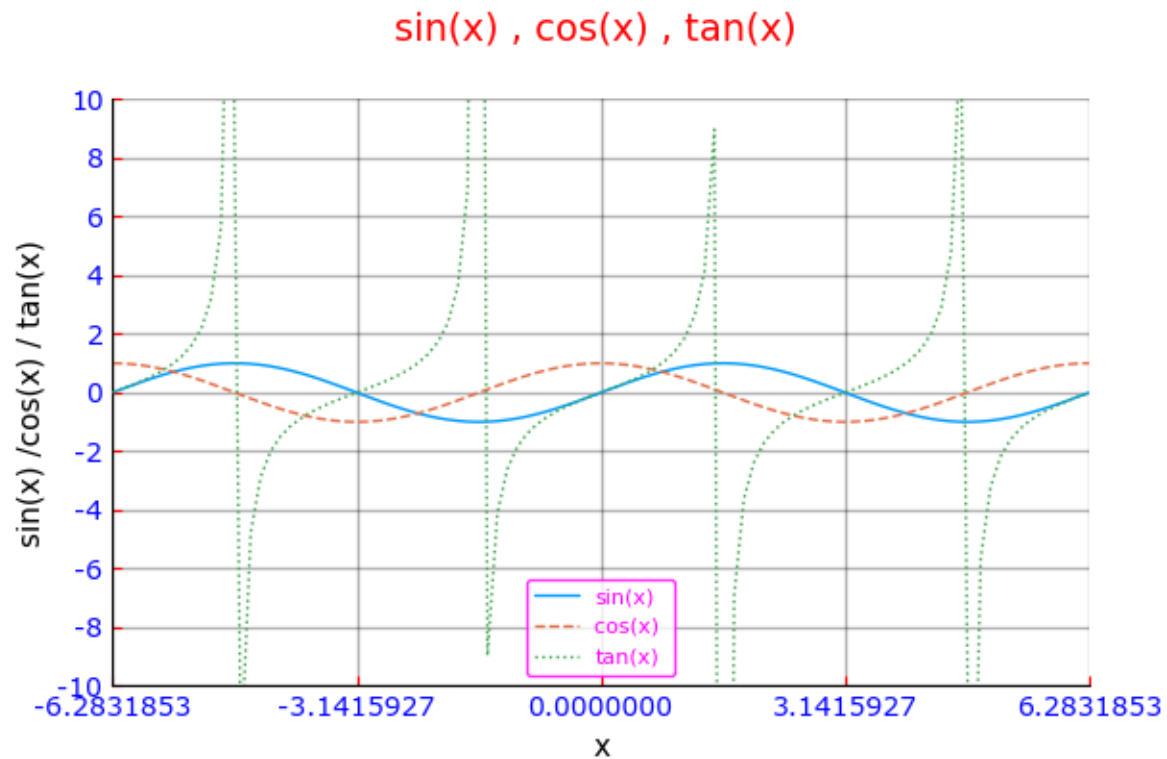
```
ylim=(-10,10),
```

```
xticks=(-2 $\pi$ : $\pi$ :2 $\pi$ ),
```

```
yticks=(-10:2:10)
```

```
#aspectratio=1/0.5, #better leave unassigned)
```

```
)
```

bde Pyplot working 5

#bde Pyplot working 5a.ipynb

using Plots

pyplot()

$x = \text{linspace}(0, 2, 20)$

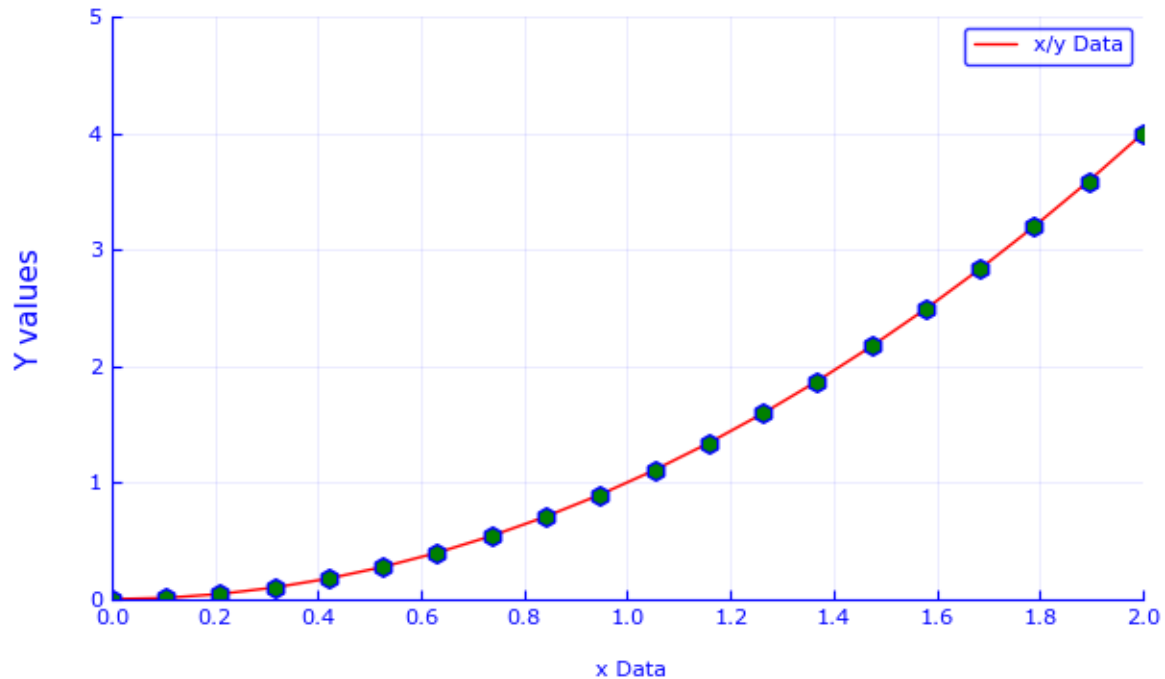
$y = x.^2$

`Plots.plot(x, y,`

`markershape=:hexagon,`

```
markersize=10,
markercolor=:green,
label="x/y Data",
xaxis=(" \n x Data",(0,2), 0:0.2:2,:identity , font(8,"Verdana")), #flip (after
identity), log,ln,log2,log10 also works in place of identity
linestyle=:solid, #solid (default), dot, dash,dashdot
linecolor=:red,
title="Data Points \n",
titlecolor=:red,
foreground=:blue,#black (default)
grid=:true, # false, otherwise
#aspectratio=:1/20, #1/1 ,2/1 #equal leave default
# alternative to tuple
ylabel="Y values\n",
ylims=(0,5),
yticks=(0:1:5),
yscale=:identity, #log,ln,log2,log10 also works
ytickfont= font(8 , "verdana")
)
```

Data Points



#bde Pyplot working 5b.ipynb

using Plots

```
pyplot()
```

```
x=linspace(0,2,20)
```

```
y=x.^2
```

```
Plots.plot([x y],
```

```
    # note : this is x/x and x/y graphs you may add many more as z/x , w/x etc...
```

```
    markershape=[:circle :rect], # valid shapes
```

```
[:none,:auto,:circle,:rect,star4,:star5,:star6,:star7,:diamond,:hexagon,:cross,
```

```
:xcross,:]
```

```
#:utriangle,:dtriangle,:rtriangle,:ltriangle,:pentagon,:heptagon,:hexagon,:octagon,
:vlane,:hline,:+,:x,:pixel]
```

```
markercolors=[:red :blue],
```

```
labels=["x Data" "x^2 Data"],
```

```
title = "X and x^2 graphs\n",
```

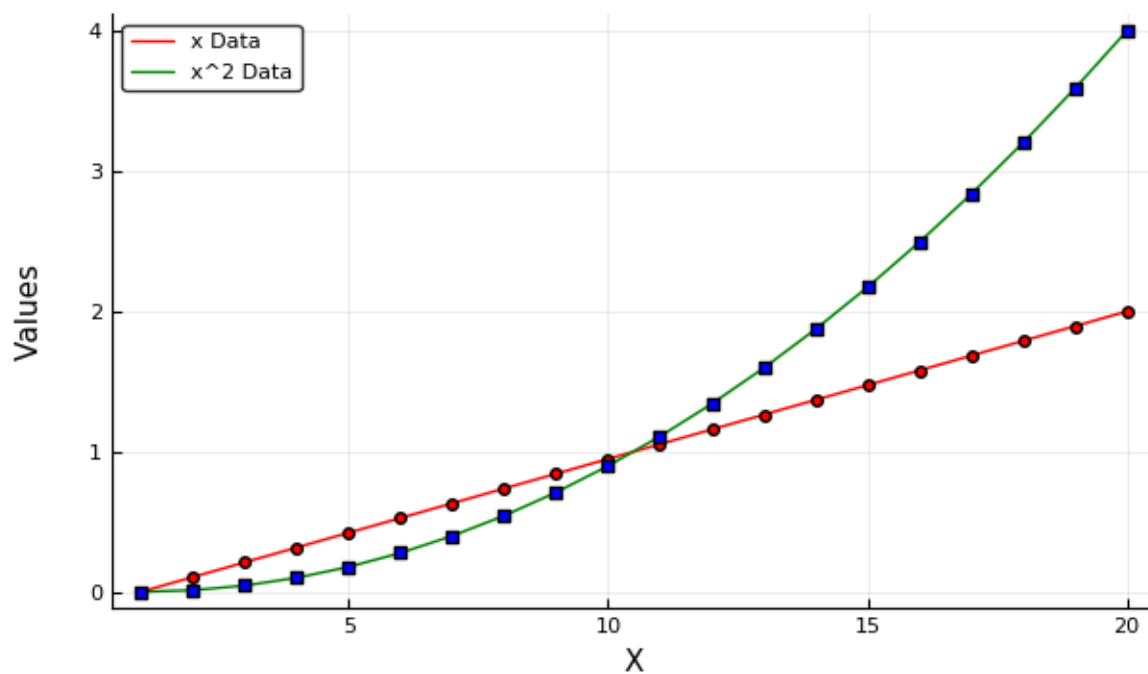
```
ylabel = "Values\n",
```

```
xlabel = "X -->",
```

```
linecolors=[:red :green]
```

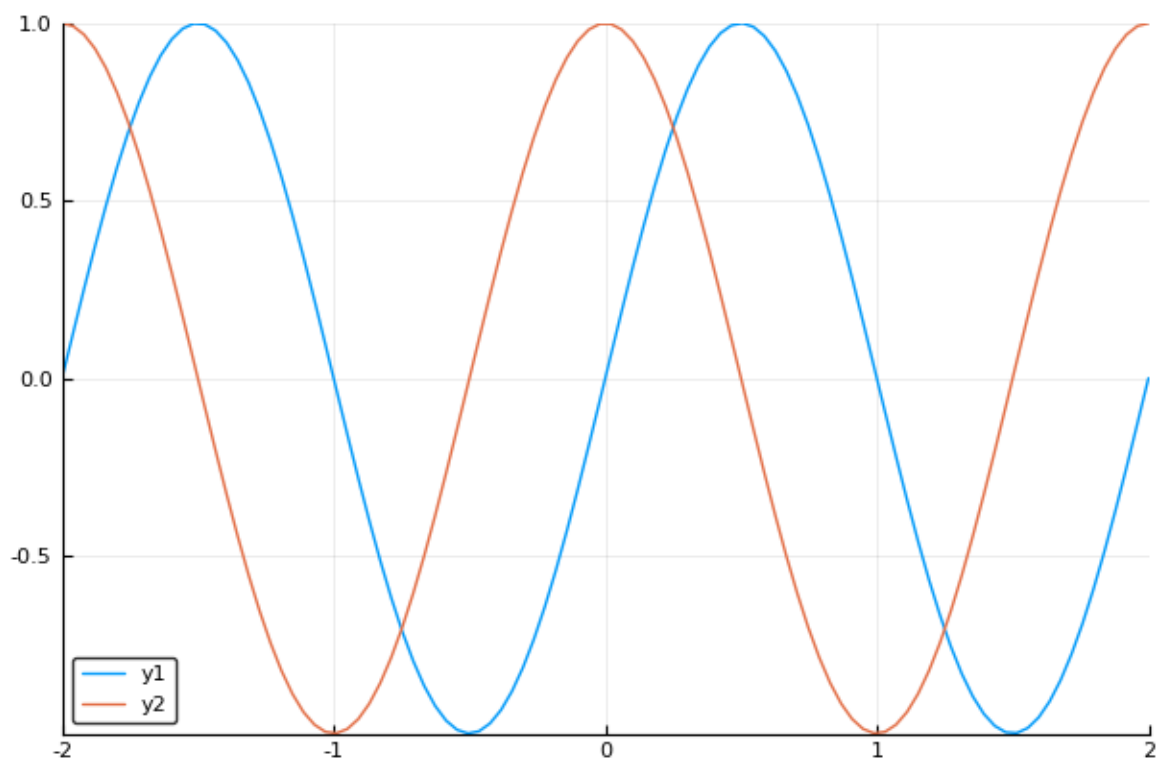
```
)
```

$f(x)=X$ and $f(x)=x^2$ graphs



#bde Pyplot working most simpler 5c.jl

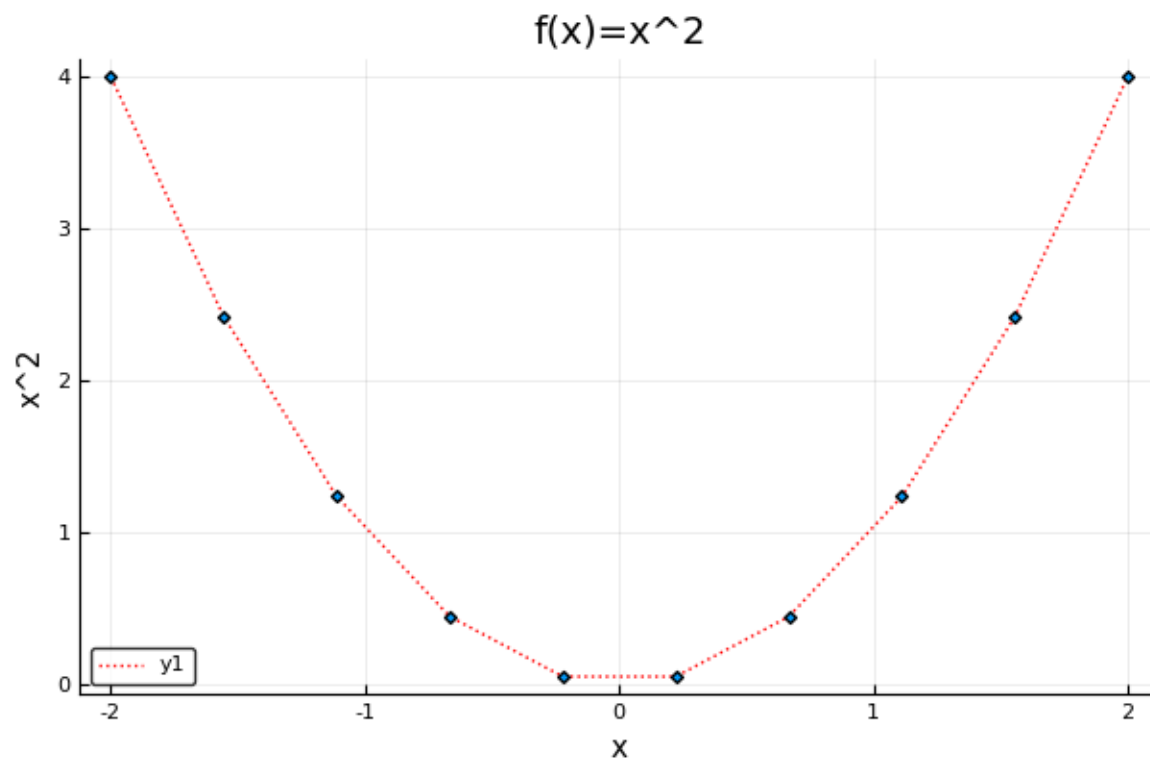
```
using Plots
pyplot()
x=linspace(-2pi,2pi,100);
y1=sin.(x);
y2=cos.(x);
plot(x/pi,[y1,y2])
#plot(x,[y1,y2])
```



#bde Pyplot working most compact 5d.jl

```
using Plots
pyplot()
x=linspace(-2, 2, 10)
```

```
y=x.^2  
plot(x,y,  
title="f(x)=x^2",  
xlabel="x",  
ylabel="x^2",  
marker=(:'diamond',3),  
line=(:'path','dot','red'),  
legend=false,  
grid=false)
```



Plotting with Gadfly

Gadfly is an R based graphic generator ported to Julia.

This is a vast plotting program including many different plotting objectives.

Produces stunning graphs but the programming is a bit different then the previous plotting programs.

We will only envisage point, single and multiline graphs.

These are the ones that we encounter in our studying context at most.

The examples presented here, may serve as templates to your existing plotting needs.

Gadfly has different sections to organise a plot:

- **Aesthetics** defines data to be plotted.
- **Geometries** defines what to be plotted, lines, points (default) or both.
- **Statistics** a middle layer between Aesthetics and Geometry, displays statistically defined transformations of the raw data (like mean, median etc..)
- **Guides** for describing axes labels, plot titles etc.
- **Coord** for determining axes limits.
- **Scales** for arranging normal (default) or log-normal, log-log plots.
- **Themes** for coloring different visuals of the plot.
- **Layers** for superposing more then one relation in one plot.

First of them is called Aesthetics, this is the data section which describes data being plotted.

When working with Gadfly, we can choose between two alternatives. Either,

- we can have a graph in a default browser,

- we can have a picture as a *.png, in the current workspace (the one indicated by `pwd()`).

Normally people prefer the former case. In this case we can see an immediate graph displayed in the default browser and if we wish, we can handle it by means of picture enhancing programs.

But nevertheless the second choice also may be preferred, especially for small data preparation steps which will not populate much the `plot()` function.

In these examples, we will only give a single program which will produce a `bdeGadflyWorking1a.png` in your workplace. This is a sample program which may serve as a guide for all other Gadfly graphing programs for converting from display to download versions.

Gadfly Aesthetics section has two different methods to present data to be plotted to Gadfly:

- Preparation of the data in the outside of the `plot()` function.
- Preparation of the data in the inside of the `plot()` function.

Both of the methods work. But, for the reason that the `plot()` function wording won't be cluttered, the first possibility is the one that is advised to be applied.

Geometry section is introduced by specifying which one will be applied from two properties of the **Geom** object:

Geom.point property is the default property for stating that the generated plot will be consisted from points only. Since it is the default property, one may not state it explicitly when only scatter plots are to be plotted.

Geom.line should be stated explicitly to override the default **Geom.point** property and generate a line plot.

When a plot of line segments joining the points would be the choice, one should explicitly state **Geom.point, Geom.line** together.

Guide section is arranged with the specification of different properties of the **Guide** object. Guide object, has properties to control the plot labels, plot title, x and y axes labels and other tools to enhance the visual appearances of the

generated plot. We will see the applications of these properties in the forthcoming examples.

The **Scales** object control the scaling of the plot whether it would be a normal or various log plots. Also, it may signal to the Gadfly that the function to be plotted is continuous or discrete. Since the default behavior is continuous function with automatically determining the discontinuing areas, generally there will no need to alter the default behaviours of this object.

Coord object has important properties to control the axes limits.

Before using Gadfly, add to the Julia kernel,

```
Pkg.add("Gadfly")
```

```
Pkg.add("Cairo")
```

Adding packages once is sufficient.

It also advised to update packages each time the Julia kernel is started:

```
Pkg.update()
```

The first example below, prepares the plotting data outside of the Plot() function and produces a picture "bdeGadflyworking1a.png" in the current workspace.

Program Listing of the bdeGadflyworking1a.jl

(written with SciTE)

(can be written with any text editor)

```
using Gadfly
```

```
using Cairo
```

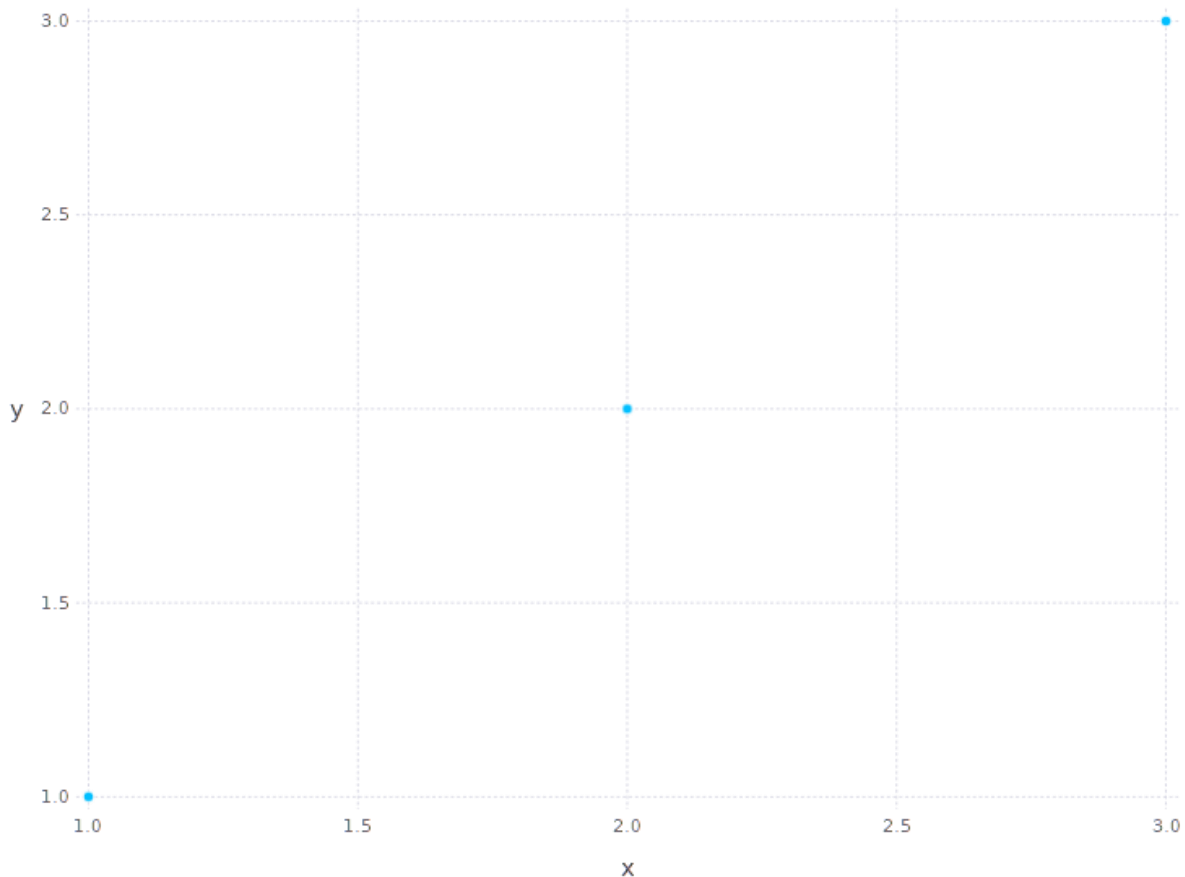
```
x=[1,2,3]
```

```
y= x
```

```
draw(PNG("bdeGadflyWorking1a.png",8inch,6inch),  
plot(x=x,y=x))  
println("success!")
```

saved “bdeGadflyworking1a.png” at a workplace within your computer.

That is the png graph produced:



As we can see Gadfly produce very clear and intuitively crafted graphs prone to publication. But they may be enhanced more, as we will see in coming examples.

The next program, “bdeGadflyworking1b.jl” defines data to be plotted inside the

plot() function. For the data preparations not having too much wording. This program generates an immediate graph displayed in a default browser.

When we instead of inserting a *.png image in a current workspace and continue the program, trying to see it in the default browser, the last program step should only be the plot() function.

We can generate a *.png image in the middle of the program steps, whereas when generating an immediate graph, we should dedicate the whole program to the generation of the plot. This is the distinction between generating a *.png image and an immediate graph.

Now, in the program below, we prepare the data inside the plot() function. For small data there won't be too much cluttering.

Program Listing of the bde Gadfly working 1b.jl:

using Gadfly

```
plot(x=linspace(-2pi,2pi,100),y=sin.(x))
```

save at any place in your computer. This place will be your workspace.

Open REPL

```
julia>include("d:\\bedri\\dropbox\\bedriemir\\Julia\\PlottingExamples\\bdeGadflyworking1b") ← enter
```

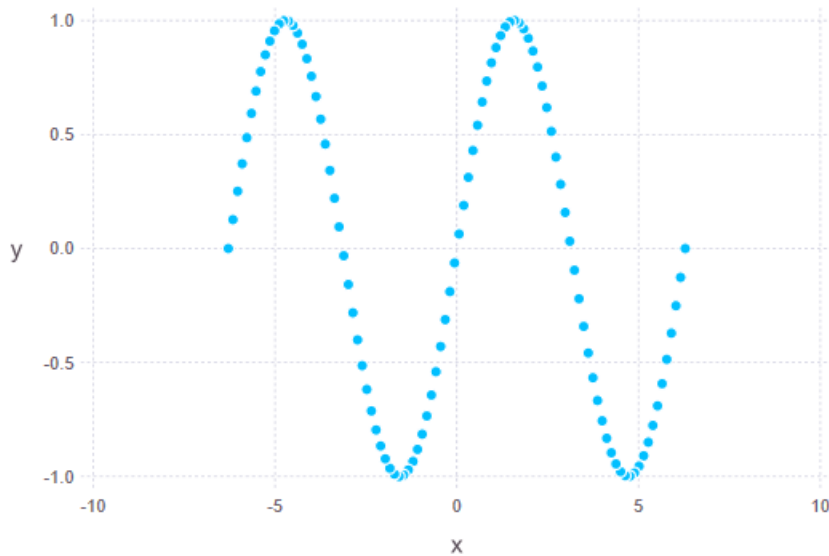
or better, change directory to your workspace :

```
julia>cd("d:\\bedri\\dropbox\\bedriemir\\Julia\\PlottingExamples") ← enter
```

julia>include("bdeGadflyworking1b") ← enter

change the the names with your own and you may continue to execute the programs residing in your workplace.

A default browser window will be opened and the graph below will be displayed :



stunning !

Not bad for a two program lines,

let's carry on further:

We can change the units of the x:

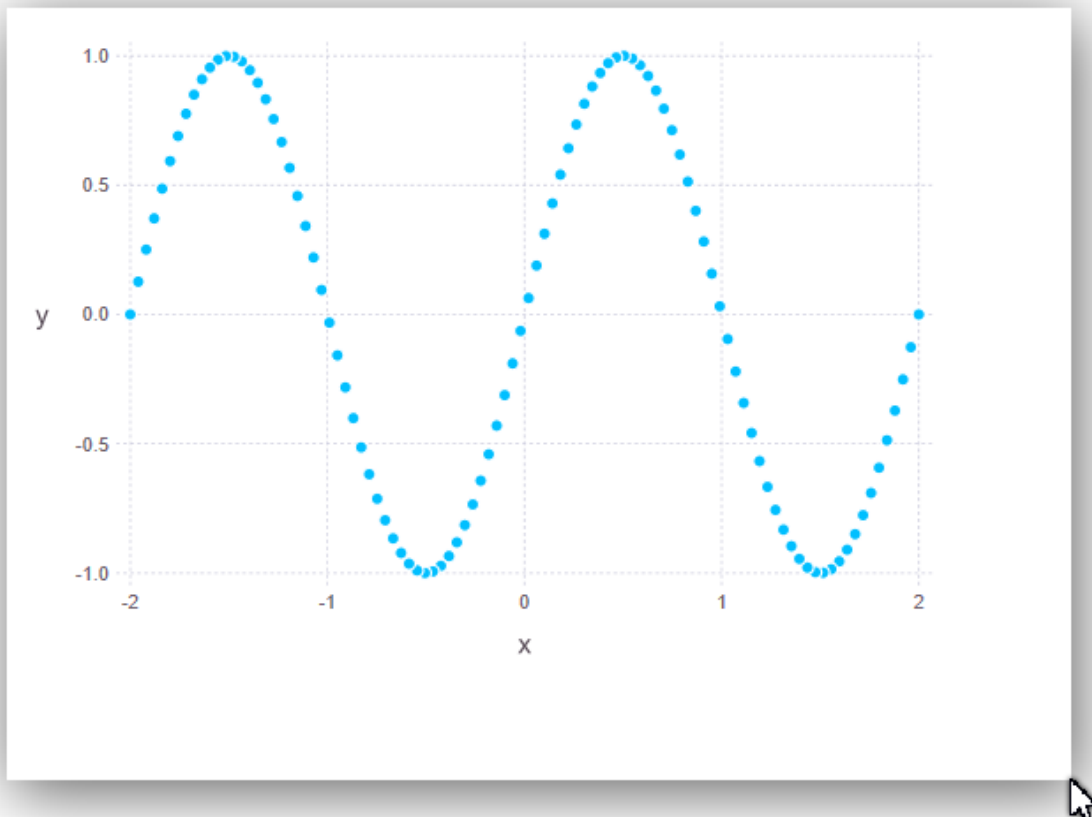
Program Listing of the "bdeGadflyworking1c.jl"

(May be written with any text editor)

using Gadfly

```
plot(x=linspace(-2pi,2pi,100),x=x/pi,y=sin.(x))
```

The produced graph :



As we can see from the above program listing, generating data within the plot program, generally makes the program reading more difficult and brings ambiguity to the graph program steps. Therefore it may be worth to prepare data outside of the plot() function. Change the the program listing of “bdeGadflyworking1c.jl” as displayed in “bdeGaflyworking1c-1.jl” as shown below. It will create the same plot, but with more clearer program steps.

Program Listing of the "bdeGadflyworking1c-1.jl"

```
using Gadfly
```

```
x=linspace(-2pi,2pi,100 )
```

```
y=sin.(x)
```

```
x=x/pi
```

```
plot(x=x, y=y)
```

```
include("D:\\bedri\\Dropbox\\bedriemir\\Julia\\PlottingExamples\\bdeGadflyworking1c-1.jl")
```

generates the same plot as the former program.

Gadfly has some special arrangements for plotting builtin and user defined named and anonymous functions.

Trigonometric functions and log,log10,log2, sqrt, functions are all defined.The builtin form is as follows:

plot([sqrt], minvalue, maxvalue) or all the functions together :

```
plot([sin,cos], 0,2pi)
```

program listing of the bdeGadflyworking1c-2.jl

```
using Gadfly
```

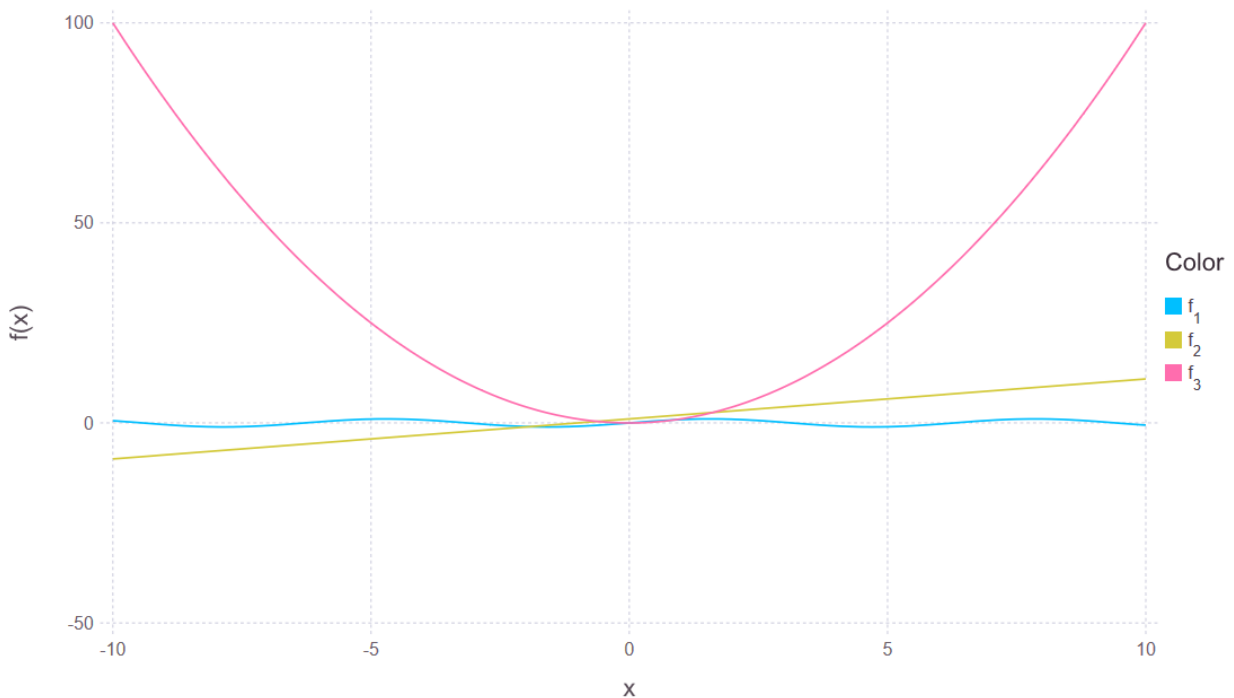
```
function oneplus(x)
```

```
    return 1+x
```

```
end
```

```
plot ([sin,oneplus, x->x*x],0,10)
```

Thr resulting plot :

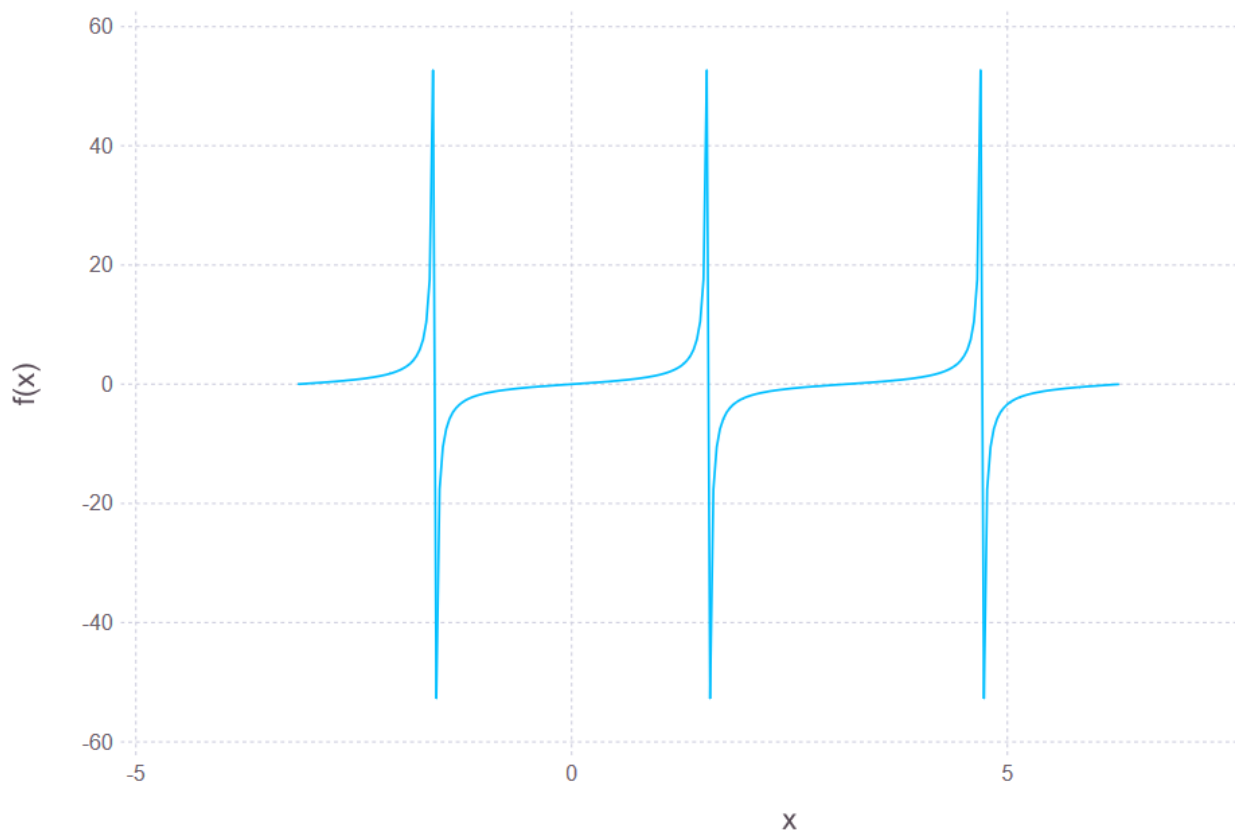


This is an outstanding opportunity when learning elementary and advanced functions as well as trigonometric functions. But the plots generated are only for inspection and far being a publication quality plots. We should work hard for producing publication quality plots. Some examples of inspection plots, are given below .

[program listing of the bdeGadflyworking1c-3.jl](#)

using Gadfly

plot([tan],-2 π ,2 π)



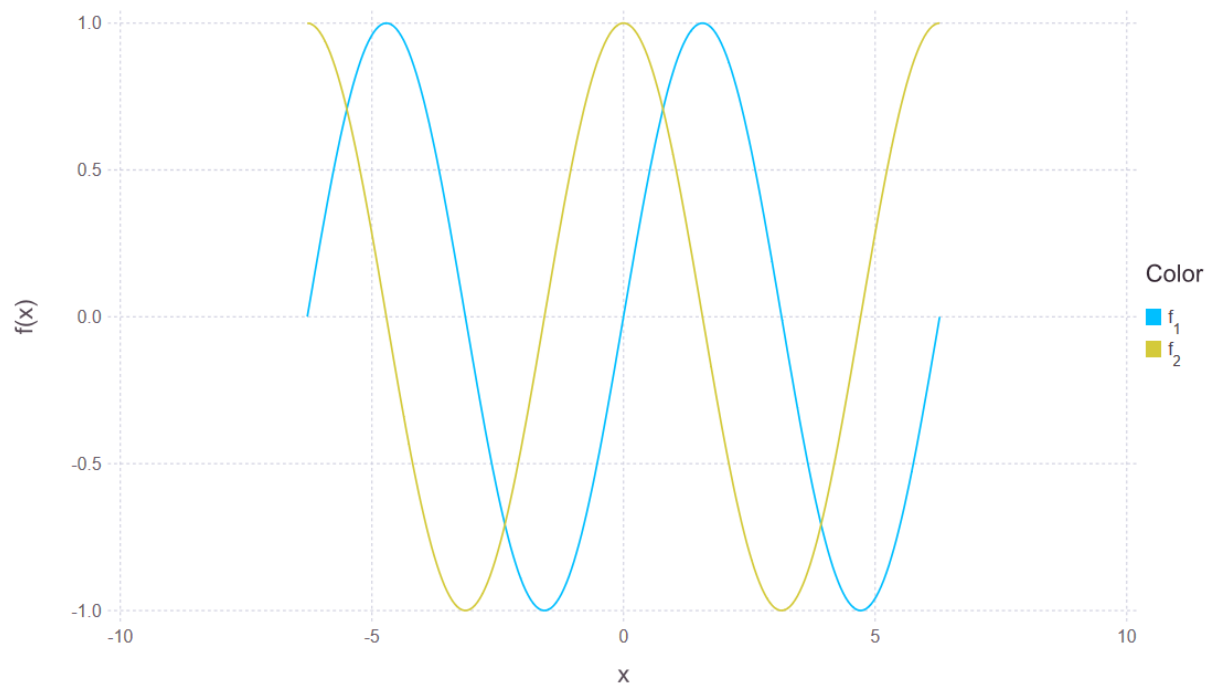
outstanding! Not very fine-tuned, but comprehensible and unbelievably easy to produce.

Also, many relations may be plotted together when presented inside the brackets and separated with comma :

[program listing of the bdeGadflyworking1c-4.jl](#)

using Gadfly

plot([sin,cos],-2 π ,2 π)



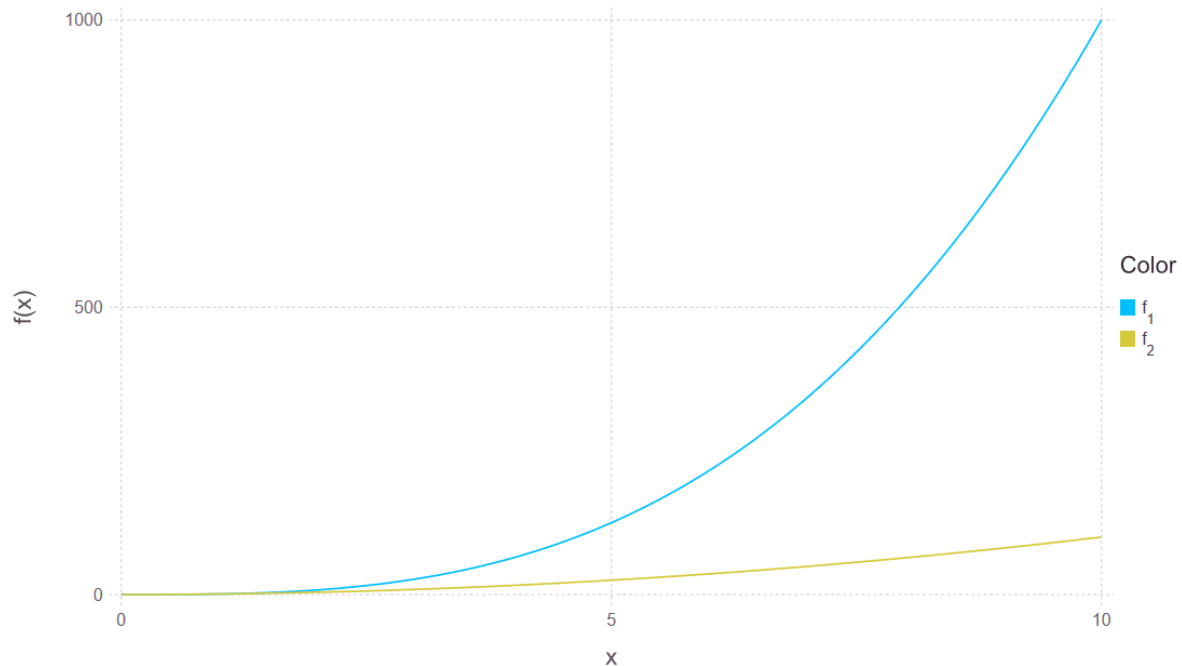
User functions may be plotted easily (but only for inspection, not for publication).

[program listing of the bdeGadflyworking1c-5.jl](#)

using Gadfly

```
function c3(x::Real)
    return x^3
end
```

```
plot([c3,x->x*x],0,10)
```



There are many more to try and see the results. But, the produced graphs are not publication grade for sure. They are only useful for inspection. Also, for multigraphs, the only functions whose ranges are “commensurable” may give a meaningful graphs when plotted together.

Anyhow, it is worth to follow the procedures that will be described soon in order to get meaningful, publication grade graphs. Let’s continue from the hard side.

Geom Object

point and line properties

We will trace the line joining the points. That will cost us only adding explicitly the geometry involved. Also there is a minor change in the plot() function, showing the power of the plot() function. The x values defined outside, are redefined within the plot() function.

Program Listing of the “bdeGadflyworking1d.jl” :

```
using Gadfly
```

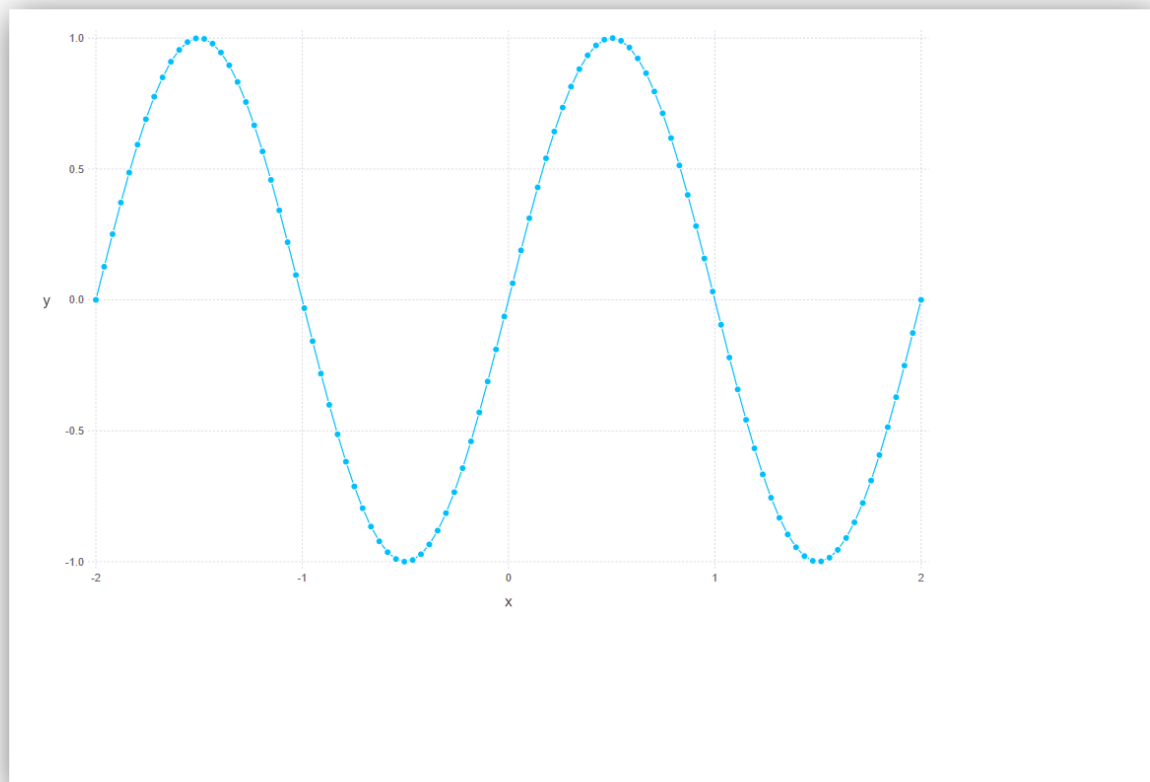
```
x=linspace(-2pi,2pi,100 )
```

```
y=sin.(x)
```

```
set_default_plot_size(24cm,16cm)
```

```
plot(x=x/pi , y=y , Geom.point, Geom.line)
```

```
include("D:\\bedri\\Dropbox\\bedriemir\\Julia\\PlottingExamples\\bde Gadfly  
working 1d.jl")
```



Actual plot is much bigger and following the plotted data is very comfortable.

Guide Object

xLabel and yLabel and title properties

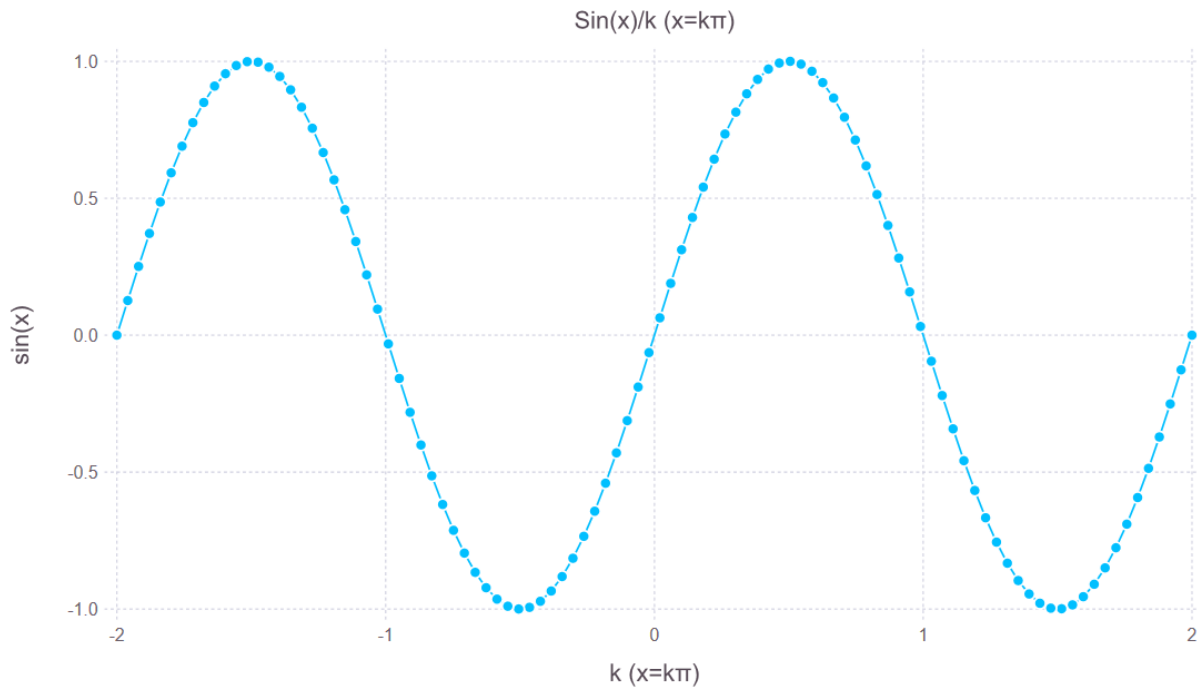
Guide object is provided for defining axes labels and plot title.

The following program has been written with Mathematica (10.1) which is a very good math editor.

Program Listing of the bdeGadflyworking1e.jl

(code environment Mathematica 10.1)

```
using Gadfly
x=linspace(-2π,2π,100)
y=sin.(x)
plot(x=x/π,
y=y,
Geom.point,
Geom.line,
Guide.xlabel("k (x=kπ)"),
Guide.ylabel("sin(x)"),
Guide.title("Sin(x)/k (x=kπ)"))
```



The produced graph is decorated following the directives of the user.

As it is directed from a program, the data points are joined with line segments and axes are labeled following the given definitions.

It is also possible to plot in vertical and horizontal labels with the axes.

[Program Listing of the bdeGadflyworking1e-1.jl](#) (tentative1f)

(code environment VSCode)

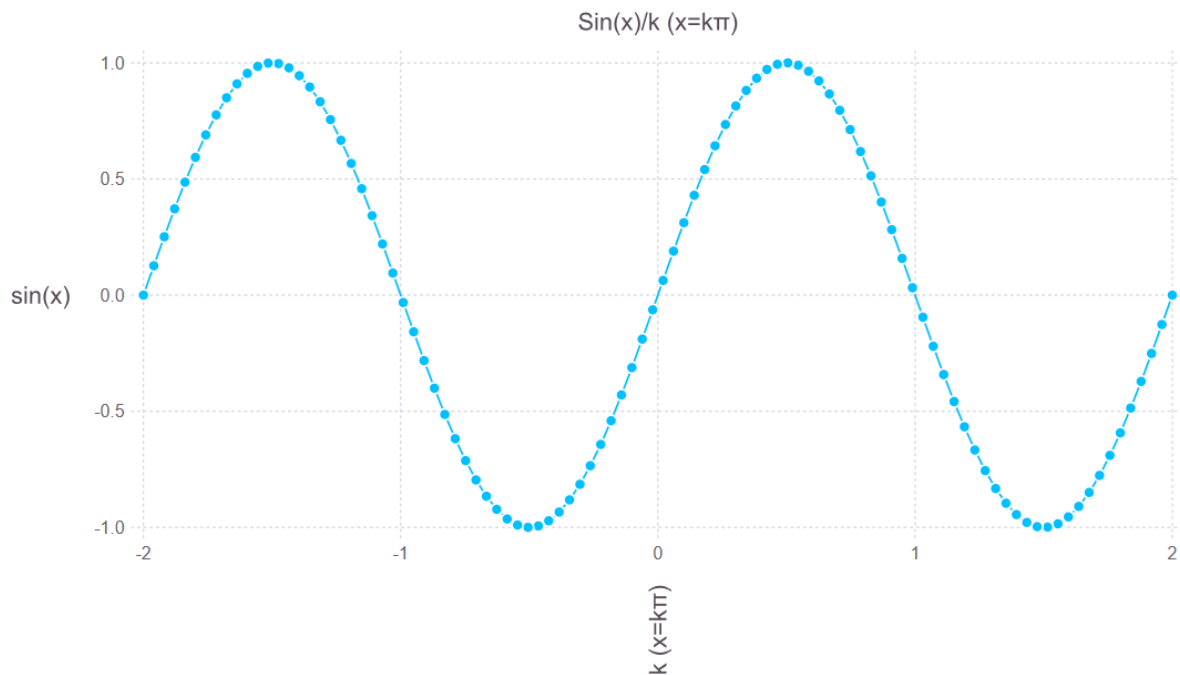
```
using Gadfly
x=linspace(-2π,2π,100)
y=sin.(x)
plot(x=x/π,
y=y,
Geom.point,
```

```

Geom.line,
Guide.xlabel("k (x=k $\pi$ )", orientation=:vertical),
Guide.ylabel("sin(x)", orientation=:horizontal),
Guide.title("Sin(x)/k (x=k $\pi$ )")

```

N.B. : Please notice =: in orientation=:vertical and orientation=:horizontal, this is indispensable.



Annotation property

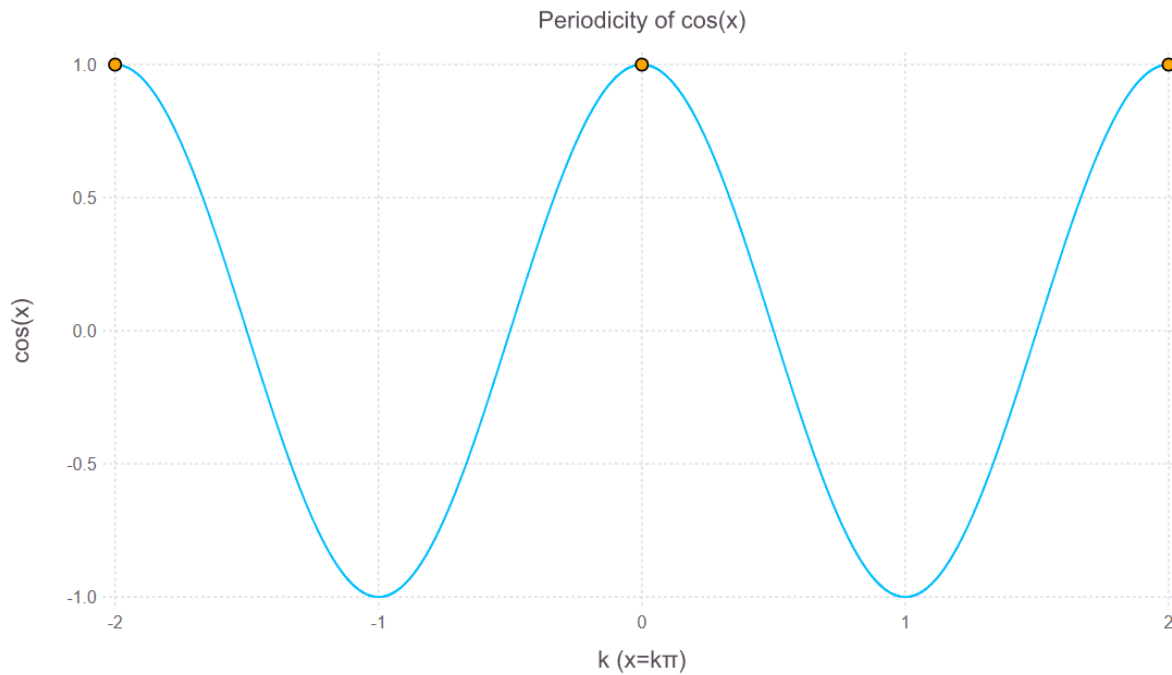
Apart from ggplot2 of R, Gadfly is also based on “Compose” graphics. One of the profits is inserting annotation objects in the tracée as shown below.

[bdeGadflyworking1g](#), program listing :

```

using Gadfly
using Compose
x=linspace(-2π,2π,10000)
y=cos.(x)
x=x/π
set_default_plot_size(20cm,12cm)
plot(x=x,y=y,Geom.line,
      Guide.annotation(
        compose(context(), circle([-2,0,2],[1,1,1],[1mm]),
          fill("orange"), stroke("black"))),
      Guide.title("Periodicity of cos(x)"),
      Guide.xlabel("k (x=kπ)"),
      Guide.ylabel("cos(x)"),
)

```



program listing `bdeGadflyworking1h.jl`:

```
using Gadfly
using Compose
x=linspace(-2π,2π,10000)
y=sin.(x)
x=x/π
set_default_plot_size(20cm,12cm)
plot(x=x,y=y,Geom.line,
      Guide.annotation(
        compose(context(), circle([-2,0,2],[0,0,0],[1mm]),
          fill("orange"), stroke("black"))),
```



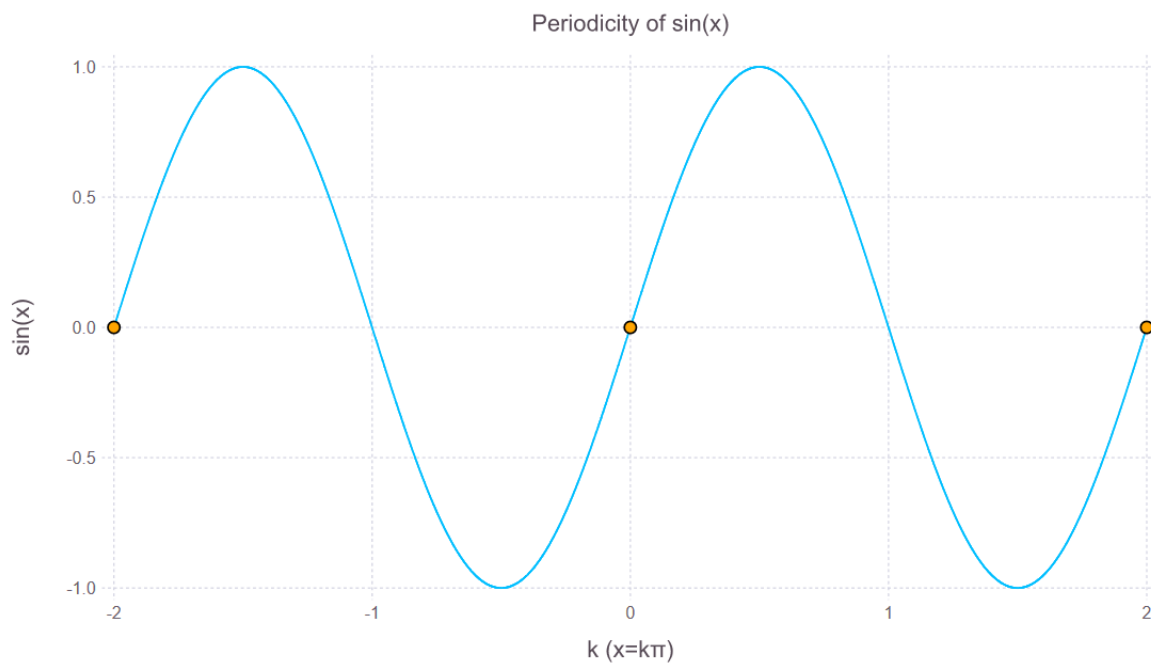
```

Guide.title("Periodicity of sin(x)",
Guide.xlabel("k (x=kπ)",
Guide.ylabel("sin(x)",
)

```

N.B.: `circle([x],[y],[1mm])` indicates a circle whose diameter is 1mm.

`circle([0,1,1], [1,0,1], [1mm])` draws a circle of diameter = 1mm, at coordinate points relevant to the axes $\{0,1\}$, $\{1,0\}$, $\{1,1\}$



xticks and yticks properties

Guide object has two useful properties, `xticks` and `yticks` for fine tuning of x and y axes.

Note that the Guide object is incapable of determining the portion of the domain and range of the function reflected to the produced plot.

Guide object knows about the portion of the domain $f(x)$ only by the declaration of the variation of the free variable x by the range function involved. For example, declaring $x = \text{linspace}(-2\pi, 2\pi, 10000)$ limits Julia for interesting only $[-2\pi, 2\pi]$ portion of the domain $f(x)$. The range of the function, reflected as the computer chosen grids for y axis for this specified portion of the function $f(x)$ is not preably known. Therefore since the range of the function for the specified portion of the domain is not exactly known before, y ticks range specified by the user would be only a void guess, without proper investigation of the function $f(x)$.

For better plots, we should wait until both axes will be strictly specified by Coord object that will be introduced soon. Then y ticks will be defined precisely over a well defined upper and lower limits over a closed interval of the range of $f(x)$.

Specifying the variation of x in the form of array declaration of the free variable in as $x = \text{linspace}(-2\pi, 2\pi, 100)$ had the strict meaning that only 100 values of the range of $f(x)$ between $[-2\pi, 2\pi]$ will be evaluated and mapped to y axis. The reflection to the y axis will be nothing other than the mapping of $[f(-2\pi), f(2\pi)]$ values over y axis. and these values are calculated from Julia as to be $[-2.44929e-16, 2.44929e-16]$. Julia crashes for tickling between these values so, we have found `Guide.yticks(ticks=[-8000:4000:8000;])` would be a good choice for this tentative plot.

This failure shows that it would be a good idea to fix the ticks before setting up the range limits. If the range upper and under limits is not specified, it would be better to leave Gadfly to determine the y ticks by its defaults.

Better in his way...

[program listing bdeGadflyworking1i.jl:](#)

[\(Code environment VSCode\)](#)

```
using Gadfly
```

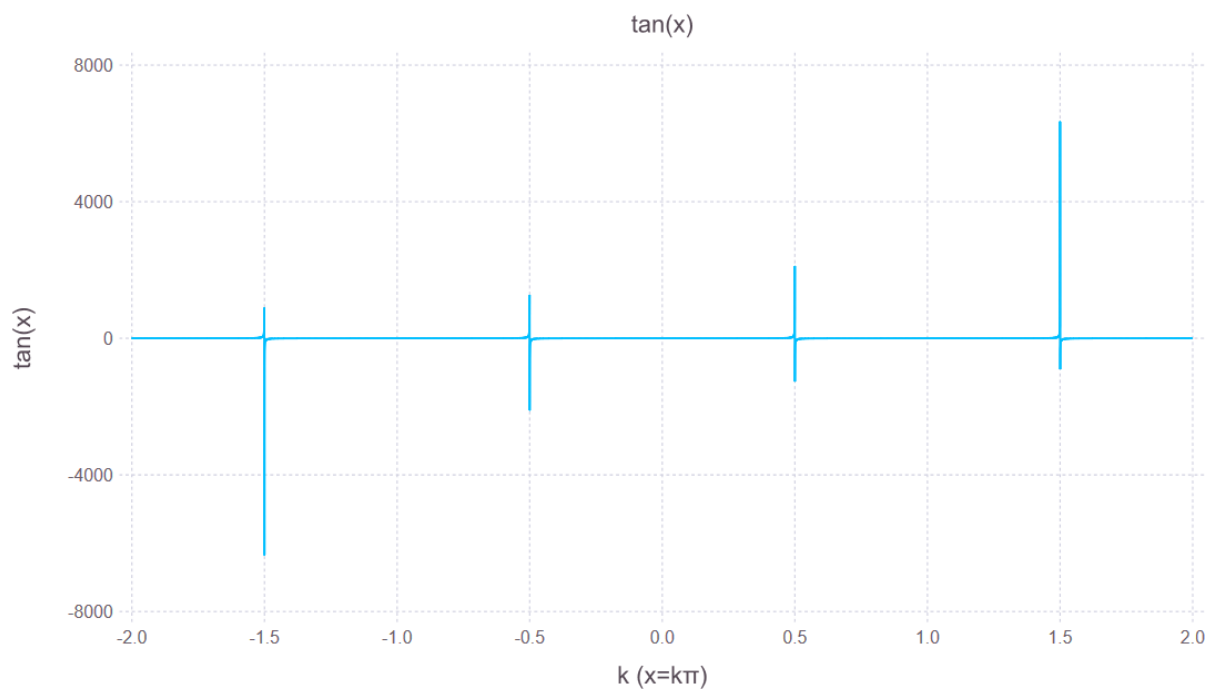
```
x=linspace(-2π,2π,10000)
```

```

y=tan.(x)
x=x/π
set_default_plot_size(20cm,12cm)
plot(x=x,y=y,Geom.line,
Geom.line,
Guide.title("tan(x)",
Guide.xlabel("k (x=kπ)",
Guide.ylabel("tan(x)",
Guide.xticks(ticks=[-2:0.5:2;]),
Guide.yticks(ticks=[-8000:4000:8000;]))

```

The produced plot is:



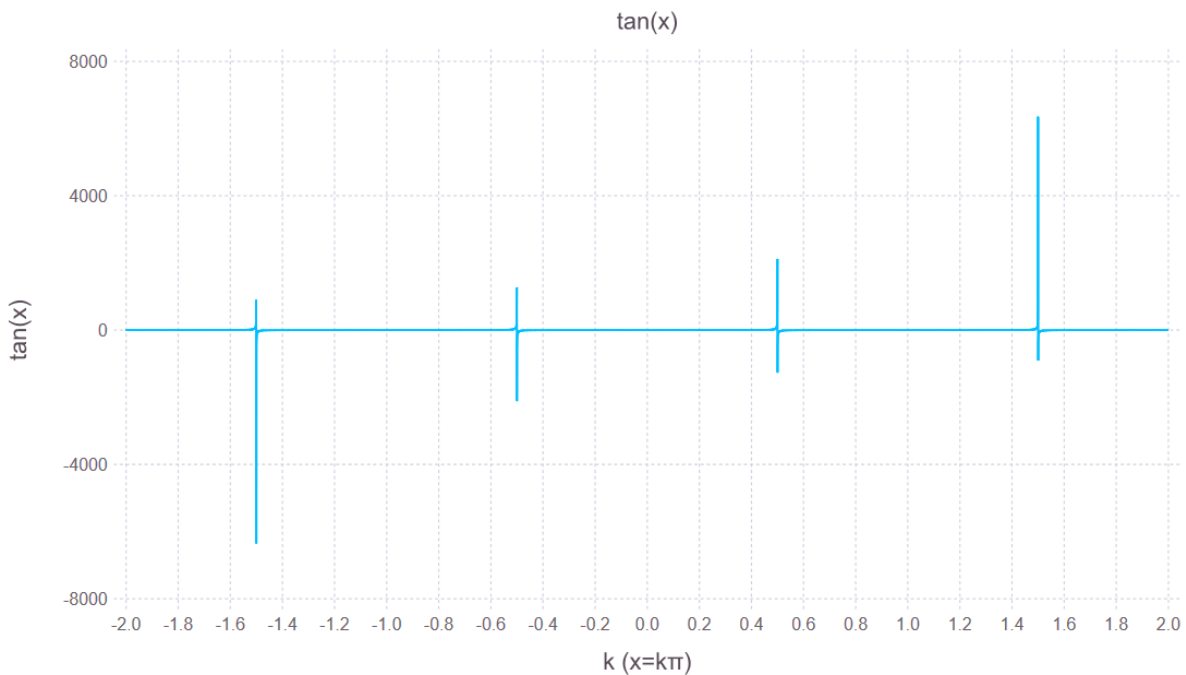
Since x and y ticks are describing as an array, it should be possible to apply all the array generating declarations but only collect() seems to be accepted.

[program listing bdeGadflyworking1j.jl](#):

(Code environment VSCode)

```
using Gadfly
x=linspace(-2π,2π,10000)
y=tan.(x)
x=x/π
set_default_plot_size(20cm,12cm)
plot(x=x,y=y,Geom.line,
Geom.line,
Guide.title("tan(x)"),
Guide.xlabel("x=kπ (k=-2,-1.8,-1.6,...,2)"),
Guide.ylabel("tan(x)"),
Guide.xticks(ticks=collect(-2:0.2:2;)),
Guide.yticks(ticks=[-8000:4000:8000;]))
```

The produced plot is:

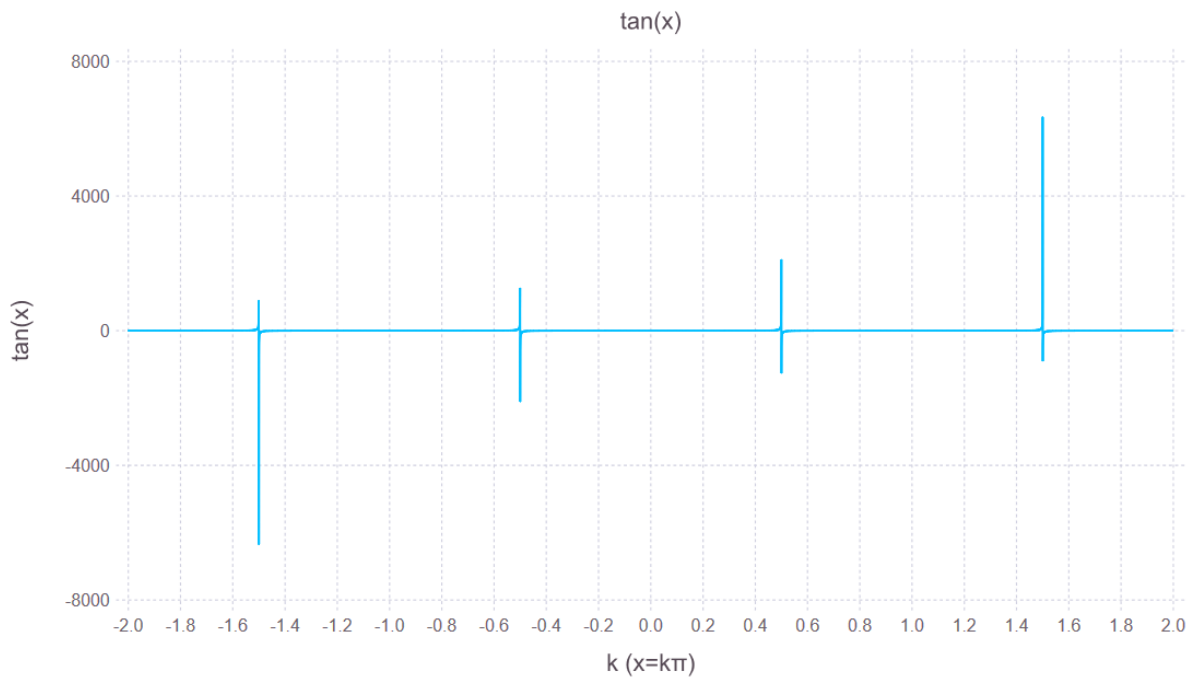


program listing [bdeGadflyworking1k.jl](#):

(Code environment VSCode)

```
using Gadfly
x=linspace(-2π,2π,10000)
y=tan.(x)
x=x/π
set_default_plot_size(20cm,12cm)
plot(x=x,y=y,Geom.line,
Geom.line,
Guide.title("tan(x)"),
Guide.xlabel("k (x=k π)"),
Guide.ylabel("tan(x)"),
Guide.xticks(ticks=collect(-2:0.2:2;)),
Guide.yticks(ticks=collect(-8000:4000:8000;))
)
```

Resulting plot :



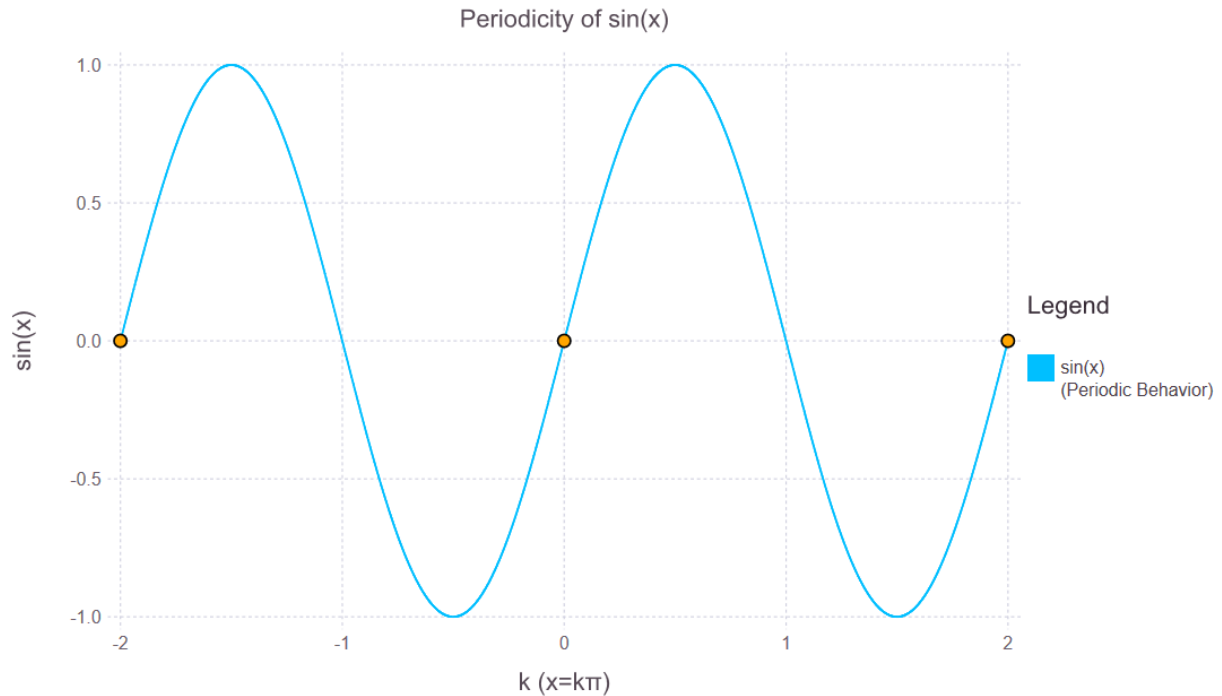
Note that, these last consecutive plots are the same.

manual_color_key property

Guide.manual_color_key is called for displaying a “legend” for the presentation of points or lines displayed in the plot. Generally it is applied for distinguishing various relations plotted together in the same plot and not necessary even superfluous for a single plot.

tentative1, program listing :

```
using Gadfly
using Compose
x=linspace(-2π,2π,10000)
y=sin.(x)
x=x/π
set_default_plot_size(20cm,12cm)
plot(x=x,y=y,Geom.line,
      Guide.annotation(
        compose(context(), circle([-2,0,2],[0,0,0],[1mm]),
          fill("orange"), stroke("black"))),
      Guide.title("Periodicity of sin(x)"),
      Guide.xlabel("k (x=k π)"),
      Guide.ylabel("sin(x)"),
      Guide.manual_color_key("Legend",["sin(x) \n (Periodic Behavior)",
        ["deepskyblue"]])
      )
```



We have applied the properties of Guide object which are most interesting for us. Now we will apply interesting properties of other objects.

Scales Object

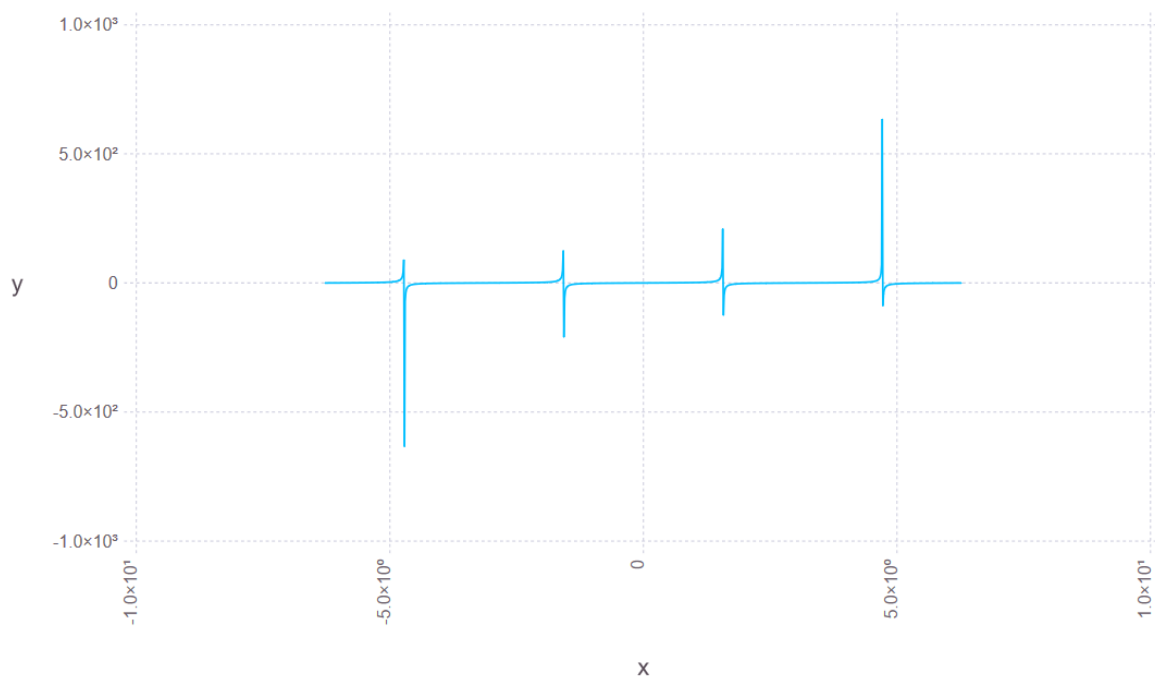
Apart from obtaining log, log-normal plots, Scales object has abilities to render as continuous or discrete scales.

The discrete scale is for the categorical data, mostly encountered in statistical works and not frequent in mathematics.

The continuous scale means that the raw data will stay as it is and no transformations will be made. This property is applied by default, therefore there is no need to state it explicitly. When we need some fine tuning, using this property, we can use them deliberately, as we can see in the program and graph below:

program: bdeGaflyworking2a.jl

```
using Gadfly
x=linspace(-2pi,2pi,1000;)
plot(x=x,y=tan.(x),Geom.line,
Scale.x_continuous(minvalue=-2pi, maxvalue=2pi,format=:scientific),
Scale.y_continuous(format=:scientific),
Guide.xticks(orientation=:vertical))
```



The min and max values declared in the prescription, is interpreted as not for absolutely following by Gadfly. The ranges are automatically calculated and the range portion request of the user with Scales object will be taken into consideration only when the situation would be favorable. Generally, plotting will be constructed according the internal decisions, rather than the user considerations. It will not the same with the coming **Coord** object however.

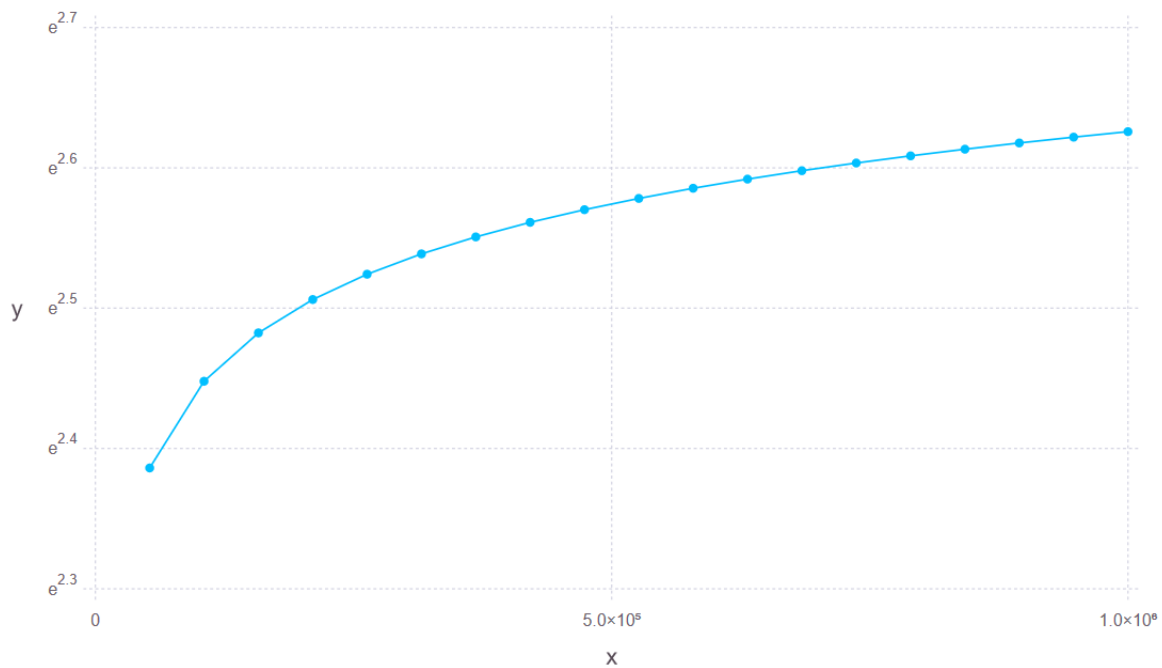
The log –normal and log-log plots may be generated using proper properties of the Scales object.

A log-normal plot is shown below :

program: bdeGaflyworking2b.jl

```
using Gadfly
x=linspace(0,10^6,20)
y=log.(x)
plot(x=x,y=y,
Geom.line, Geom.point,
Scale.y_log)
```

Displayed graph :



And a log-log plot :

program: bdeGaflyworking2c.jl

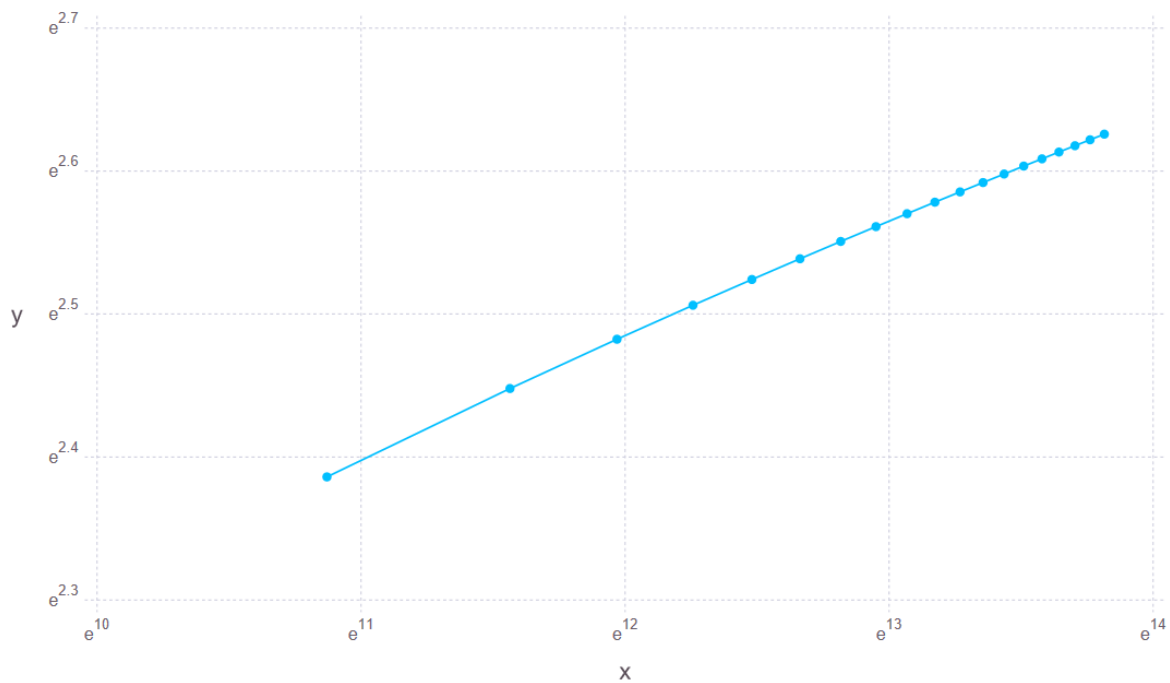
```
using Gadfly
x=linspace(0,10^6,20)
```

```

y=log.(x)
plot(x=x,y=y,
Geom.line,Geom.point,
Scale.x_log,
Scale.y_log)

```

The graph produced:



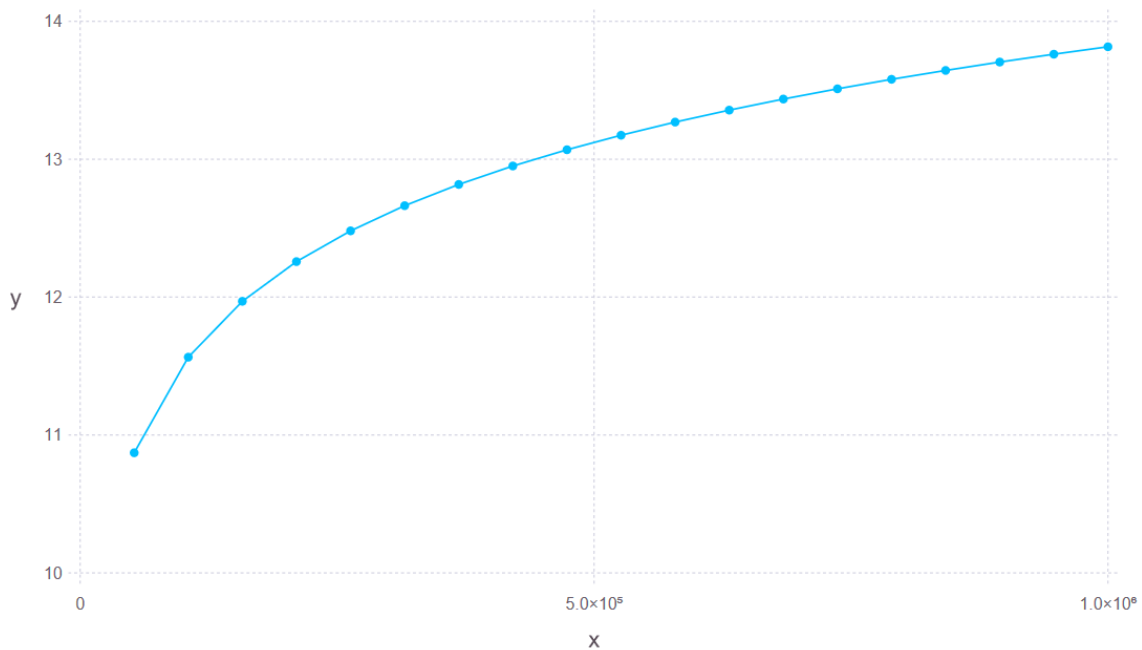
Normal plot :

program: [bdeGaflyworking2d.jl](#)

```

using Gadfly
x=linspace(0,10^6,20)
y=log.(x)
plot(x=x,y=y,
Geom.line,Geom.point)

```



Coord Object

Coord object is for hard setting the limits of axes. Gadfly will absolutely follow the specified hard limits.

Only available property is `Coord.cartesian` property whose most used arguments are:

- `xmin` (Hard minimum value on the x-axis),
- `ymin` (Hard minimum value on the y-axis),
- `xmax` (Hard maximum value on the x-axis),
- `ymax` (Hard maximum value on the y-axis).

Coordinate object fixes the values ultimately for both axes. These hard limits are+ followed absolutely from Gadfly.

A demonstrative plot is as follows :

tentative3a.jl

using Gadfly

```

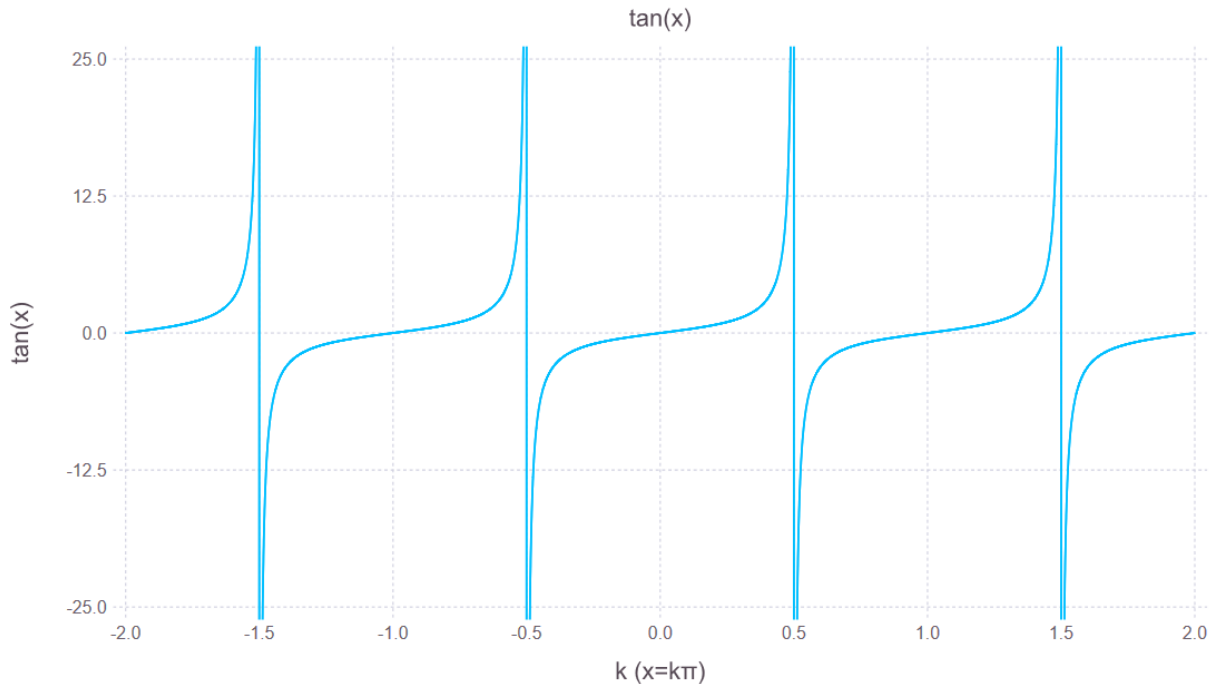
x=linspace(-2π,2π,10000)
y=tan.(x)
x=x/π
set_default_plot_size(20cm,12cm)
plot(x=x,y=y,Geom.line,
Geom.line,
Scale.x_continuous(minvalue=-2, maxvalue=2),

Coord.cartesian(xmin=-2, xmax=2, ymin=-25, ymax=25),

Guide.title("tan(x) / x/π"),
Guide.xlabel("k (x=kπ)",
Guide.ylabel("tan(x)",
Guide.xticks(ticks=[-2:0.5:2;]),
Guide.yticks(ticks=[-25:12.5:25;]))

```

the generated plot is:



At last, a meaningful plot, displaying what the user have prescribed.

Compare this plot with what had been produced in program `bdeGadfly` `working1i.jl`, the improvement is obvious.

We have applied most useful properties of the objects of the Aesthetics section.

We will now, study the Theme section as the next and colorful section.

The Theme Section

This section is for colors, line width, point size etc.

Parameters

These parameters can either be used with `Theme` or `style`

- `default_color`: When the color aesthetic is not bound, geometry uses this color for drawing. (Color)
- `point_size`: Size of points in the point, boxplot, and beeswarm geometries. (Measure)
- `point_size_min`: Minimum size of points in the point geometry. (Measure)
- `point_size_max`: Maximum size of points in the point geometry. (Measure)
- `point_shapes`: Shapes of points in the point geometry. (Function in circle, square, diamond, cross, xcross, utriangle, dtriangle, star1, star2, hexagon, octagon, hline, vline)
- `line_width`: Width of lines in the line geometry. (Measure)
- `line_style`: Style of lines in the line geometry. (Symbol in :solid, :dash, :dot, :dashdot, :dashdotdot, or Vector of Measures)
- `panel_fill`: Background color used in the main plot panel. (Color or Nothing)
- `panel_opacity`: Opacity of the plot background panel. (Float in [0.0, 1.0])
- `panel_stroke`: Border color of the main plot panel. (Color or Nothing)
- `background_color`: Background color for the entire plot. If nothing, no background. (Color or Nothing)
- `plot_padding`: How much padding should be put around the plot as a whole (Measure)
- `grid_color`: Color of grid lines. (Color or Nothing)
- `grid_color_focused`: In the D3 backend, mousing over the plot makes the grid lines emphasised by transitioning to this color. (Color or Nothing)
- `grid_line_width`: Width of grid lines. (Measure)
- `minor_label_font`: Font used for minor labels such as guide entries and labels. (String)
- `minor_label_font_size`: Font size used for minor labels. (Measure)
- `minor_label_color`: Color used for minor labels. (Color)
- `major_label_font`: Font used for major labels such as guide titles and axis labels. (String)
- `major_label_font_size`: Font size used for major labels. (Measure)
- `major_label_color`: Color used for major labels. (Color)
- `key_position`: Where key should be placed relative to the plot panel. One of :left, :right, :top, :bottom, or :none. Setting to :none disables the key. (Symbol)
- `key_title_font`: Font used for titles of keys. (String)
- `key_title_font_size`: Font size used for key titles. (Measure)
- `key_title_color`: Color used for key titles. (Color)
- `key_label_font`: Font used for key entry labels. (String)
- `key_label_font_size`: Font size used for key entry labels. (Measure)
- `key_label_color`: Color used for key entry labels. (Color)
- `key_max_columns`: Maximum number of columns for key entry labels. (Int)
- `bar_spacing`: Spacing between bars in [Geom.bar](#). (Measure)
- `boxplot_spacing`: Spacing between boxplots in [Geom.boxplot](#). (Measure)
- `errorbar_cap_length`: Length of caps on error bars. (Measure)

- `highlight_width`: Width of lines drawn around plot geometry like points, and boxplot rectangles. (Measure)
- `discrete_highlight_color` and `continuous_highlight_color`: Color used to outline plot geometry. This is a function that alters (e.g. darkens) the fill color of the geometry. (Function)
- `lowlight_color`: Color used to draw background geometry, such as `Geom.ribbon`. This is a function that alters the fill color of the geometry. (Function)
- `lowlight_opacity`: Opacity of background geometry such as [Geom.ribbon](#). (Float64)
- `middle_color`: Color altering function used to draw the midline in boxplots. (Function)
- `middle_width`: Width of the middle line in boxplots. (Measure)
- `guide_title_position`: One of `:left`, `:center`, `:right` indicating the placement of the title of color key guides. (Symbol)
- `colorkey_swatch_shape`: The shape used in color swatches in the color key guide. Either `:circle` or `:square` (Symbol)
- `bar_highlight`: Color used to stroke bars in bar plots. If a function is given, it's used to transform the fill color of the bars to obtain a stroke color. (Function, Color, or Nothing)
- `discrete_color_scheme`:
A `DiscreteColorScale` see [Scale.color_discrete_hue](#)
- `continuous_color_scheme`:
A `ContinuousColorScale` see [Scale.color_continuous](#)

Normally, very few of these parameters are needed even for publication quality plots. But, for a meticulous work, any of these parameters may be addressed.

Let us have a colored plot :

[Program listing of "bdeGadflyworking4a.jl"](#)

(Written in the notebook environment, tested the execution in the notebook environment, copied to VSCode, executed in REPL as Julia script.)

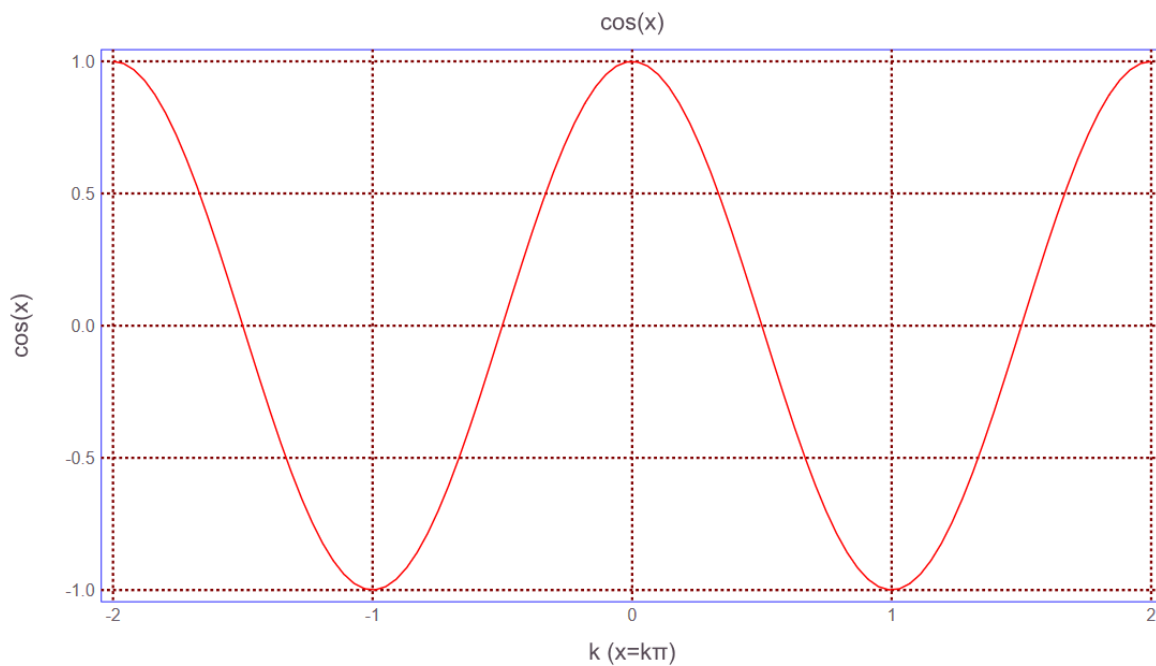
using Gadfly

```
x=linspace(-2π , 2π ,100);
```

```

y = cos.(x);
x=x/π;
plot(x=x,y=y,
Geom.line,
Guide.xlabel("k (x=kπ)"),
Guide.ylabel("cos(x)"),
Guide.title("cos(x)"),
Theme(default_color=colorant"red",
line_width=1px,
panel_stroke="blue",
grid_color="maroon",
grid_line_width=1.5px)
)

```



Colorful graphs are more expressive than their black & white counterparts.

Note that, in this case the default ticks choice of the Gadfly was adequate, therefore we have not attempted to alter them.

The Layer Section

The Layer section is for arranging the show of many relations in the same plot.

Each relation is called a “Layer” and Gadfly supports the superimposition of many “Layers” in the same plot.

Note that the relations will share the same coordinate axes and in order to obtain a meaningful display, the domains and the ranges of all the layers should be “commensurate”, i.e. for a measurable variation of x , they should all give a response detectable by eyes.

For example, let us plot sine and cosine curves together.

program listing of `bdeGadflyworking5a.jl` :

```
using Gadfly
x=linspace(-2π,2π,100)
plot(
    layer(
        x=x, y=sin.(x), Geom.point
    ),
    layer(
        x=x, y=cos.(x), Geom.line
    )
)
```

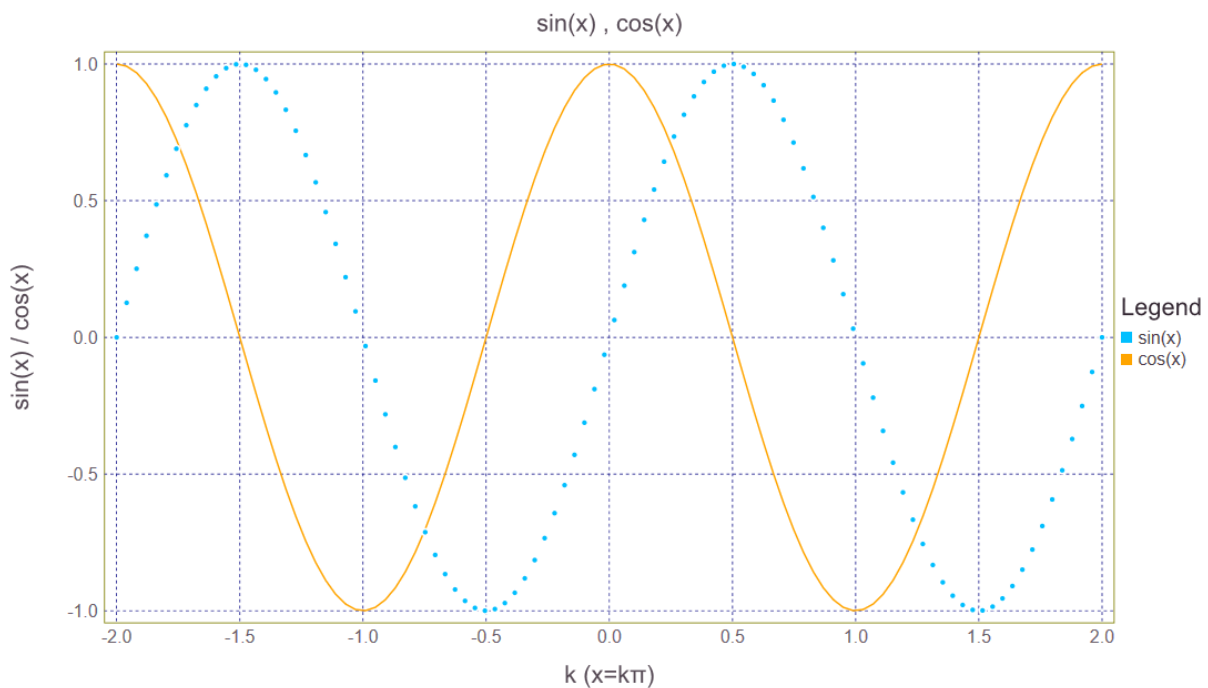


```

    )
plot(
  relation1,
  relation2,
  Guide.manual_color_key("Legend",["sin(x)", "cos(x)"],
    ["deepskyblue", "orange"]),
  Guide.title("sin(x) , cos(x)"),
  Guide.xlabel("k (x=kπ)"),
  Guide.ylabel("sin(x) / cos(x)"),
  Guide.xticks(ticks=collect(-2:0.5:2;)),
  Guide.yticks(ticks=collect(-1:0.5:1;)),
  Theme(
    panel_stroke="olive",
    grid_color="navy",
    grid_line_width=0.5px
  )
)

```

The resulting plot :



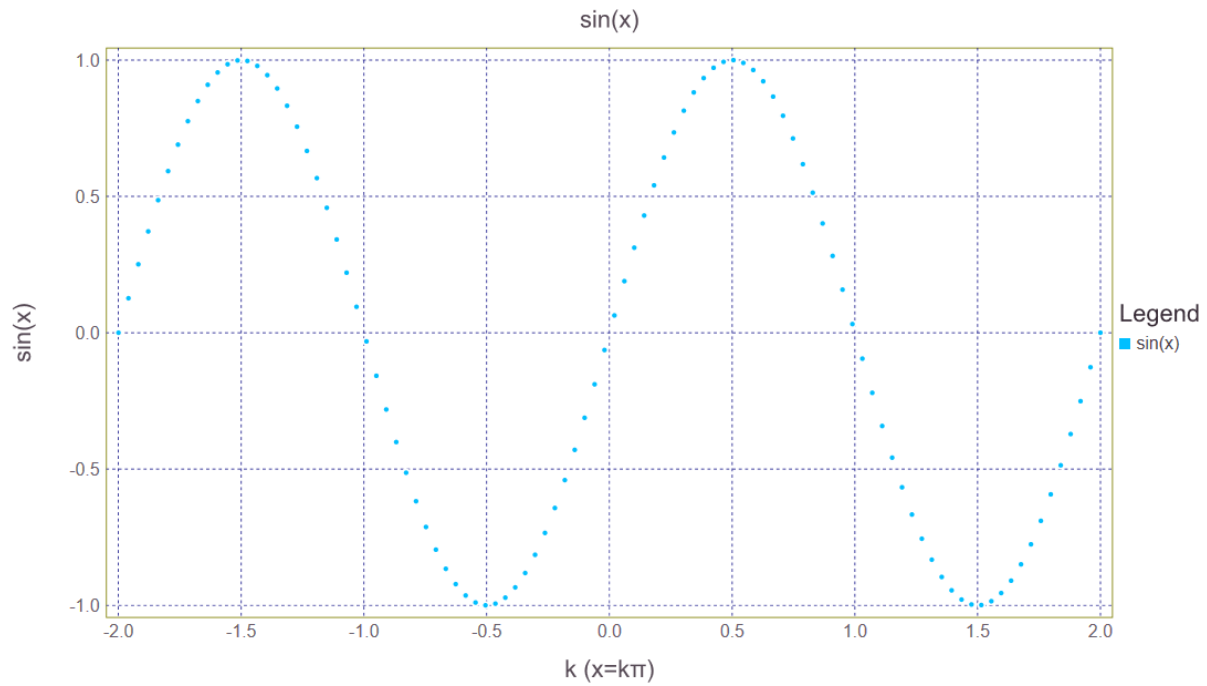
We can also plot layers one by one, independently from others :

program listing of bdeGadflyworking5b-1.jl :

```
using Gadfly
x=linspace(-2π,2π,100;)
relation1=layer( x=x, y=sin.(x),x=x/π,
                Geom.point,

                Theme(
                    default_color=colorant"deepskyblue",
                    point_size=2px
                )
            )
relation2=layer( x=x, y=cos.(x),x=x/π,
                Geom.line,
                Theme(
                    default_color=colorant"orange",
                    line_width=1px
                )
            )
plot(
    relation1,
    Guide.manual_color_key("Legend",["sin(x)"],
        ["deepskyblue"]),
    Guide.title("sin(x)"),
    Guide.xlabel("k (x=kπ)"),
    Guide.ylabel("sin(x)"),
    Guide.xticks(ticks=collect(-2:0.5:2;)),
    Guide.yticks(ticks=collect(-1:0.5:1;)),
    Theme(
        panel_stroke="olive",
        grid_color="navy",
        grid_line_width=0.5px
    )
)
```

Resulting plot:



program listing of bdeGadflyworking5b-2.jl :

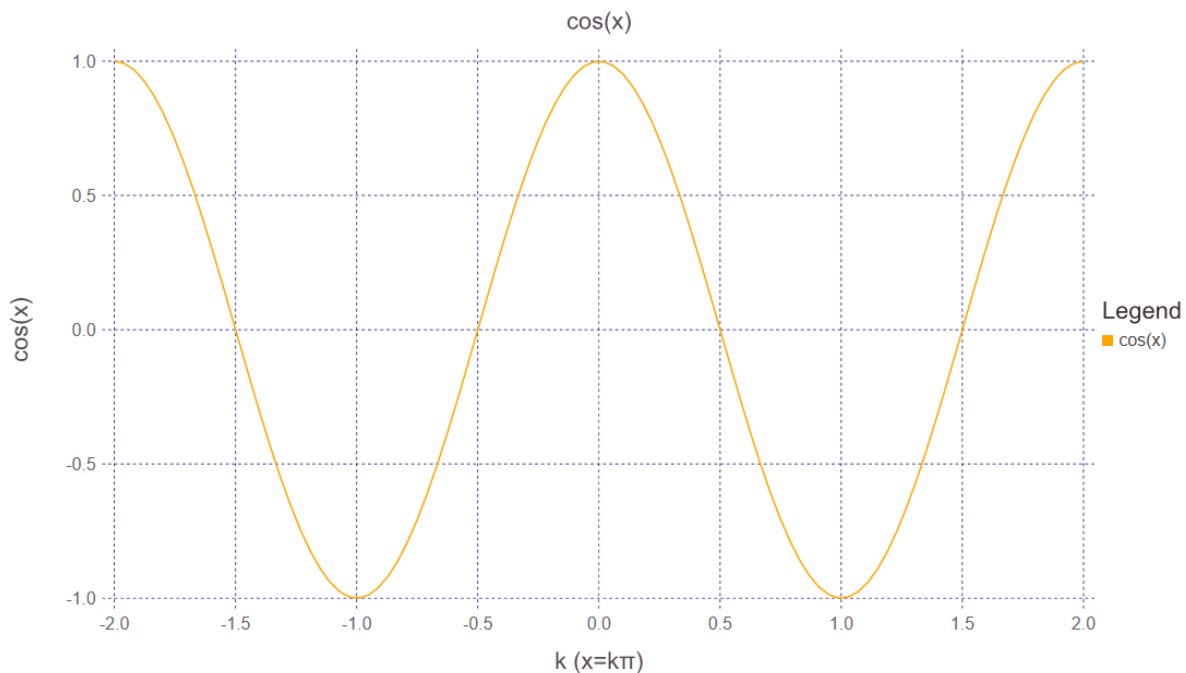
```
using Gadfly
x=linspace(-2π,2π,100;)
relation1=layer( x=x, y=sin.(x),x=x/π,
                Geom.point,

                Theme(
                    default_color=colorant"deepskyblue",
                    point_size=2px
                )
)
relation2=layer( x=x, y=cos.(x),x=x/π,
                Geom.line,
                Theme(
                    default_color=colorant"orange",
                    line_width=1px
                )
)
plot(
    relation2,
    Guide.manual_color_key("Legend",["cos(x)"],
```

```

    ["orange"]),
    Guide.title("cos(x)",
    Guide.xlabel("k (x=kπ)",
    Guide.ylabel("cos(x)",
    Guide.xticks(ticks=collect(-2:0.5:2;)),
    Guide.yticks(ticks=collect(-1:0.5:1;)),
    Theme(
        #panel_stroke="olive",
        grid_color="navy",
        grid_line_width=0.5px
    )
)

```



We are concluding here, but there is many more to discover.

For the material presented here, new examples should be produced, using all kinds of relations and all kinds of presentation tools found in the given examples.

Please e-mail your discoveries to bedri@bedriemir.com. Your examples will be included in the next release of this manuscript, by your name and information suggested by you.

In all the cases, I strongly suggest for the moment, that Julia programs are developed and executed in notebook environment, as notebook versions with extension *.ipynb. (ipynb = IPython notebook, showing the invisible tight liason of Julia with Python).

In notebook environment, development of Julia programs are more easier, program execution is seemingly faster and one gets program results with plots (if there is any) without hassle.

For the time being, Julia scripts may be developed in VS Code, Sub lime Text and Juno environment.

The best is VSCode (Visual Studio Code), this is a new Microsoft Code Scripting Environment.

Sublime text also works good after a good implementation (Read well the manual, this is also for VSCode and Juno) .

Juno may only be accessed with JuliaPro, although very well designed, craches very frequently. Therefore, it not advised to use it for now, until a sound upgrade will be made.

For the moment, it will be wise to run Julia scripts (the ones with *.jl extension) in REPL platform. REPL works well.

Gadfly gives outstanding results. But his learning curve is far being a steep one. I hope these examples will be helpful.

After this publication, any plot within the auspices of this text, will be extremely easy, using open source Julia. Just follow the examples, be familiar with them and this will facilitate to use them in any scientific publication.

Have a wonderful programming time with Julia Programming Language.

(Sorry ! If you don't speak English, you are not from our Galaxy !)

B.D. Emir

<http://www.bedriemir.com>

bedri@bedriemir.com

Running programs (scripts) with *.jl extension :

They are basically the same programs and for whose developed in notebook environment and saved as *.ipynb, changing the extension from *.ipynb to *.jl will suffice.

Program lines terminating with ; are not relevant in julia scripts (the ones that have *.jl extensions). These terminal semicolons (;) are only significant in notebook programs (the ones that have *.ipynb extensions). Terminating semicolons (;) only avoids showing the outputs of individual cells in Julia notebooks. They are useless otherwise.

Suppose we have Julia script test.jl, stored in e:\dynamic_content\Julia Programs\test.jl

We can then execute this program using either in Julia desktop (REPL) or Visual Studio Code (free) or theoretically with Sublime Text or Juno (based on Atom editor, if we have had build our Julia infrastructure, using free version of Juliapro).

Note that for the time being, Visual Studio Code is very solid and easy to use, but incapable of showing plots. This will not be a problem, since we can run all the programs in REPL environment.

Actually VSCode even latest version has a very annoying annotation which means nothing in fact. When clicked to “execute code” button  then sometimes, an annotation :



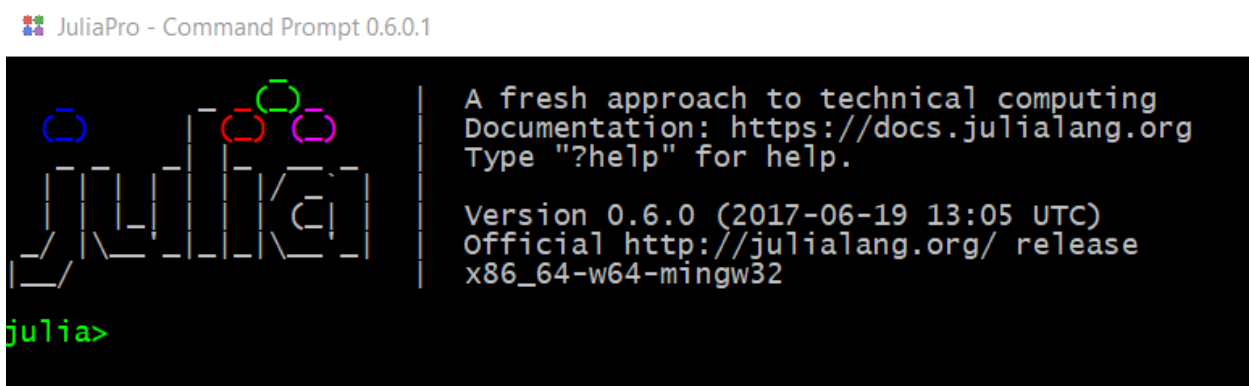
is opened. That should not intimidate us, just press Close and run the script again by pressing to run button. Your script will surely run.

Şto éto c delayete :

Open system from control panel. Choose advanced settings, press environmental variables. Create a new environmental variable **JULIA_HOME** pointing to the bin folder where resides julia.exe. Insert this to the environmental system and call REPL by:

>julia ← ENTER

then comes REPL (Read Execute Print Loop)



enter :

julia> cd ("e:\\dynamic_content\\Julia Programs") ← ENTER

julia>PWD() ← ENTER Note : (PWD = Print Working Directory)

julia>"e:\\dynamic_content\\Julia Programs"

now we are at the same directory where test.jl resides, we can call and execute test.jl as:

julia>include("test.jl") ← ENTER

then test.jl is executed and the results printed in REPL. If there is a plot it is opened in a separate directory. No other plot may be generated without closing the opened plot window.

A screenshot of the JuliaPro Command Prompt interface. The title bar reads "JuliaPro - Command Prompt 0.6.0.1". The main window has a dark background. On the left, there is a stylized logo for Julia, composed of various colored brackets and lines. On the right, white text provides information: "A fresh approach to technical computing", "Documentation: <https://docs.julialang.org>", "Type '?help' for help.", "Version 0.6.0 (2017-06-19 13:05 UTC)", "Official <http://julialang.org/> release", and "x86_64-w64-mingw32". Below this, the command prompt shows a series of commands entered in green text: "julia> cd(\"e:\\dynamic_content\\Julia Programs\")", "julia> pwd()", which returns "\"e:\\dynamic_content\\Julia Programs\"", and "julia> include(\"test.jl\")".

Using Juno would be much more plausible, but unfortunately, Juno has not enough stability yet.

Visual Studio Code is more or less the same with Juno, more stable, but can not generate plots for the present version. If we have a program which does not plot anything, it might be very easy to run this Julia script (the one that has .js extension) in the VS Code environment.

Shown below, is the dialog for "bde PlotlyJS 5b.jl" saved in same folder.

```
plotlyjs()
```

```
x=linspace(0,2,20)
```

$$y=x.^2$$

```
plot([x y],
```

```
markershape=[:circle :rect],
```

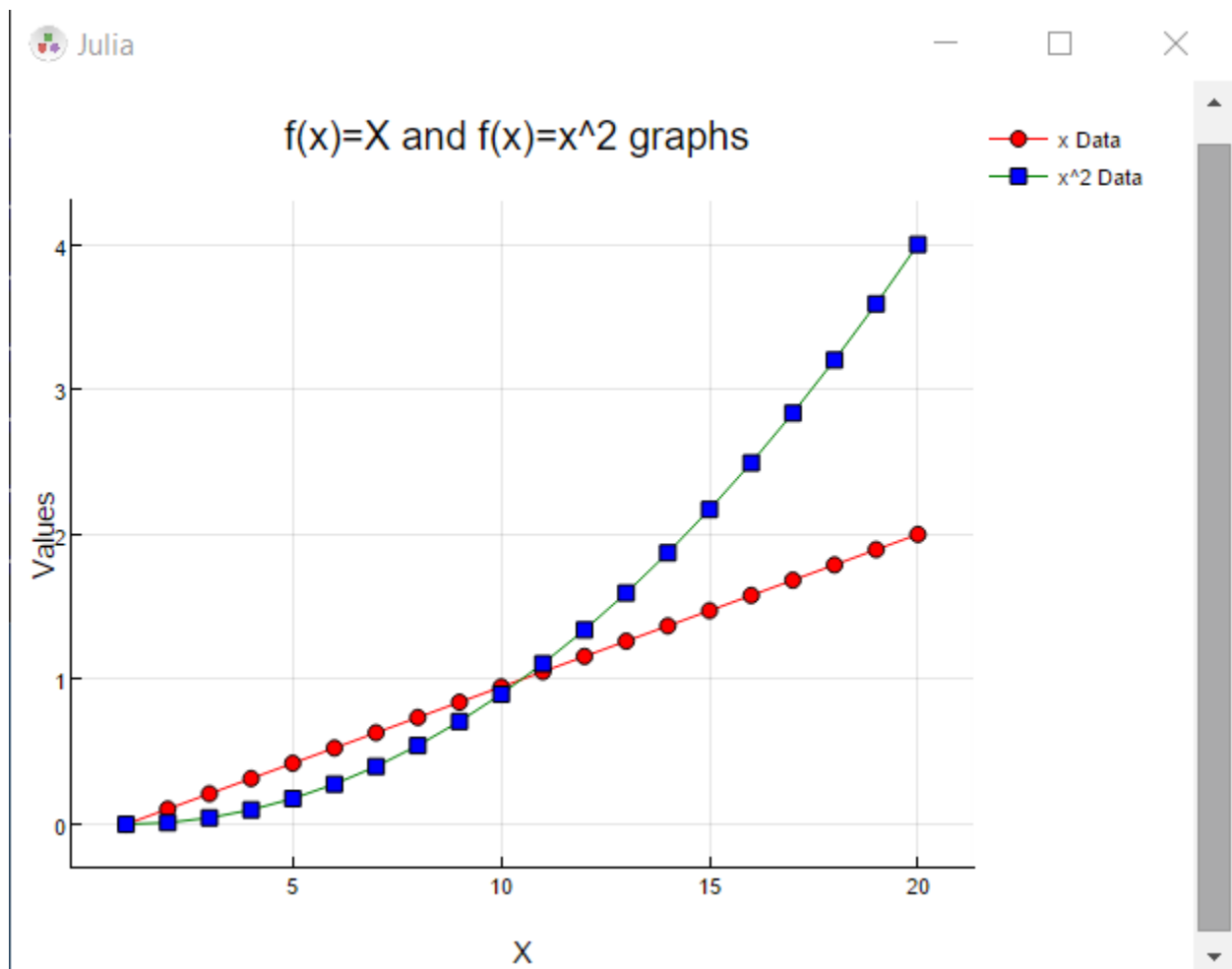
```
markercolors=[:red :blue],
```

```
labels=["x Data" "x^2 Data"],
```

```
title = "f(x)=X and f(x)=x^2 graphs\n",
```

```
top_margin=25px,  
ylabel = "Values\n",  
xlabel = "X",  
linecolors=[:red :green])
```

Opened graph window from REPL :



Related Programs:

[bde PlotlyJS working 1a.jl](#)

[bde PlotlyJS working 1b.jl](#)

[bde PlotlyJS working 1c.jl](#)

[bde plotlyJS working 1d.jl](#)

[bde PlotlyJS working 2a.jl](#)

[bde PlotlyJS working 2b.jl](#)

[bde PlotlyJS working 3a.jl](#)

[bde PlotlyJS working 3b.jl](#)

[bde PlotlyJS working 3c.jl](#)

[bde PlotlyJS working 3d.jl](#)

[bde PlotlyJS working 3e.jl](#)

[bde PlotlyJS working 4a.jl](#)

[bde PlotlyJS working 4b.jl](#)

[bde PlotlyJS working 4c.jl](#)

[bde PlotlyJS working 4d.jl](#)

[bde PlotlyJS working 4e.jl](#)

[bde PlotlyJS working 4f.jl](#)

[bde PlotlyJS working 5a.jl](#)

[bde PlotlyJS working 5b.jl](#)

[bde PlotlyJS working most simpler 5c.jl](#)

[bde PlotlyJS working most compact 5d.jl](#)

[bde Pyplot working 1a.jl](#)

[bde Pyplot working 1b.jl](#)

[bde Pyplot working 1c.jl](#)

[bde Pyplot working 1d.jl](#)

[bde Pyplot working 2a.jl](#)

[bde Pyplot working 2b.jl](#)

[bde Pyplot working 3a.jl](#)

[bde Pyplot working 3b.jl](#)

[bde Pyplot working 3c.jl](#)

[bde Pyplot working 3d.jl](#)

[bde Pyplot working 3e.jl](#)

[bde PlotlyJS working 4a.jl](#)

[bde Pyplot working 4b.jl](#)

[bde Pyplot working 4c.jl](#)

[bde Pyplot working 4d.jl](#)

[bde PlotlyJS working 4e.jl](#)

[bde Pyplot working 4f.jl](#)

[bde Pyplot working 5a.jl](#)

[bde Pyplot working 5b.jl](#)

[bde Pyplot working most simpler 5c.jl](#)

[bde Pyplot working most compact 5d.jl](#)

[bdeGadflyworking1a.jl](#)

[bdeGadflyworking1b.jl](#)

[bdeGadflyworking1c](#)

[bdeGadflyworking1c-1.jl](#)

[bdeGadflyworking1c-2.jl](#)

[bdeGadflyworking1c-3.jl](#)

[bdeGadflyworking1c-4.jl](#)

[bdeGadflyworking1c-5.jl](#)

[bdeGadflyworking.1d](#)

[bdeGadflyworking.1e](#)

[bdeGadflyworking.1f](#)

[bdeGadflyworking.1g](#)

[bdeGadflyworking1h.jl](#)

[bdeGadflyworking1i.jl](#)

[bdeGadflyworking1j.jl](#)

[bdeGadflyworking1k.jl](#)

[bdeGadflyworking1l.jl](#)

[bdeGadflyworking2a.jl](#)

[bdeGadflyworking2b.jl](#)

[bdeGadflyworking2c.jl](#)

[bdeGadflyworking3a.jl](#)

[bdeGadflyworking4a.jl](#)

[bdeGadflyworking5a.jl](#)

[bdeGadflyworking5b-1.jl](#)

[bdeGadflyworking5b-2.jl](#)

